

APEX INSTITUTE OF TECHNOLOGY

Department of Computer Science & Engineering

MAJOR PROJECT TECHNICAL DOCUMENTATION

EduGuard AI

AI-Powered Student Performance Prediction & Early Warning System

APPENDIX A — SOURCE CODE

Complete Production Source Code, Neural Network Implementations & REST Service Endpoints

Project Domain:	Artificial Intelligence / Educational Data Mining
Architecture:	Next.js 14 (App Router), TypeScript Express Microservice, Python FastAPI Deep Learning Service, MongoDB Atlas
Model Topology:	Artificial Neural Network (ANN) Regression (Input 6 → Dense 64 → Dense 32 → Dense 16 → Linear 1)
Target Evaluation:	Final Academic Examination Score ($R^2 \geq 0.94$, $\text{MAE} \approx 2.45$, $\text{RMSE} \approx 3.12$)
Total Verified Source Files:	32 Production Source Files (100% Complete & Unmodified)
Academic Year:	2025 – 2026

APPENDIX A — TABLE OF CONTENTS

A.1 Frontend Source Code

- A.1.1 Home / Dashboard (apps/web/src/app/page.tsx, apps/web/src/app/dashboard/page.tsx, apps/web/src/components/dashboard/AttentionRequiredTable.tsx)
- A.1.2 Student Management (apps/web/src/app/students/page.tsx)
- A.1.3 Student Profile (apps/web/src/app/students/[id]/page.tsx)
- A.1.4 Attendance Management (apps/web/src/app/attendance/page.tsx)
- A.1.5 AI Prediction / Early Warning (apps/web/src/app/early-warnings/page.tsx, apps/web/src/components/students/PredictionRunModal.tsx)
- A.1.6 Teacher Dashboard (apps/web/src/app/dashboard/page.tsx — Teacher & Admin View)

A.2 Backend Source Code

- A.2.1 Server/Application (server.ts, app.ts, environment.ts, database.ts)
- A.2.2 Student API (studentRoutes.ts, studentController.ts, Student.ts)
- A.2.3 Attendance API (attendanceRoutes.ts, attendanceController.ts, AttendanceRecord.ts, studentMetricService.ts)
- A.2.4 Prediction API (predictionRoutes.ts, predictionController.ts, Prediction.ts)
- A.2.5 Risk and Recommendation Logic (riskFactorService.ts, recommendationService.ts)
- A.2.6 Intervention API (interventionRoutes.ts, interventionController.ts, Intervention.ts)
- A.2.7 ML Client Service (mlClientService.ts)

A.3 Machine Learning Source Code

- A.3.1 Data Preprocessing (generate_dataset.py)
- A.3.2 ANN Model (train.py — Model Architecture)
- A.3.3 Training (train.py — Training Pipeline & Optimization)
- A.3.4 Evaluation (train.py — Metric Validation)
- A.3.5 Inference (predictor.py — Production Predictor Engine)
- A.3.6 FastAPI ML Service (main.py — High-Performance API)

A.4 API Endpoint Summary

A.1 Frontend Source Code

A.1.1 Home / Dashboard

Root Application Router & Session Gate

File: apps/web/src/app/page.tsx

TYPESCRIPT

Purpose & Architecture: Directs users to /dashboard if an active session is hydrated, or redirects to /login for unauthenticated users.

```
'use client';

import { useEffect } from 'react';
import { useRouter } from 'next/navigation';
import { useAuth } from '@context/AuthContext';

export default function Home() {
  const { user, isLoading } = useAuth();
  const router = useRouter();

  useEffect(() => {
    if (!isLoading) {
      if (user) {
        router.push('/dashboard');
      } else {
        router.push('/login');
      }
    }
  }, [user, isLoading, router]);

  return (
    <div className="flex h-screen w-screen items-center justify-center bg-slate-50">
      <div className="h-8 w-8 animate-spin rounded-full border-4 border-solid border-blue-600 border-r-transparent"></div>
    </div>
  );
}
```

Main Dashboard & Student Academic Dashboard

File: apps/web/src/app/dashboard/page.tsx

TYPESCRIPT

Purpose & Architecture: Implements the institutional executive dashboard for faculty and personalized performance overview for students. Features cohort filtering, expected score calculation, real-time risk gauges, metric breakdowns, and suggested pedagogical actions.

```
'use client';

import React, { useEffect, useState } from 'react';
import Link from 'next/link';
import { AppLayout } from '@components/layout/AppLayout';
import { AttentionRequiredTable } from '@components/dashboard/AttentionRequiredTable';
import { dashboardApi, studentsApi, programsApi, departmentsApi } from '@lib/apiClient';
import { DashboardSummaryDTO } from '@eduguard/shared';
import { LoadingSkeleton } from '@components/ui/LoadingSkeleton';
import {
  RefreshCw,
  ArrowRight,
  CalendarCheck,
  Award,
  FileCheck,
  Clock,
  Activity,
  GraduationCap,
  Filter,
  Sparkles,
  Info,
} from 'lucide-react';
```

```

import { useAuth } from '@context/AuthContext';

export default function DashboardPage() {
  const { user, isStudent } = useAuth();
  const [data, setData] = useState<DashboardSummaryDTO | null>(null);
  const [studentData, setStudentData] = useState<any | null>(null);
  const [loading, setLoading] = useState<boolean>(true);
  const [error, setError] = useState<string | null>(null);

  // Cohort filters (Requirement 18)
  const [programs, setPrograms] = useState<any[]>([]);
  const [departments, setDepartments] = useState<any[]>([]);
  const [programFilter, setProgramFilter] = useState('ALL');
  const [deptFilter, setDeptFilter] = useState('ALL');
  const [yearFilter, setYearFilter] = useState('ALL');
  const [semesterFilter, setSemesterFilter] = useState('ALL');

  useEffect(() => {
    async function loadMeta() {
      try {
        const [progRes, deptRes] = await Promise.all([
          programsApi.getPrograms(),
          departmentsApi.getDepartments(),
        ]);
        if (progRes.data) setPrograms(progRes.data);
        if (deptRes.data) setDepartments(deptRes.data);
      } catch (e) {
        console.error('Failed to load metadata', e);
      }
    }
    loadMeta();
  }, []);

  const loadDashboard = async () => {
    setLoading(true);
    setError(null);
    try {
      if (user?.role === 'STUDENT') {
        const res = await studentsApi.getMyProfile();
        if (res.data) {
          setStudentData(res.data);
        }
      } else {
        const res = await dashboardApi.getSummary({
          program: programFilter !== 'ALL' ? programFilter : undefined,
          department: deptFilter !== 'ALL' ? deptFilter : undefined,
          year: yearFilter !== 'ALL' ? yearFilter : undefined,
          semester: semesterFilter !== 'ALL' ? semesterFilter : undefined,
        });
        if (res.data) {
          setData(res.data);
        }
      }
    } catch (err: any) {
      setError(err.response?.data?.message || 'Failed to load live dashboard information.');
```

```

const resetFilters = () => {
  setProgramFilter('ALL');
  setDeptFilter('ALL');
  setYearFilter('ALL');
  setSemesterFilter('ALL');
};

// =====
// 1. STUDENT VIEW
// =====
if (isStudent) {
  const latestPred = studentData?.latestPrediction;
  const calc = studentData?.calculatedMetrics || {};
  const score = latestPred ? Math.round(latestPred.predictedScore) : null;
  const risk = latestPred?.riskLevel;

  return (
    <AppLayout>
      <div className="space-y-6 max-w-5xl mx-auto pb-12">
        {/* Header */}
        <div className="border-b border-slate-200 pb-4">
          <h1 className="text-2xl font-bold tracking-tight text-slate-900">
            My Academic Dashboard
          </h1>
          <p className="mt-1 text-xs text-slate-500">
            Welcome, {studentData?.name || 'Student'}. View your academic standing, expected score, and suggested support.
          </p>
        </div>

        {loading ? (
          <LoadingSkeleton rows={4} />
        ) : error ? (
          <div className="rounded-lg border border-red-200 bg-red-50 p-4 text-xs text-red-700">
            {error}
          </div>
        ) : (
          <div className="space-y-6">
            {/* Expected Score & Status Highlight */}
            <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
              <div className="rounded-2xl border border-slate-200 bg-white p-6 shadow-xs flex flex-col justify-between">
                <span className="text-xs font-bold uppercase tracking-wider text-slate-500">
                  Expected Score
                </span>
                <div className="mt-3">
                  <span className="text-4xl font-extrabold text-slate-900">
                    {score !== null ? score : '-'}
                  </span>
                  <span className="text-base font-semibold text-slate-400"> / 100</span>
                </div>
                <p className="mt-2 text-xs text-slate-500">
                  AI estimation based on your coursework, assessments, and attendance.
                </p>
              </div>

              <div className="rounded-2xl border border-slate-200 bg-white p-6 shadow-xs flex flex-col justify-between">
                <span className="text-xs font-bold uppercase tracking-wider text-slate-500">
                  Academic Risk Level
                </span>
                <div className="mt-3">
                  {risk === 'HIGH' ? (
                    <span className="inline-flex items-center gap-2 px-4 py-2 rounded-full text-xs font-bold bg-red-100 text-r
ed-800 border border-red-200">
                      <span>🔴</span>
                      <span>HIGH RISK</span>
                    </span>
                  ) : risk === 'MEDIUM' ? (
                    <span className="inline-flex items-center gap-2 px-4 py-2 rounded-full text-xs font-bold bg-amber-100 text
-amber-800 border border-amber-200">
                      <span>🟡</span>
                      <span>MEDIUM RISK</span>
                    </span>
                  ) : (
                    <span className="inline-flex items-center gap-2 px-4 py-2 rounded-full text-xs font-bold bg-emerald-100 te
xt-emerald-800 border border-emerald-200">
                      <span>🟢</span>
                      <span>LOW RISK</span>
                    </span>
                  )
                </div>
              </div>
            </div>
          </div>
        )
      </AppLayout>
    )
  );
}

```

```

    <p className="mt-2 text-xs text-slate-500">
      {risk === 'HIGH'
        ? 'Mentoring and dedicated attention advised.'
        : risk === 'MEDIUM'
        ? 'Stay on top of upcoming assignments and practical labs.'
        : 'Progressing well within expected institutional targets.'}
    </p>
  </div>
</div>

{/* 6 Metric Cards */}
<div className="grid grid-cols-2 md:grid-cols-3 lg:grid-cols-6 gap-3">
  <div className="rounded-xl border border-slate-200 bg-white p-4 shadow-xs">
    <span className="text-[11px] font-semibold text-slate-500 flex items-center gap-1">
      <CalendarCheck className="h-3.5 w-3.5 text-blue-600" /> Attendance
    </span>
    <p className="text-xl font-bold text-slate-900 mt-2">
      {calc.attendance?.formattedValue || '-'}
    </p>
    <span className="text-[10px] text-slate-400">Class lectures</span>
  </div>

  <div className="rounded-xl border border-slate-200 bg-white p-4 shadow-xs">
    <span className="text-[11px] font-semibold text-slate-500 flex items-center gap-1">
      <GraduationCap className="h-3.5 w-3.5 text-blue-600" /> Previous Marks
    </span>
    <p className="text-xl font-bold text-slate-900 mt-2">
      {calc.previousScore?.formattedValue || '-'}
    </p>
    <span className="text-[10px] text-slate-400">Prior semesters</span>
  </div>

  <div className="rounded-xl border border-slate-200 bg-white p-4 shadow-xs">
    <span className="text-[11px] font-semibold text-slate-500 flex items-center gap-1">
      <Award className="h-3.5 w-3.5 text-blue-600" /> Assessments
    </span>
    <p className="text-xl font-bold text-slate-900 mt-2">
      {calc.internalMarks?.formattedValue || '-'}
    </p>
    <span className="text-[10px] text-slate-400">Internal score</span>
  </div>

  <div className="rounded-xl border border-slate-200 bg-white p-4 shadow-xs">
    <span className="text-[11px] font-semibold text-slate-500 flex items-center gap-1">
      <FileCheck className="h-3.5 w-3.5 text-blue-600" /> Assignments
    </span>
    <p className="text-xl font-bold text-slate-900 mt-2">
      {calc.assignmentCompletion?.formattedValue || '-'}
    </p>
    <span className="text-[10px] text-slate-400">Coursework</span>
  </div>

  <div className="rounded-xl border border-slate-200 bg-white p-4 shadow-xs">
    <span className="text-[11px] font-semibold text-slate-500 flex items-center gap-1">
      <Clock className="h-3.5 w-3.5 text-blue-600" /> Study Hours
    </span>
    <p className="text-xl font-bold text-slate-900 mt-2">
      {calc.studyHours?.formattedValue || '-'}
    </p>
    <span className="text-[10px] text-slate-400">Weekly logs</span>
  </div>

  <div className="rounded-xl border border-slate-200 bg-white p-4 shadow-xs">
    <span className="text-[11px] font-semibold text-slate-500 flex items-center gap-1">
      <Activity className="h-3.5 w-3.5 text-blue-600" /> Participation
    </span>
    <p className="text-xl font-bold text-slate-900 mt-2">
      {calc.participation?.formattedValue || '-'}
    </p>
    <span className="text-[10px] text-slate-400">In-class score</span>
  </div>
</div>

{/* Recommendations */}
<div className="rounded-2xl border border-slate-200 bg-white p-6 shadow-xs space-y-3">
  <h3 className="text-base font-bold text-slate-900">
    Recommended Support for Your Academic Success
  </h3>
  <ul className="space-y-2 text-xs text-slate-600">

```

7/92

```

<span className="text-xs font-bold uppercase tracking-wider text-slate-500 flex items-center gap-1.5">
  <Filter className="h-3.5 w-3.5 text-blue-600" />
  <span>Filter Dashboard Cohort</span>
</span>
{(programFilter !== 'ALL' || deptFilter !== 'ALL' || yearFilter !== 'ALL' || semesterFilter !== 'ALL') && (
  <button
    type="button"
    onClick={resetFilters}
    className="text-xs text-blue-600 hover:text-blue-800 font-medium"
  >
    Reset Filter (Show All College)
  </button>
)}
</div>

<div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-4 gap-3 text-xs">
  /* Program */
  <div>
    <label className="block text-[11px] font-semibold text-slate-600 mb-1">Program</label>
    <select
      value={programFilter}
      onChange={(e) => {
        setProgramFilter(e.target.value);
        setDeptFilter('ALL');
        setYearFilter('ALL');
        setSemesterFilter('ALL');
      }}
      className="w-full rounded-lg border border-slate-200 bg-slate-50/70 p-2 text-xs font-semibold text-slate-800 focus:outline-none"
    >
      <option value="ALL">All Programs (College-Wide)</option>
      {programs.map((p) => (
        <option key={p._id} value={p.name}>
          {p.name} ({p.durationYears} Years)
        </option>
      ))}
    </select>
  </div>

  /* Department */
  <div>
    <label className="block text-[11px] font-semibold text-slate-600 mb-1">Department</label>
    <select
      value={deptFilter}
      onChange={(e) => {
        setDeptFilter(e.target.value);
      }}
      className="w-full rounded-lg border border-slate-200 bg-slate-50/70 p-2 text-xs font-semibold text-slate-800 focus:outline-none"
    >
      <option value="ALL">All Departments</option>
      {availableDepts.map((d) => (
        <option key={d._id} value={d.name}>
          {d.name}
        </option>
      ))}
    </select>
  </div>

  /* Year */
  <div>
    <label className="block text-[11px] font-semibold text-slate-600 mb-1">Year</label>
    <select
      value={yearFilter}
      onChange={(e) => {
        const y = e.target.value;
        setYearFilter(y);
        setSemesterFilter('ALL');
      }}
      className="w-full rounded-lg border border-slate-200 bg-slate-50/70 p-2 text-xs font-semibold text-slate-800 focus:outline-none"
    >
      <option value="ALL">All Years</option>
      {availableYears.map((y) => (
        <option key={y} value={y}>
          {y === 1 ? '1st Year' : y === 2 ? '2nd Year' : y === 3 ? '3rd Year' : `${y}th Year`}
        </option>
      ))}
    </select>
  </div>

```

```

</div>

{/* Semester */}
<div>
  <label className="block text-[11px] font-semibold text-slate-600 mb-1">Semester</label>
  <select
    value={semesterFilter}
    onChange={(e) => setSemesterFilter(e.target.value)}
    className="w-full rounded-lg border border-slate-200 bg-slate-50/70 p-2 text-xs font-semibold text-slate-800 focus:outline-none"
  >
    <option value="ALL">All Semesters</option>
    {semOptions.map((s) => (
      <option key={s} value={s}>
        Semester {s}
      </option>
    ))}
  </select>
</div>
</div>
</div>

{/* ===== */}
{/* 6 SUMMARY CARDS (Requirement 18) */}
{/* Total Students | High Risk | Med Risk | Low Risk | Avg Pred | Avg Att */}
{/* ===== */}
{loading || !data ? (
  <div className="grid grid-cols-2 md:grid-cols-3 lg:grid-cols-6 gap-3">
    {[1, 2, 3, 4, 5, 6].map((i) => (
      <div key={i} className="h-24 rounded-2xl bg-slate-100 animate-pulse" />
    ))}
  </div>
) : (
  <div className="grid grid-cols-2 md:grid-cols-3 lg:grid-cols-6 gap-3">
    {/* Total Students */}
    <div className="rounded-xl border border-slate-200 bg-white p-4 shadow-xs flex flex-col justify-between">
      <span className="text-[11px] font-bold uppercase tracking-wider text-slate-500">
        Total Students
      </span>
      <p className="text-2xl font-black text-slate-900 mt-2">
        {data.totalStudents}
      </p>
      <span className="text-[10px] text-slate-400">Cohort size</span>
    </div>

    {/* High Risk */}
    <div className="rounded-xl border border-red-200 bg-red-50/40 p-4 shadow-xs flex flex-col justify-between">
      <span className="text-[11px] font-bold uppercase tracking-wider text-red-700 flex items-center gap-1">
        <span>🔴</span> High Risk
      </span>
      <p className="text-2xl font-black text-red-600 mt-2">
        {data.highRiskCount}
      </p>
      <span className="text-[10px] text-red-600/70">Urgent support</span>
    </div>

    {/* Medium Risk */}
    <div className="rounded-xl border border-amber-200 bg-amber-50/40 p-4 shadow-xs flex flex-col justify-between">
      <span className="text-[11px] font-bold uppercase tracking-wider text-amber-700 flex items-center gap-1">
        <span>🟡</span> Medium Risk
      </span>
      <p className="text-2xl font-black text-amber-600 mt-2">
        {data.mediumRiskCount}
      </p>
      <span className="text-[10px] text-amber-600/70">Watchlist review</span>
    </div>

    {/* Low Risk */}
    <div className="rounded-xl border border-emerald-200 bg-emerald-50/40 p-4 shadow-xs flex flex-col justify-between">
      <span className="text-[11px] font-bold uppercase tracking-wider text-emerald-700 flex items-center gap-1">
        <span>🟢</span> Low Risk
      </span>
      <p className="text-2xl font-black text-emerald-600 mt-2">
        {data.lowRiskCount}
      </p>
      <span className="text-[10px] text-emerald-600/70">On track</span>
    </div>

    {/* Average Predicted Score */}

```

```

<div className="rounded-xl border border-slate-200 bg-white p-4 shadow-xs flex flex-col justify-between">
  <span className="text-[11px] font-bold uppercase tracking-wider text-slate-500 flex items-center gap-1">
    <Sparkles className="h-3 w-3 text-blue-600" /> Avg Score
  </span>
  <p className="text-2xl font-black text-slate-900 mt-2">
    {data.averagePredictedScore !== undefined ? `${data.averagePredictedScore}` : '-'}
    <span className="text-xs font-semibold text-slate-400"> / 100</span>
  </p>
  <span className="text-[10px] text-slate-400">Expected performance</span>
</div>

{/* Average Attendance */}
<div className="rounded-xl border border-slate-200 bg-white p-4 shadow-xs flex flex-col justify-between">
  <span className="text-[11px] font-bold uppercase tracking-wider text-slate-500 flex items-center gap-1">
    <CalendarCheck className="h-3 w-3 text-blue-600" /> Avg Attendance
  </span>
  <p className="text-2xl font-black text-slate-900 mt-2">
    {data.averageAttendance !== undefined ? `${data.averageAttendance}%` : '-'}
  </p>
  <span className="text-[10px] text-slate-400">Class lectures & labs</span>
</div>
</div>
)}

{/* ===== */}
{/* STUDENTS REQUIRING ATTENTION TABLE (Requirement 17 & 18) */}
{/* ===== */}
{data && (
  <div className="space-y-3">
    <div className="flex items-center justify-between">
      <div>
        <h3 className="text-base font-bold text-slate-900">
          Students Requiring Attention
        </h3>
        <p className="text-xs text-slate-500">
          Students identified as high or medium risk based on academic records and attendance.
        </p>
      </div>
      <Link
        href="/students"
        className="text-xs font-semibold text-blue-600 hover:text-blue-800 flex items-center gap-1"
      >
        <span>View All Students</span>
        <ArrowRight className="h-3 w-3" />
      </Link>
    </div>

    <AttentionRequiredTable students={data.studentsRequiringAttention} />
  </div>
)}
</div>
</AppLayout>
);
}

```

Attention Required Priority Queue Component

TYPESCRIPT

File: apps/web/src/components/dashboard/AttentionRequiredTable.tsx

Purpose & Architecture: Renders the priority queue on the executive dashboard displaying students flagged with high/medium academic risk alongside specific flags.

```

'use client';

import React from 'react';
import Link from 'next/link';
import { ArrowRight } from 'lucide-react';

interface AttentionStudent {
  _id: string;
  studentId: string;
  name: string;
  program?: string;
  year?: number;
  yearLabel?: string;
}

```

```

semester?: number;
section?: string;
department?: string;
className?: string;
predictedScore: number;
riskLevel: 'LOW' | 'MEDIUM' | 'HIGH';
topRiskFactor?: string;
lastInterventionStatus?: string;
attendance?: number;
previousScore?: number;
}

export function AttentionRequiredTable({
  students,
}): {
  students: AttentionStudent[];
}) {
  if (!students || students.length === 0) {
    return (
      <div className="p-8 text-center text-xs text-slate-500 bg-white rounded-xl border border-slate-200">
        No students currently requiring immediate academic attention.
      </div>
    );
  }

  return (
    <div className="overflow-x-auto rounded-xl border border-slate-200 bg-white shadow-xs">
      <table className="min-w-full divide-y divide-slate-200 text-left text-xs">
        <thead className="bg-slate-50 text-slate-600 font-bold uppercase tracking-wider text-[11px]">
          <tr>
            <th scope="col" className="px-5 py-3.5">Student Name</th>
            <th scope="col" className="px-5 py-3.5">Year</th>
            <th scope="col" className="px-5 py-3.5">Semester</th>
            <th scope="col" className="px-5 py-3.5 text-center">Predicted Score</th>
            <th scope="col" className="px-5 py-3.5 text-center">Risk</th>
            <th scope="col" className="px-5 py-3.5 text-right">Action</th>
          </tr>
        </thead>
        <tbody className="divide-y divide-slate-100 text-slate-800">
          {students.map((student) => {
            const riskConfig =
              student.riskLevel === 'HIGH'
                ? { label: 'High', dot: '●', style: 'text-red-700 bg-red-50 border-red-200' }
                : student.riskLevel === 'MEDIUM'
                ? { label: 'Medium', dot: '●', style: 'text-amber-700 bg-amber-50 border-amber-200' }
                : { label: 'Low', dot: '●', style: 'text-emerald-700 bg-emerald-50 border-emerald-200' };

            const yearText =
              student.yearLabel ||
              (student.year ? `${student.year === 1 ? '1st' : student.year === 2 ? '2nd' : student.year === 3 ? '3rd' : `${student.year}th`} Year` : '-');

            return (
              <tr key={student._id} className="hover:bg-slate-50/80 transition-colors">
                <td className="px-5 py-3.5 font-medium text-slate-900 whitespace-nowrap">
                  <div className="font-bold text-slate-900">{student.name}</div>
                  <div className="text-[11px] text-slate-400 font-mono">
                    {student.studentId} &bull; {student.program || 'B.Tech'}
                  </div>
                </td>
                <td className="px-5 py-3.5 font-semibold text-slate-700 whitespace-nowrap">
                  {yearText}
                </td>
                <td className="px-5 py-3.5 font-semibold text-slate-700 whitespace-nowrap">
                  Semester {student.semester || '-'}
                </td>
                <td className="px-5 py-3.5 text-center font-bold text-slate-900 whitespace-nowrap">
                  {Math.round(student.predictedScore)} / 100
                </td>
                <td className="px-5 py-3.5 text-center whitespace-nowrap">
                  <span
                    className={`inline-flex items-center gap-1.5 px-3 py-1 rounded-full text-xs font-bold border ${riskConfig.style}`>
                      >
                        <span>{riskConfig.dot}</span>
                        <span>{riskConfig.label}</span>
                      </span>
                </td>
                <td className="px-5 py-3.5 text-right whitespace-nowrap">

```

```

    {(() => {
      const sId = typeof student._id === 'object' && student._id !== null
        ? String((student._id as any)._id || (student._id as any).toString())
        : String(student._id || student.studentId || '');

      return (
        <Link
          href={` /students/${sId}`}
          className="inline-flex items-center gap-1 rounded-md bg-blue-50 border border-blue-200 px-3 py-1 text-xs
font-semibold text-blue-700 hover:bg-blue-100 transition-colors"
        >
          <span>Profile</span>
          <ArrowRight className="h-3 w-3" />
        </Link>
      );
    })()
  </td>
</tr>
);
}}
</tbody>
</table>
</div>
);
}

```

A.1.2 Student Management

Student Cohort Directory Controller

TYPESCRIPT

File: apps/web/src/app/students/page.tsx

Purpose & Architecture: Provides comprehensive cohort management with full-text search, multi-dimensional filtering (Program, Department, Year, Semester, Section, Risk Status), pagination controls, navigation to profile records, and API communication.

```

'use client';

import React, { useState, useEffect } from 'react';
import Link from 'next/link';
import { AppLayout } from '@components/layout/AppLayout';
import { EmptyState } from '@components/ui/EmptyState';
import { LoadingSkeleton } from '@components/ui/LoadingSkeleton';
import { studentsApi, programsApi, departmentsApi } from '@lib/apiClient';
import { StudentDTO } from '@eduguard/shared';
import {
  Users,
  Search,
  Plus,
  ArrowRight,
  X,
  AlertCircle,
  CheckCircle2,
  Filter,
} from 'lucide-react';
import { useAuth } from '@context/AuthContext';

export default function StudentsPage() {
  const { isAdmin, isTeacher } = useAuth();
  const [students, setStudents] = useState<StudentDTO[]>([]);
  const [loading, setLoading] = useState(true);

  // Metadata
  const [programs, setPrograms] = useState<any[]>([]);
  const [departments, setDepartments] = useState<any[]>([]);

  // Filters (Requirement 18)
  const [search, setSearch] = useState('');
  const [programFilter, setProgramFilter] = useState('ALL');
  const [departmentFilter, setDepartmentFilter] = useState('ALL');
  const [yearFilter, setYearFilter] = useState('ALL');
  const [semesterFilter, setSemesterFilter] = useState('ALL');
  const [sectionFilter, setSectionFilter] = useState('ALL');
  const [riskFilter, setRiskFilter] = useState('ALL');

  const [page, setPage] = useState(1);

```

```

const [totalPages, setTotalPages] = useState(1);
const [totalCount, setTotalCount] = useState(0);

// Modal
const [showAddModal, setShowAddModal] = useState(false);
const [formData, setFormData] = useState({
  studentId: '',
  name: '',
  email: '',
  phone: '',
  program: 'B.Tech',
  department: 'Computer Science',
  year: 2,
  semester: 3,
  section: 'A',
});
const [formLoading, setFormLoading] = useState(false);
const [formError, setFormError] = useState<string | null>(null);

// Load programs & departments metadata
useEffect(() => {
  async function loadMeta() {
    try {
      const [progRes, deptRes] = await Promise.all([
        programsApi.getPrograms(),
        departmentsApi.getDepartments(),
      ]);
      if (progRes.data) setPrograms(progRes.data);
      if (deptRes.data) setDepartments(deptRes.data);
    } catch (e) {
      console.error('Failed to load metadata', e);
    }
  }
  loadMeta();
}, []);

const fetchStudents = async () => {
  setLoading(true);
  try {
    const res = await studentsApi.getStudents({
      search: search.trim() || undefined,
      program: programFilter === 'ALL' ? undefined : programFilter,
      department: departmentFilter === 'ALL' ? undefined : departmentFilter,
      year: yearFilter === 'ALL' ? undefined : yearFilter,
      semester: semesterFilter === 'ALL' ? undefined : semesterFilter,
      section: sectionFilter === 'ALL' ? undefined : sectionFilter,
      riskLevel: riskFilter === 'ALL' ? undefined : riskFilter,
      page,
      limit: 15,
    });
    if (res.data) {
      setStudents(res.data);
      if (res.meta) {
        setTotalPages(res.meta.totalPages || 1);
        setTotalCount(res.meta.total || 0);
      }
    }
  } catch (e) {
    console.error(e);
  } finally {
    setLoading(false);
  }
};

useEffect(() => {
  const timer = setTimeout(() => {
    fetchStudents();
  }, 200);
  return () => clearTimeout(timer);
}, [search, programFilter, departmentFilter, yearFilter, semesterFilter, sectionFilter, riskFilter, page]);

// Determine allowed years based on program
const currentProgDoc = programs.find((p) => p.name === programFilter);
const maxYears = currentProgDoc?.durationYears || (programFilter === 'B.Tech' ? 4 : programFilter === 'ALL' ? 4 : 3);
const yearOptions = Array.from({ length: maxYears }, (_, i) => i + 1);

// Filtered departments for program
const availableDepts = programFilter === 'ALL' ? departments : departments.filter((d) => d.program === programFilter);

```

```

const handleCreateStudent = async (e: React.FormEvent) => {
  e.preventDefault();
  setFormLoading(true);
  setFormError(null);
  try {
    await studentsApi.createStudent({
      ...formData,
      course: formData.program,
    });
    setShowAddModal(false);
    setFormData({
      studentId: '',
      name: '',
      email: '',
      phone: '',
      program: 'B.Tech',
      department: 'Computer Science',
      year: 2,
      semester: 3,
      section: 'A',
    });
    fetchStudents();
  } catch (err: any) {
    setFormError(err.response?.data?.message || 'Failed to create student.');
```

```

  } finally {
    setFormLoading(false);
  }
};

// Dependent semesters based on selected year
const semOptions =
  yearFilter !== 'ALL'
    ? [Number(yearFilter) * 2 - 1, Number(yearFilter) * 2]
    : Array.from({ length: maxYears * 2 }, (_, i) => i + 1);

const clearFilters = () => {
  setSearch('');
  setProgramFilter('ALL');
  setDepartmentFilter('ALL');
  setYearFilter('ALL');
  setSemesterFilter('ALL');
  setSectionFilter('ALL');
  setRiskFilter('ALL');
  setPage(1);
};

return (
  <AppLayout>
    <div className="space-y-6 max-w-5xl mx-auto pb-12">
      { /* DEMO DATA Banner */ }
      <div className="rounded-xl border border-blue-200 bg-blue-50/70 p-3.5 flex items-center justify-between text-xs text-blue-800">
        <div className="flex items-center gap-2">
          <AlertCircle className="h-4 w-4 text-blue-600 shrink-0" />
          <span>
            <strong className="font-semibold">DEMO DATA MODE:</strong> Student records, attendance, and assessment grades are
            synthetic demo data created for college multi-program testing.
          </span>
        </div>
        <span className="text-[10px] font-bold uppercase tracking-wider px-2 py-0.5 bg-blue-200/60 rounded text-blue-900">
          120+ Students Enrolled
        </span>
      </div>

      { /* Top Header */ }
      <div className="flex flex-col sm:flex-row sm:items-center sm:justify-between gap-4 border-b border-slate-200 pb-4">
        <div>
          <h1 className="text-2xl font-bold tracking-tight text-slate-900">
            Enrolled Students ({totalCount})
          </h1>
          <p className="mt-1 text-xs text-slate-500">
            College directory across B.Tech, BBA, and B.Sc degree programs.
          </p>
        </div>

        {isAdmin && (
          <button
            type="button"
            onClick={() => {

```

```

        setFormError(null);
        setShowAddModal(true);
    })
    className="inline-flex items-center gap-1.5 rounded-lg bg-blue-600 px-3.5 py-2 text-xs font-semibold text-white shadow-xs hover:bg-blue-700 transition-colors"
  >
    <Plus className="h-4 w-4" />
    <span>Add Student</span>
  </button>
</div>

{/* ===== */}
{/* MULTI-CRITERIA FILTERS BAR (Requirements 16 & 17) */}
{/* Search | Program | Department | Year | Semester | Section | Risk */}
{/* ===== */}
<div className="rounded-2xl border border-slate-200 bg-white p-5 shadow-xs space-y-3">
  <div className="flex items-center justify-between">
    <span className="text-xs font-bold uppercase tracking-wider text-slate-500 flex items-center gap-1.5">
      <Filter className="h-3.5 w-3.5 text-blue-600" />
      <span>Filter Student Cohort</span>
    </span>
    <button
      type="button"
      onClick={clearFilters}
      className="text-xs text-blue-600 hover:text-blue-800 font-medium"
    >
      Reset Filters
    </button>
  </div>

  <div className="grid grid-cols-1 sm:grid-cols-2 md:grid-cols-4 lg:grid-cols-8 gap-2 text-xs">
    <div className="sm:col-span-2 relative">
      <Search className="pointer-events-none absolute left-3 top-2.5 h-3.5 w-3.5 text-slate-400" />
      <input
        type="text"
        value={search}
        onChange={(e) => {
          setSearch(e.target.value);
          setPage(1);
        }}
        placeholder="Search name or ID..."
        className="w-full rounded-lg border border-slate-200 pl-8 pr-3 py-2 text-xs text-slate-800 focus:border-blue-500 focus:outline-none"
      />
    </div>

    <div>
      <div>
        <select
          value={programFilter}
          onChange={(e) => {
            setProgramFilter(e.target.value);
            setYearFilter('ALL');
            setSemesterFilter('ALL');
            setPage(1);
          }}
          className="w-full rounded-lg border border-slate-200 p-2 text-xs text-slate-700 focus:border-blue-500 focus:outline-none font-medium"
        >
          <option value="ALL">All Programs</option>
          {programs.map((p) => (
            <option key={p._id} value={p.name}>
              {p.name} ({p.durationYears} Yrs)
            </option>
          ))}
        </select>
      </div>

      <div>
        <select
          value={departmentFilter}
          onChange={(e) => {
            setDepartmentFilter(e.target.value);
            setPage(1);
          }}
          className="w-full rounded-lg border border-slate-200 p-2 text-xs text-slate-700 focus:border-blue-500 focus:outline-none font-medium"
        >
          <option value="ALL">All Departments</option>
          {departments.map((d) => (
            <option key={d._id} value={d.name}>
              {d.name} ({d.durationYears} Yrs)
            </option>
          ))}
        </select>
      </div>
    </div>
  </div>

```

```

ine-none"
    >
    <option value="ALL">All Depts</option>
    {availableDepts.map((d) => (
      <option key={d._id} value={d.name}>
        {d.name}
      </option>
    ))}
  </select>
</div>

{/* Year Filter */}
<div>
  <select
    value={yearFilter}
    onChange={(e) => {
      const y = e.target.value;
      setYearFilter(y);
      setSemesterFilter('ALL');
      setPage(1);
    }}
    className="w-full rounded-lg border border-slate-200 p-2 text-xs text-slate-700 focus:border-blue-500 focus:outline"
  >
    <option value="ALL">All Years</option>
    {yearOptions.map((y) => (
      <option key={y} value={y}>
        {y === 1 ? '1st Year' : y === 2 ? '2nd Year' : y === 3 ? '3rd Year' : `${y}th Year`}
      </option>
    ))}
  </select>
</div>

{/* Semester Filter (Dependent on Year) */}
<div>
  <select
    value={semesterFilter}
    onChange={(e) => {
      setSemesterFilter(e.target.value);
      setPage(1);
    }}
    className="w-full rounded-lg border border-slate-200 p-2 text-xs text-slate-700 focus:border-blue-500 focus:outline"
  >
    <option value="ALL">All Semesters</option>
    {semOptions.map((s) => (
      <option key={s} value={s}>
        Semester {s}
      </option>
    ))}
  </select>
</div>

{/* Section Filter (Requirement 16) */}
<div>
  <select
    value={sectionFilter}
    onChange={(e) => {
      setSectionFilter(e.target.value);
      setPage(1);
    }}
    className="w-full rounded-lg border border-slate-200 p-2 text-xs text-slate-700 focus:border-blue-500 focus:outline"
  >
    <option value="ALL">All Sections</option>
    <option value="A">Section A</option>
    <option value="B">Section B</option>
    <option value="C">Section C</option>
  </select>
</div>

{/* Risk Filter */}
<div>
  <select
    value={riskFilter}
    onChange={(e) => {
      setRiskFilter(e.target.value);
      setPage(1);
    }}
  >

```

```

        className="w-full rounded-lg border border-slate-200 p-2 text-xs font-semibold text-slate-700 focus:border-blue-
500 focus:outline-none"
      >
        <option value="ALL">All Risk Levels</option>
        <option value="HIGH">🔴 High Risk</option>
        <option value="MEDIUM">🟡 Medium Risk</option>
        <option value="LOW">🟢 Low Risk</option>
      </select>
    </div>
  </div>
</div>

{ /* ===== */
{ /* STUDENTS TABLE */
{ /* ===== */
<div className="rounded-2xl border border-slate-200 bg-white p-6 shadow-xs space-y-4">
  <div className="flex items-center justify-between border-b border-slate-100 pb-3">
    <h3 className="text-sm font-bold text-slate-900">
      Enrolled Students ({totalCount})
    </h3>
    <span className="text-xs text-slate-500">
      Page {page} of {totalPages}
    </span>
  </div>

  {loading ? (
    <LoadingSkeleton rows={6} />
  ) : students.length === 0 ? (
    <EmptyState
      icon={Users}
      title="No Students Found"
      description="No students match the selected filter criteria. Try adjusting or clearing your filters."
    >
  ) : (
    <div className="overflow-x-auto">
      <table className="w-full text-left text-xs">
        <thead>
          <tr className="border-b border-slate-200 bg-slate-50 text-[11px] font-bold uppercase tracking-wider text-slate
-500">
            <th className="py-3 px-4">Student</th>
            <th className="py-3 px-4">Program & Dept</th>
            <th className="py-3 px-4">Cohort</th>
            <th className="py-3 px-4 text-center">Score Estimate</th>
            <th className="py-3 px-4 text-center">Risk Level</th>
            <th className="py-3 px-4 text-right">Action</th>
          </tr>
        </thead>
        <tbody className="divide-y divide-slate-100">
          {students.map((stu) => {
            const latest = stu.latestPrediction;
            const score = latest ? Math.round(latest.predictedScore) : null;
            const risk = latest?.riskLevel;
            const sId = typeof stu._id === 'object' && stu._id !== null
              ? String((stu._id as any)._id || (stu._id as any).toString())
              : String(stu._id || stu.studentId || '');

            return (
              <tr key={sId || stu.studentId} className="hover:bg-slate-50/60 transition-colors">
                <td className="py-3.5 px-4 whitespace-nowrap">
                  <Link
                    href={` /students/${sId}`}
                    className="font-bold text-slate-900 hover:text-blue-600 transition-colors"
                  >
                    {stu.name}
                  </Link>
                  <div className="text-[11px] text-slate-400 font-mono">
                    {stu.studentId}
                  </div>
                </td>
                <td className="py-3.5 px-4 whitespace-nowrap">
                  <span className="font-semibold text-slate-800">
                    {stu.program || stu.course || 'B.Tech'}
                  </span>
                  <div className="text-[11px] text-slate-500">{stu.department}</div>
                </td>
                <td className="py-3.5 px-4 whitespace-nowrap text-slate-600">
                  <span>{stu.year} Year &bull; Sem {stu.semester}</span>
                  <span className="ml-1 text-slate-400 font-mono">{Sec {stu.section}}</span>
                </td>

```

```

        <td className="py-3.5 px-4 text-center font-bold text-slate-900 whitespace-nowrap">
            {score !== null ? `${score} / 100` : '-'}
        </td>
        <td className="py-3.5 px-4 text-center whitespace-nowrap">
            {risk === 'HIGH' ? (
                <span className="inline-flex items-center gap-1 px-2.5 py-0.5 rounded-full text-xs font-bold bg-red-50 text-red-700 border border-red-200">
                    <span>🔴</span>
                    <span>High</span>
                </span>
            ) : risk === 'MEDIUM' ? (
                <span className="inline-flex items-center gap-1 px-2.5 py-0.5 rounded-full text-xs font-bold bg-amber-50 text-amber-700 border border-amber-200">
                    <span>🟡</span>
                    <span>Medium</span>
                </span>
            ) : risk === 'LOW' ? (
                <span className="inline-flex items-center gap-1 px-2.5 py-0.5 rounded-full text-xs font-bold bg-emerald-50 text-emerald-700 border border-emerald-200">
                    <span>🟢</span>
                    <span>Low</span>
                </span>
            ) : (
                <span className="text-slate-400 italic text-[11px]">Unrated</span>
            )
        </td>
        <td className="py-3.5 px-4 text-right whitespace-nowrap">
            <Link
                href={` /students/${sId}`}
                className="inline-flex items-center gap-1 rounded-md bg-blue-50 border border-blue-200 px-3 py-1 text-xs font-semibold text-blue-700 hover:bg-blue-100 transition-colors"
            >
                <span>Profile</span>
                <ArrowRight className="h-3 w-3" />
            </Link>
        </td>
    </tr>
</tbody>
</table>

    { /* Pagination */ }
    {totalPages > 1 && (
        <div className="pt-4 border-t border-slate-100 flex items-center justify-between">
            <span className="text-xs text-slate-500">
                Showing page {page} of {totalPages}
            </span>
            <div className="flex items-center gap-1">
                <button
                    type="button"
                    disabled={page <= 1}
                    onClick={() => setPage((p) => Math.max(1, p - 1))}
                    className="rounded border border-slate-200 px-3 py-1 text-xs font-semibold text-slate-700 hover:bg-slate-50 disabled:opacity-50"
                >
                    Previous
                </button>
                <button
                    type="button"
                    disabled={page >= totalPages}
                    onClick={() => setPage((p) => Math.min(totalPages, p + 1))}
                    className="rounded border border-slate-200 px-3 py-1 text-xs font-semibold text-slate-700 hover:bg-slate-50 disabled:opacity-50"
                >
                    Next
                </button>
            </div>
        </div>
    )}
</div>
    )}
</div>
</div>

    { /* Add Student Modal */ }
    {showAddModal && (
        <div className="fixed inset-0 z-50 flex items-center justify-center bg-slate-900/40 backdrop-blur-xs p-4">
            <div className="w-full max-w-md rounded-2xl border border-slate-200 bg-white p-6 shadow-xl space-y-4">

```

```

<div className="flex items-center justify-between border-b border-slate-100 pb-3">
  <h3 className="text-base font-bold text-slate-900">Add New Student</h3>
  <button
    type="button"
    onClick={() => setShowAddModal(false)}
    className="text-slate-400 hover:text-slate-700 p-1"
  >
    <X className="h-4 w-4" />
  </button>
</div>

{formError && (
  <div className="rounded-xl border border-red-200 bg-red-50 p-3 text-xs text-red-700 flex items-center gap-1.5">
    <AlertCircle className="h-4 w-4 shrink-0" />
    <span>{formError}</span>
  </div>
)}

<form onSubmit={handleCreateStudent} className="space-y-3 text-xs">
  <div>
    <label className="block text-slate-600 font-semibold mb-1">Student ID *</label>
    <input
      type="text"
      required
      placeholder="e.g. STU2026099"
      value={formData.studentId}
      onChange={(e) => setFormData({ ...formData, studentId: e.target.value })}
      className="w-full rounded-lg border border-slate-200 p-2 font-mono uppercase"
    />
  </div>

  <div>
    <label className="block text-slate-600 font-semibold mb-1">Full Name *</label>
    <input
      type="text"
      required
      placeholder="e.g. Aarav Sharma"
      value={formData.name}
      onChange={(e) => setFormData({ ...formData, name: e.target.value })}
      className="w-full rounded-lg border border-slate-200 p-2"
    />
  </div>

  <div>
    <label className="block text-slate-600 font-semibold mb-1">Email *</label>
    <input
      type="email"
      required
      placeholder="student@eduguard.demo"
      value={formData.email}
      onChange={(e) => setFormData({ ...formData, email: e.target.value })}
      className="w-full rounded-lg border border-slate-200 p-2"
    />
  </div>

  <div className="grid grid-cols-2 gap-2">
    <div>
      <label className="block text-slate-600 font-semibold mb-1">Program</label>
      <select
        value={formData.program}
        onChange={(e) => setFormData({ ...formData, program: e.target.value })}
        className="w-full rounded-lg border border-slate-200 p-2"
      >
        <option value="B.Tech">B.Tech</option>
        <option value="BBA">BBA</option>
        <option value="B.Sc">B.Sc</option>
      </select>
    </div>
    <div>
      <label className="block text-slate-600 font-semibold mb-1">Department</label>
      <select
        value={formData.department}
        onChange={(e) => setFormData({ ...formData, department: e.target.value })}
        className="w-full rounded-lg border border-slate-200 p-2"
      >
        <option value="Computer Science">Computer Science</option>
        <option value="Information Technology">Information Technology</option>
        <option value="Electronics & Communication">Electronics & Comm</option>
        <option value="Business Administration">Business Admin</option>
      </select>
    </div>
  </div>

```

```

    </select>
  </div>
</div>

<div className="grid grid-cols-3 gap-2">
  <div>
    <label className="block text-slate-600 font-semibold mb-1">Year</label>
    <select
      value={formData.year}
      onChange={(e) => setFormData({ ...formData, year: Number(e.target.value) })}
      className="w-full rounded-lg border border-slate-200 p-2"
    >
      <option value={1}>1st</option>
      <option value={2}>2nd</option>
      <option value={3}>3rd</option>
      <option value={4}>4th</option>
    </select>
  </div>
  <div>
    <label className="block text-slate-600 font-semibold mb-1">Semester</label>
    <select
      value={formData.semester}
      onChange={(e) => setFormData({ ...formData, semester: Number(e.target.value) })}
      className="w-full rounded-lg border border-slate-200 p-2"
    >
      {[1, 2, 3, 4, 5, 6, 7, 8].map((s) => (
        <option key={s} value={s}>
          Sem {s}
        </option>
      ))}
    </select>
  </div>
  <div>
    <label className="block text-slate-600 font-semibold mb-1">Section</label>
    <select
      value={formData.section}
      onChange={(e) => setFormData({ ...formData, section: e.target.value })}
      className="w-full rounded-lg border border-slate-200 p-2"
    >
      <option value="A">A</option>
      <option value="B">B</option>
      <option value="C">C</option>
    </select>
  </div>
</div>

<div className="pt-3 border-t border-slate-100 flex items-center justify-end gap-2">
  <button
    type="button"
    onClick={() => setShowAddModal(false)}
    className="rounded-lg border border-slate-200 px-4 py-2 font-semibold text-slate-600 hover:bg-slate-50"
  >
    Cancel
  </button>
  <button
    type="submit"
    disabled={formLoading}
    className="rounded-lg bg-blue-600 px-5 py-2 font-semibold text-white hover:bg-blue-700 disabled:opacity-60"
  >
    {formLoading ? 'Adding...' : 'Add Student'}
  </button>
</div>
</form>
</div>
</div>
))}
</AppLayout>
);
}

```

A.1.3 Student Profile

Student Academic Profile & Performance Hub

File: apps/web/src/app/students/[id]/page.tsx

TYPESCRIPT

Purpose & Architecture: Displays full student biographical and academic status: current metric radars, expected ANN score, risk level badge, flagged attention areas, personalized recommendations, historical semester course grades, assessment breakdowns, counseling interventions, and manual ANN prediction execution.

```
'use client';

import React, { useEffect, useState } from 'react';
import { useParams } from 'next/navigation';
import Link from 'next/link';
import { AppLayout } from '@components/layout/AppLayout';
import { LoadingSkeleton } from '@components/ui/LoadingSkeleton';
import { InterventionModal } from '@components/students/InterventionModal';
import { MetricDetailModal } from '@components/students/MetricDetailModal';
import { studentsApi, predictionsApi } from '@lib/apiClient';
import {
  ArrowLeft,
  ArrowRight,
  CalendarCheck,
  Award,
  BookOpen,
  FileCheck,
  Clock,
  Activity,
  PlusCircle,
  Play,
  AlertCircle,
  CheckCircle2,
  ChevronDown,
  ChevronRight,
  GraduationCap,
  Sparkles,
} from 'lucide-react';
import { useAuth } from '@context/AuthContext';
import { StudentFullAcademicHistoryDTO, YearAcademicHistory, SemesterAcademicHistory } from '@eduguard/shared';

export default function StudentDetailPage() {
  const params = useParams();
  const { isAdmin, isTeacher } = useAuth();
  const studentId = params?.id as string;

  const [student, setStudent] = useState<any | null>(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);

  // Prediction execution state
  const [predicting, setPredicting] = useState(false);
  const [predictSuccess, setPredictSuccess] = useState<string | null>(null);
  const [predictError, setPredictError] = useState<string | null>(null);

  // Intervention & Metric Detail modal
  const [showInterventionModal, setShowInterventionModal] = useState(false);
  const [activeMetricModal, setActiveMetricModal] = useState<string | null>(null);

  // Accordion state: Year & Semester open/close
  // Active semester auto-expanded
  const [expandedYears, setExpandedYears] = useState<Record<number, boolean>>({});
  const [expandedSemesters, setExpandedSemesters] = useState<Record<number, boolean>>({});

  const fetchProfile = async () => {
    setLoading(true);
    setError(null);
    try {
      const res = await studentsApi.getStudentById(studentId);
      if (res.data) {
        setStudent(res.data);

        // Auto-expand current year and semester
        const curYear = res.data.year || Math.ceil((res.data.semester || 1) / 2);
        const curSem = res.data.semester || 1;
        setExpandedYears((prev) => ({ ...prev, [curYear]: true }));
        setExpandedSemesters((prev) => ({ ...prev, [curSem]: true }));
      }
    } catch (err: any) {
      setError(err.response?.data?.message || 'Failed to load student profile.');
```

```

});

useEffect(() => {
  if (studentId) {
    fetchProfile();
  }
}, [studentId]);

const toggleYear = (yearNum: number) => {
  setExpandedYears((prev) => ({ ...prev, [yearNum]: !prev[yearNum] }));
};

const toggleSemester = (semNum: number) => {
  setExpandedSemesters((prev) => ({ ...prev, [semNum]: !prev[semNum] }));
};

const handleRunPrediction = async () => {
  setPredicting(true);
  setPredictSuccess(null);
  setPredictError(null);

  try {
    const res = await predictionsApi.generate(student._id);
    if (res.data) {
      setPredictSuccess('AI Prediction updated successfully.');
```

```
      await fetchProfile();
      setTimeout(() => setPredictSuccess(null), 4000);
    }
  } catch (err: any) {
    const msg =
      err.response?.status === 503
        ? 'AI prediction microservice is starting up. Please try again in a few seconds.'
        : err.response?.data?.message || 'Unable to generate prediction. Please check student records.';
    setPredictError(msg);
    setTimeout(() => setPredictError(null), 6000);
  } finally {
    setPredicting(false);
  }
};

if (loading) {
  return (
    <AppLayout>
      <div className="space-y-6 max-w-5xl">
        <LoadingSkeleton rows={6} />
      </div>
    </AppLayout>
  );
}

if (error || !student) {
  return (
    <AppLayout>
      <div className="rounded-xl border border-red-200 bg-red-50 p-6 text-center max-w-lg mx-auto mt-12">
        <p className="text-sm font-semibold text-red-700">{error || 'Student not found.'}</p>
        <Link
          href="/students"
          className="mt-4 inline-flex items-center gap-1.5 text-xs font-semibold text-blue-600 hover:text-blue-800"
        >
          <ArrowLeft className="h-4 w-4" /> Back to Students
        </Link>
      </div>
    </AppLayout>
  );
}

const latestPred = student.latestPrediction;
const calc = student?.calculatedMetrics || {};
const academicHistory: StudentFullAcademicHistoryDTO = student.academicHistory;

const score = latestPred ? Math.round(latestPred.predictedScore) : null;
const risk = latestPred?.riskLevel;

// Extract human-friendly areas of attention
const getAttentionPoints = (): string[] => {
  if (!latestPred || !latestPred.riskFactors || latestPred.riskFactors.length === 0) {
    const att = calc.attendance?.value ?? 100;
    const internal = calc.internalMarks?.value ?? 100;
    const assign = calc.assignmentCompletion?.value ?? 100;
  }
}
```

```

const points: string[] = [];
if (att < 75) points.push(`Low attendance (${calc.attendance?.formattedValue || att + '%'})`);
if (internal < 50) points.push(`Low assessment performance (${calc.internalMarks?.formattedValue || internal + '%'})`);
if (assign < 70) points.push(`Low assignment completion (${calc.assignmentCompletion?.formattedValue || assign + '%'})`);
return points.length > 0 ? points : ['All primary metrics meet institutional targets'];
}

return latestPred.riskFactors.map((rf: any) => {
  const f = rf.factor.toLowerCase();
  if (f.includes('attendance')) return 'Low attendance in lectures and practical sessions';
  if (f.includes('internal')) return 'Low assessment performance in internal tests';
  if (f.includes('previous')) return 'Prior semester academic performance is below target';
  if (f.includes('assignment')) return 'Pending or incomplete coursework assignments';
  if (f.includes('study')) return 'Low weekly independent self-study hours';
  if (f.includes('participation')) return 'Low active in-class participation';
  return rf.description || rf.factor;
});
};

const getRecommendations = (): string[] => {
  if (latestPred?.recommendations && latestPred.recommendations.length > 0) {
    return latestPred.recommendations;
  }
  return [
    'Discuss attendance with the student and identify scheduling barriers',
    'Encourage completion and submission of pending coursework assignments',
    'Provide tutorial support and review difficult subject concepts',
    'Suggest a structured weekly independent study timetable',
  ];
};

const attentionPoints = getAttentionPoints();
const recommendations = getRecommendations();

const standardAssessments = [
  { key: 'Mid 1', label: 'Mid 1', defaultMax: 25 },
  { key: 'Mid 2', label: 'Mid 2', defaultMax: 25 },
  { key: 'Internal Lab 1', label: 'Internal Lab 1', defaultMax: 10 },
  { key: 'Internal Lab 2', label: 'Internal Lab 2', defaultMax: 10 },
  { key: 'External Lab', label: 'External Lab', defaultMax: 25 },
  { key: 'Assignment', label: 'Assignment', defaultMax: 10 },
];

return (
  <AppLayout>
    <div className="space-y-6 max-w-5xl mx-auto pb-12">
      { /* ===== */ }
      { /* TOP PROFILE HEADER */ }
      { /* ===== */ }
      <div className="rounded-2xl border border-slate-200 bg-white p-6 shadow-xs">
        <div className="flex flex-col sm:flex-row sm:items-center sm:justify-between gap-4">
          <div className="flex items-start gap-4">
            <Link
              href="/students"
              className="mt-1 rounded-lg border border-slate-200 bg-white p-2 text-slate-500 hover:bg-slate-50 hover:text-slate-800 transition-colors"
              title="Back to Students list"
            >
              <ArrowLeft className="h-4 w-4" />
            </Link>
          </div>
          <div className="flex items-center gap-3 flex-wrap">
            <h1 className="text-2xl font-bold tracking-tight text-slate-900">{student.name}</h1>
            <span className="inline-flex items-center rounded-md bg-slate-100 px-2.5 py-0.5 text-xs font-mono font-bold text-slate-700 border border-slate-200">
              {student.studentId}
            </span>
          </div>
          <div className="flex items-center gap-2 text-xs font-medium text-slate-600 mt-1.5 flex-wrap">
            <span className="font-semibold text-blue-700 bg-blue-50 px-2 py-0.5 rounded border border-blue-100">
              {student.program || student.course || 'B.Tech'}
            </span>
            <span>&bull;</span>
            <span>{student.department}</span>
            <span>&bull;</span>
            <span>{student.year} Year</span>
            <span>&bull;</span>
            <span>Semester {student.semester}</span>
            <span>&bull;</span>
          </div>
        </div>
      </div>
    </div>
  </AppLayout>
);

```

```

        <span>Section {student.section}</span>
      </div>
    </div>
  </div>

  <div className="flex items-center gap-2">
    {(isAdmin || isTeacher) && (
      <>
        <button
          type="button"
          onClick={handleRunPrediction}
          disabled={predicting}
          className="inline-flex items-center gap-1.5 rounded-lg border border-blue-200 bg-blue-50 px-3.5 py-2 text-xs
font-semibold text-blue-700 shadow-xs hover:bg-blue-100 transition-colors disabled:opacity-60"
        >
          <Play className={`h-3.5 w-3.5 ${predicting ? 'animate-spin' : ''}`} />
          <span>{predicting ? 'Running ANN...' : 'Run AI Prediction'}</span>
        </button>

        <button
          type="button"
          onClick={() => setShowInterventionModal(true)}
          className="inline-flex items-center gap-1.5 rounded-lg bg-blue-600 px-3.5 py-2 text-xs font-semibold text-wh
ite shadow-xs hover:bg-blue-700 transition-colors"
        >
          <PlusCircle className="h-3.5 w-3.5" />
          <span>Add Intervention</span>
        </button>
      </>
    )}
  </div>
</div>

{ /* Feedback Banners */}
{predictSuccess && (
  <div className="mt-4 rounded-xl border border-emerald-200 bg-emerald-50 p-3.5 text-xs font-semibold text-emerald-800
flex items-center gap-2">
    <CheckCircle2 className="h-4 w-4 text-emerald-600 shrink-0" />
    <span>{predictSuccess}</span>
  </div>
)}
{predictError && (
  <div className="mt-4 rounded-xl border border-red-200 bg-red-50 p-3.5 text-xs font-semibold text-red-800 flex items-
center gap-2">
    <AlertCircle className="h-4 w-4 text-red-600 shrink-0" />
    <span>{predictError}</span>
  </div>
)}
</div>

{ /* ===== */}
{ /* CURRENT PERFORMANCE SUMMARY (6 Academic Features) */}
{ /* ===== */}
<div>
  <h2 className="text-xs font-bold uppercase tracking-wider text-slate-500 mb-3 px-1">
    Current Performance
  </h2>
  <div className="grid grid-cols-2 md:grid-cols-3 lg:grid-cols-6 gap-3">
    { /* 1. Attendance */}
    <div className="rounded-xl border border-slate-200 bg-white p-4 shadow-xs flex flex-col justify-between hover:border
-blue-300 transition-colors">
      <div>
        <span className="text-[11px] font-semibold text-slate-500 flex items-center gap-1">
          <CalendarCheck className="h-3.5 w-3.5 text-blue-600" /> Attendance
        </span>
        <p className="text-xl font-bold text-slate-900 mt-2">
          {calc.attendance?.formattedValue || '-'}
        </p>
        <p className="text-[11px] text-slate-500 mt-1">
          {calc.attendance?.subtext || 'From attendance logs'}
        </p>
      </div>
      <button
        type="button"
        onClick={() => setActiveMetricModal('attendance')}
        className="mt-3 inline-flex items-center gap-1 text-[11px] font-bold text-blue-600 hover:text-blue-800 transitio
n-colors pt-2 border-t border-slate-100"
      >
        <span>View Attendance</span>
      </button>
    </div>
  </div>

```

```

        <ArrowRight className="h-3 w-3" />
      </button>
    </div>

    /* 2. Previous Performance */
    <div className="rounded-xl border border-slate-200 bg-white p-4 shadow-xs flex flex-col justify-between">
      <div>
        <span className="text-[11px] font-semibold text-slate-500 flex items-center gap-1">
          <GraduationCap className="h-3.5 w-3.5 text-blue-600" /> Previous Marks
        </span>
        <p className="text-xl font-bold text-slate-900 mt-2">
          {calc.previousScore?.formattedValue || '-'}
        </p>
        <p className="text-[11px] text-slate-500 mt-1">
          {calc.previousScore?.subtext || 'From previous semesters'}
        </p>
      </div>
    </div>

    /* 3. Current Assessment Performance */
    <div className="rounded-xl border border-slate-200 bg-white p-4 shadow-xs flex flex-col justify-between">
      <div>
        <span className="text-[11px] font-semibold text-slate-500 flex items-center gap-1">
          <Award className="h-3.5 w-3.5 text-blue-600" /> Assessment Marks
        </span>
        <p className="text-xl font-bold text-slate-900 mt-2">
          {calc.internalMarks?.formattedValue || '-'}
        </p>
        <p className="text-[11px] text-slate-500 mt-1">
          Assessment Performance
        </p>
      </div>
    </div>

    /* 4. Assignments */
    <div className="rounded-xl border border-slate-200 bg-white p-4 shadow-xs flex flex-col justify-between">
      <div>
        <span className="text-[11px] font-semibold text-slate-500 flex items-center gap-1">
          <FileCheck className="h-3.5 w-3.5 text-blue-600" /> Assignments
        </span>
        <p className="text-xl font-bold text-slate-900 mt-2">
          {calc.assignmentCompletion?.formattedValue || '-'}
        </p>
        <p className="text-[11px] text-slate-500 mt-1">
          {calc.assignmentCompletion?.subtext || 'Course assignments'}
        </p>
      </div>
    </div>

    /* 5. Study Hours */
    <div className="rounded-xl border border-slate-200 bg-white p-4 shadow-xs flex flex-col justify-between">
      <div>
        <span className="text-[11px] font-semibold text-slate-500 flex items-center gap-1">
          <Clock className="h-3.5 w-3.5 text-blue-600" /> Study Hours
        </span>
        <p className="text-xl font-bold text-slate-900 mt-2">
          {calc.studyHours?.formattedValue || '-'}
        </p>
        <p className="text-[11px] text-slate-500 mt-1">
          {calc.studyHours?.subtext || 'Weekly hours logged'}
        </p>
      </div>
    </div>

    /* 6. Participation */
    <div className="rounded-xl border border-slate-200 bg-white p-4 shadow-xs flex flex-col justify-between">
      <div>
        <span className="text-[11px] font-semibold text-slate-500 flex items-center gap-1">
          <Activity className="h-3.5 w-3.5 text-blue-600" /> Participation
        </span>
        <p className="text-xl font-bold text-slate-900 mt-2">
          {calc.participation?.formattedValue || '-'}
        </p>
        <p className="text-[11px] text-slate-500 mt-1">
          {calc.participation?.subtext || 'Class engagement'}
        </p>
      </div>
    </div>
  </div>

```

```

</div>

{ /* ===== */
{ /* AI PERFORMANCE PREDICTION */
{ /* ===== */
<div className="rounded-2xl border border-slate-200 bg-white p-6 shadow-xs space-y-5">
  <div className="flex items-center justify-between border-b border-slate-100 pb-3">
    <div>
      <h3 className="text-base font-bold text-slate-900 flex items-center gap-2">
        <Sparkles className="h-4 w-4 text-blue-600" />
        <span>AI Performance Prediction</span>
      </h3>
      <p className="text-xs text-slate-500 mt-0.5">
        Neural network score estimate based on student attendance and academic records.
      </p>
    </div>
    {latestPred?.createdAt && (
      <span className="text-[11px] text-slate-400">
        Last Evaluated: {new Date(latestPred.createdAt).toLocaleDateString()}
      </span>
    )}
  </div>

  {latestPred ? (
    <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
      <div className="rounded-xl border border-slate-200 bg-slate-50/70 p-5 flex flex-col justify-between">
        <span className="text-xs font-bold uppercase tracking-wider text-slate-500">
          Expected Score
        </span>
        <div className="mt-2">
          <span className="text-4xl font-extrabold text-slate-900">{score}</span>
          <span className="text-base font-semibold text-slate-400"> / 100</span>
        </div>
        <p className="text-xs text-slate-500 mt-2">
          Model estimation for end-of-semester overall academic performance.
        </p>
      </div>

      <div className="rounded-xl border border-slate-200 bg-slate-50/70 p-5 flex flex-col justify-between">
        <span className="text-xs font-bold uppercase tracking-wider text-slate-500">
          Risk Level
        </span>
        <div className="mt-2">
          {risk === 'HIGH' ? (
            <span className="inline-flex items-center gap-1.5 px-3.5 py-1.5 rounded-full text-xs font-bold bg-red-100 text-red-800 border border-red-200">
              <span>●</span>
              <span>HIGH RISK</span>
            </span>
          ) : risk === 'MEDIUM' ? (
            <span className="inline-flex items-center gap-1.5 px-3.5 py-1.5 rounded-full text-xs font-bold bg-amber-100 text-amber-800 border border-amber-200">
              <span>●</span>
              <span>MEDIUM RISK</span>
            </span>
          ) : (
            <span className="inline-flex items-center gap-1.5 px-3.5 py-1.5 rounded-full text-xs font-bold bg-emerald-100 text-emerald-800 border border-emerald-200">
              <span>●</span>
              <span>LOW RISK</span>
            </span>
          )}
        </div>
        <p className="text-xs text-slate-500 mt-2">
          {risk === 'HIGH'
            ? 'Student requires prompt academic support and faculty mentoring.'
            : risk === 'MEDIUM'
            ? 'Student is on the watchlist; monitor upcoming assessment submissions.'
            : 'Student is demonstrating consistent on-track academic progress.'}
        </p>
      </div>
    </div>
  ) : (
    <div className="text-center py-6 text-xs text-slate-500 bg-slate-50 rounded-xl">
      No prediction generated yet. Click &quot;Run AI Prediction&quot; to evaluate this student.
    </div>
  )}

  { /* Areas That May Need Attention & Recommended Support */}

```

```
s text-left"
```

```

    { /* Semesters inside Year */}
    {isYearExpanded && (
      <div className="p-4 space-y-4 divide-y divide-slate-100">
        {yearItem.semesters.map((semItem: SemesterAcademicHistory) => {
          const isSemExpanded = !!expandedSemesters[semItem.semester];
          const isCurrentSemester = semItem.semester === student.semester;

          return (
            <div key={semItem.semester} className="pt-3 first:pt-0">
              { /* Semester Header */}
              <div
                onClick={() => toggleSemester(semItem.semester)}
                className="flex items-center justify-between cursor-pointer py-2 px-3 rounded-lg bg-slate-50/70
                hover:bg-blue-50/50 transition-colors"
              >
                <div className="flex items-center gap-2">
                  {isSemExpanded ? (
                    <ChevronDown className="h-3.5 w-3.5 text-blue-600" />
                  ) : (
                    <ChevronRight className="h-3.5 w-3.5 text-blue-600" />
                  )}
                  <span className="text-xs font-bold text-slate-800">
                    Semester {semItem.semester}
                  </span>
                  {isCurrentSemester && (
                    <span className="rounded-full bg-blue-100 text-blue-700 text-[10px] font-semibold px-2 py-0.
5">
                      Active Semester
                    </span>
                  )}
                </div>
                <div className="text-xs text-slate-600">
                  {semItem.totalMaximum > 0 ? (
                    <span className="font-semibold text-slate-700">
                      Assessment Performance: {semItem.assessmentPerformance}%
                    </span>
                  ) : (
                    <span className="text-slate-400 italic">No marks recorded</span>
                  )}
                </div>
              </div>
            </div>
          )}
        })}

    { /* Subjects and Marks */}
    {isSemExpanded && (
      <div className="mt-3 space-y-3 pl-2 sm:pl-4">
        {semItem.subjects.length > 0 ? (
          semItem.subjects.map((sub) => (
            <div
              key={sub.subjectId}
              className="rounded-lg border border-slate-200 bg-white p-3.5 space-y-2.5 shadow-2xs"
            >
              <div className="flex items-center justify-between flex-wrap gap-2">
                <div>
                  <span className="text-xs font-bold text-slate-900">
                    {sub.name}
                  </span>
                  <span className="ml-2 font-mono text-[11px] text-slate-500">
                    ({sub.code})
                  </span>
                </div>
                <div>
                  {sub.totalMaximum > 0 ? (
                    <span className="text-xs font-bold text-blue-700 bg-blue-50 px-2 py-0.5 rounded bord
er border-blue-100">
                      Assessment: {sub.totalObtained} / {sub.totalMaximum} ({sub.assessmentPerformanc
e}%)
                    </span>
                  ) : (
                    <span className="text-[11px] text-slate-400 italic">
                      No marks entered
                    </span>
                  )}
                </div>
              </div>
            </div>
          )}
        })}

        { /* 6 Standard Assessments Grid */}
        <div className="grid grid-cols-2 sm:grid-cols-3 lg:grid-cols-6 gap-2 pt-1 text-xs">
          {standardAssessments.map((sa) => {
            const mark = sub.marks[sa.key];
            return (
              <div

```

```

        key={sa.key}
        className={`p-2 rounded-md border text-center ${
          mark
            ? 'bg-slate-50/80 border-slate-200 text-slate-800'
            : 'bg-slate-50/30 border-slate-100 text-slate-400'
        }`}
      >
        <div className="text-[11px] font-medium text-slate-500">
          {sa.label}
        </div>
        <div className="font-semibold text-xs mt-0.5">
          {mark ? (
            <span>
              {mark.obtainedMarks}{' '}
              <span className="text-slate-400 font-normal">
                / {mark.maximumMarks}
              </span>
            </span>
          ) : (
            <span className="text-slate-400">--</span>
          )}
        </div>
      </div>
    );
  }}}
</div>
</div>
))
) : (
  <p className="text-xs text-slate-400 italic py-2">
    No curriculum subjects scheduled for this semester.
  </p>
)
</div>
))
</div>
);
)}}
</div>
))
</div>
);
)}}
</div>
) : (
  <p className="text-xs text-slate-500 italic py-4 text-center">
    No academic history records found.
  </p>
)
</div>

{/* ===== */}
{/* INTERVENTION HISTORY */}
{/* ===== */}
<div className="rounded-2xl border border-slate-200 bg-white p-6 shadow-xs space-y-4">
  <div className="flex items-center justify-between border-b border-slate-100 pb-3">
    <div>
      <h3 className="text-base font-bold text-slate-900">Intervention History</h3>
      <p className="text-xs text-slate-500 mt-0.5">
        Past and ongoing academic support interventions created by faculty.
      </p>
    </div>
    {(isAdmin || isTeacher) && (
      <button
        type="button"
        onClick={() => setShowInterventionModal(true)}
        className="inline-flex items-center gap-1.5 rounded-lg border border-slate-200 bg-slate-50 px-3 py-1.5 text-xs font-semibold text-slate-700 hover:bg-slate-100 transition-colors"
      >
        <PlusCircle className="h-3.5 w-3.5" />
        <span>Add Intervention</span>
      </button>
    )}
  </div>

  {student.interventions && student.interventions.length > 0 ? (
    <div className="divide-y divide-slate-100">
      {student.interventions.map((inv: any) => (
        <div key={inv._id} className="py-3 text-xs space-y-1">

```

```

        <div className="flex items-center justify-between">
          <span className="font-bold text-slate-900 text-sm">{inv.title}</span>
          <span
            className={`inline-flex items-center rounded-full px-2.5 py-0.5 text-[11px] font-semibold ${
              inv.status === 'COMPLETED'
                ? 'bg-emerald-50 text-emerald-700 border border-emerald-200'
                : inv.status === 'IN_PROGRESS'
                ? 'bg-amber-50 text-amber-700 border border-amber-200'
                : 'bg-blue-50 text-blue-700 border border-blue-200'
            }`}
          >
            {inv.status}
          </span>
        </div>
        <p className="text-slate-600">{inv.description}</p>
        <div className="flex items-center gap-4 text-[11px] text-slate-400 pt-1 flex-wrap">
          <span>Type: {inv.type?.replace(/_/g, ' ')}</span>
          {inv.dueDate && (
            <span>Due: {new Date(inv.dueDate).toLocaleDateString()}</span>
          )}
          {inv.outcome && (
            <span className="text-emerald-700 font-semibold">
              Outcome: {inv.outcome}
            </span>
          )}
        </div>
      </div>
    ) : (
      <p className="text-xs text-slate-500 italic py-4 text-center">
        No interventions currently scheduled for this student.
      </p>
    )}
  </div>
</div>

{/* Intervention Modal */}
{showInterventionModal && (
  <InterventionModal
    studentId={student._id}
    studentName={student.name}
    predictionId={latestPred?._id}
    onSuccess={() => {
      fetchProfile();
      setShowInterventionModal(false);
    }}
    onClose={() => setShowInterventionModal(false)}
  />
)}

{/* Metric Detail Modal (Requirements 13 & 14: Subject-wise breakdown & History) */}
{activeMetricModal && (
  <MetricDetailModal
    metricKey={activeMetricModal}
    calculatedData={calc[activeMetricModal]}
    onClose={() => setActiveMetricModal(null)}
  />
)}
</AppLayout>
);
}

```

A.1.4 Attendance Management

Cohort Attendance Recording Sheet

File: apps/web/src/app/attendance/page.tsx

TYPESCRIPT

Purpose & Architecture: Interactive classroom attendance marking interface adhering to cohort hierarchy (Program -> Department -> Year -> Semester -> Section -> Subject -> Date). Supports Present/Absent/Not Marked status toggles, calculates real-time attendance percentage, and commits updates via batch API.

```

'use client';

import React, { useState, useEffect, useCallback } from 'react';
import { AppLayout } from '@components/layout/AppLayout';
import { LoadingSkeleton } from '@components/ui/LoadingSkeleton';
import { EmptyState } from '@components/ui/EmptyState';
import {
  attendanceApi,
  subjectsApi,
  programsApi,
  departmentsApi,
} from '@lib/apiClient';
import { useAuth } from '@context/AuthContext';
import {
  CalendarCheck,
  CheckCircle2,
  AlertCircle,
  Save,
  Users,
  Calendar,
  Filter,
  RotateCcw,
  Sparkles,
} from 'lucide-react';

interface SheetStudent {
  studentId: string;
  studentNumber: string;
  studentName: string;
  department?: string;
  section?: string;
  semester?: number;
  status: 'PRESENT' | 'ABSENT' | 'NOT_MARKED';
  currentAttendance?: {
    totalClasses: number;
    presentClasses: number;
    absentClasses: number;
    percentage: number;
    formatted: string;
  };
};

export default function AttendancePage() {
  const { isTeacher, isAdmin } = useAuth();

  const [programs, setPrograms] = useState<any[]>([]);
  const [departments, setDepartments] = useState<any[]>([]);
  const [subjects, setSubjects] = useState<any[]>([]);

  // Selection states
  const [selectedProgram, setSelectedProgram] = useState('B.Tech');
  const [selectedDepartment, setSelectedDepartment] = useState('Computer Science');
  const [selectedYear, setSelectedYear] = useState<number>(2);
  const [selectedSemester, setSelectedSemester] = useState<number>(3);
  const [selectedSection, setSelectedSection] = useState('A');
  const [selectedSubjectId, setSelectedSubjectId] = useState<string>('');

  // Default to 2026-09-01 (start of active lecture term with full 30-day attendance history)
  const [selectedDate, setSelectedDate] = useState<string>('2026-09-01');

  const [sheetLoading, setSheetLoading] = useState(false);
  const [saving, setSaving] = useState(false);
  const [message, setMessage] = useState<{ text: string; isError: boolean } | null>(null);

  // Student rows in attendance sheet
  const [studentsSheet, setStudentsSheet] = useState<SheetStudent[]>([]);

  // Initial load: programs & departments metadata
  useEffect(() => {
    async function loadMeta() {
      try {
        const [progRes, deptRes] = await Promise.all([
          programsApi.getPrograms(),
          departmentsApi.getDepartments(),
        ]);
        if (progRes.data && progRes.data.length > 0) {
          setPrograms(progRes.data);
        }
      }
    }
  });

```

```

    if (deptRes.data && deptRes.data.length > 0) {
      setDepartments(deptRes.data);
      const matched = deptRes.data.find((d: any) => d.program === 'B.Tech') || deptRes.data[0];
      setSelectedDepartment(matched.name);
    }
  } catch (e: any) {
    console.error('Failed to load initial metadata', e);
  }
}
loadMeta();
}, []);

// Derived options based on program duration
const filteredDepartments = departments.filter((d) => d.program === selectedProgram);
const currentProgramDoc = programs.find((p) => p.name === selectedProgram);
const maxYears = currentProgramDoc?.durationYears || (selectedProgram === 'B.Tech' ? 4 : 3);
const yearOptions = Array.from({ length: maxYears }, (_, i) => i + 1);
const semesterOptions = [(selectedYear - 1) * 2 + 1, (selectedYear - 1) * 2 + 2];

// Program change handler ensuring dependent state is updated
const handleProgramChange = (progName: string) => {
  setSelectedProgram(progName);
  const progDepts = departments.filter((d) => d.program === progName);
  if (progDepts.length > 0) {
    setSelectedDepartment(progDepts[0].name);
  }
  setSelectedYear(1);
  setSelectedSemester(1);
};

// Year change handler updating dependent semester options
const handleYearChange = (yearNum: number) => {
  setSelectedYear(yearNum);
  const newSem = (yearNum - 1) * 2 + 1;
  setSelectedSemester(newSem);
};

// Fetch subjects when program, department, semester change
useEffect(() => {
  async function loadSubjects() {
    if (!selectedProgram || !selectedDepartment || !selectedSemester) return;
    try {
      const res = await subjectsApi.getSubjects({
        program: selectedProgram,
        department: selectedDepartment,
        semester: selectedSemester,
      });
      if (res.data && res.data.length > 0) {
        setSubjects(res.data);
        setSelectedSubjectId(res.data[0]._id);
      } else {
        setSubjects([]);
        setSelectedSubjectId('');
      }
    } catch (e) {
      console.error('Failed to load subjects', e);
    }
  }
  loadSubjects();
}, [selectedProgram, selectedDepartment, selectedSemester]);

// Fetch attendance sheet from MongoDB for selected parameters and date
const loadSheet = useCallback(async () => {
  if (!selectedDepartment || !selectedSemester || !selectedSection || !selectedDate) return;

  setSheetLoading(true);
  setMessage(null);
  try {
    const res = await attendanceApi.getSheet({
      program: selectedProgram,
      department: selectedDepartment,
      year: selectedYear,
      semester: selectedSemester,
      section: selectedSection,
      subjectId: selectedSubjectId || undefined,
      date: selectedDate,
    });
    if (res.data && Array.isArray(res.data.students)) {

```

```

        setStudentsSheet(res.data.students);
    } else {
        setStudentsSheet([]);
    }
} catch (err: any) {
    setMessage({
        text: err.response?.data?.message || 'Failed to load attendance sheet.',
        isError: true,
    });
} finally {
    setSheetLoading(false);
}
}, [
    selectedProgram,
    selectedDepartment,
    selectedYear,
    selectedSemester,
    selectedSection,
    selectedSubjectId,
    selectedDate,
]);

useEffect(() => {
    loadSheet();
}, [loadSheet]);

const handleToggleStatus = (studentId: string, newStatus: 'PRESENT' | 'ABSENT') => {
    setStudentsSheet((prev) =>
        prev.map((s) => (s.studentId === studentId ? { ...s, status: newStatus } : s))
    );
};

const handleMarkAll = (status: 'PRESENT' | 'ABSENT') => {
    setStudentsSheet((prev) => prev.map((s) => ({ ...s, status })));
};

const handleSaveAttendance = async () => {
    if (!selectedSubjectId) {
        setMessage({ text: 'Please select a subject to record attendance.', isError: true });
        return;
    }

    // Filter only students that have been marked
    const markedStudents = studentsSheet.filter(
        (s) => s.status === 'PRESENT' || s.status === 'ABSENT'
    );

    if (markedStudents.length === 0) {
        setMessage({
            text: 'No attendance marked yet. Please mark at least one student before saving.',
            isError: true,
        });
        return;
    }

    setSaving(true);
    setMessage(null);
    try {
        const records = markedStudents.map((s) => ({
            studentId: s.studentId,
            status: s.status,
        }));

        await attendanceApi.saveBatch({
            subjectId: selectedSubjectId,
            date: selectedDate,
            records,
        });

        setMessage({
            text: `Attendance saved successfully for ${records.length} students on ${selectedDate}. MongoDB updated.`,
            isError: false,
        });

        // Reload sheet to reflect updated statistics from MongoDB
        await loadSheet();

        setTimeout(() => setMessage(null), 5000);
    } catch (err: any) {

```

```

    setMessage({
      text: err.response?.data?.message || 'Failed to save attendance records.',
      isError: true,
    });
  } finally {
    setSaving(false);
  }
};

// Calculations for session status bar (Requirement 14)
const totalCount = studentsSheet.length;
const presentCount = studentsSheet.filter((s) => s.status === 'PRESENT').length;
const absentCount = studentsSheet.filter((s) => s.status === 'ABSENT').length;
const notMarkedCount = studentsSheet.filter((s) => s.status === 'NOT_MARKED').length;
// Day Attendance: Present / Total Enrolled x 100
const dayAttendancePct =
  totalCount > 0 ? Math.round((presentCount / totalCount) * 1000) / 10 : 0;

return (
  <AppLayout>
    <div className="space-y-6 max-w-7xl mx-auto">
      {/* ===== */}
      {/* DEMO DATA NOTICE (Requirement 22) */}
      {/* ===== */}
      <div className="flex items-center justify-between rounded-xl border border-amber-300 bg-amber-50/80 px-4 py-2.5 text-xs text-amber-900 shadow-2xs">
        <div className="flex items-center gap-2">
          <span className="inline-flex items-center rounded-md bg-amber-200 px-2 py-0.5 font-bold text-amber-950 uppercase tracking-wider text-[10px]">
            DEMO DATA
          </span>
          <span>
            Real-time date-based attendance with MongoDB persistence. Each student maintains realistic individual attendance percentages.
          </span>
        </div>
        <span className="hidden sm:inline text-[11px] font-mono font-medium text-amber-700">
          Active Term: Sep - Oct 2026
        </span>
      </div>

      {/* Page Header */}
      <div className="flex flex-col sm:flex-row sm:items-center sm:justify-between gap-4">
        <div>
          <h1 className="text-2xl font-bold tracking-tight text-slate-900">
            Attendance Management
          </h1>
          <p className="text-sm text-slate-500 mt-1">
            Select date and cohort to view or record lecture attendance. Stored directly in MongoDB.
          </p>
        </div>

        <div className="flex items-center gap-2">
          <button
            type="button"
            onClick={loadSheet}
            disabled={sheetLoading}
            className="inline-flex items-center gap-1.5 rounded-lg border border-slate-200 bg-white px-3 py-2 text-xs font-semibold text-slate-700 hover:bg-slate-50 shadow-2xs transition-colors">
            <RotateCcw className={`h-3.5 w-3.5 text-slate-500 ${sheetLoading ? 'animate-spin' : ''}`} />
            <span>Refresh Date</span>
          </button>
        </div>
      </div>

      {/* ===== */}
      {/* SELECTION BAR */}
      {/* ===== */}
      <div className="rounded-2xl border border-slate-200 bg-white p-5 shadow-xs space-y-4">
        <div className="flex items-center gap-2 text-xs font-bold uppercase tracking-wider text-slate-500">
          <Filter className="h-4 w-4 text-blue-600" />
          <span>Select Class, Subject & Lecture Date</span>
        </div>

        <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-4 gap-3 text-xs">
          {/* Program */}
          <div>
            <label className="block text-slate-600 font-semibold mb-1">Program</label>

```

```

        <select
          value={selectedProgram}
          onChange={(e) => handleProgramChange(e.target.value)}
          className="w-full rounded-lg border border-slate-200 p-2 text-xs text-slate-800 font-medium focus:border-blue-50
0 focus:outline-none"
        >
          {programs.map((p) => (
            <option key={p.name} value={p.name}>
              {p.name} ({p.durationYears} Years)
            </option>
          ))}
        </select>
      </div>

      {/* Department */}
      <div>
        <label className="block text-slate-600 font-semibold mb-1">Department</label>
        <select
          value={selectedDepartment}
          onChange={(e) => setSelectedDepartment(e.target.value)}
          className="w-full rounded-lg border border-slate-200 p-2 text-xs text-slate-800 focus:border-blue-500 focus:outl
ine-none"
        >
          {filteredDepartments.length > 0 ? (
            filteredDepartments.map((d) => (
              <option key={d._id || d.name} value={d.name}>
                {d.name} {d.code && d.code !== d.name ? `(${d.code})` : ''}
              </option>
            ))
          ) : (
            <option value="Computer Science">Computer Science</option>
          )}
        </select>
      </div>

      {/* Year (Dependent on Program duration) */}
      <div>
        <label className="block text-slate-600 font-semibold mb-1">Year</label>
        <select
          value={selectedYear}
          onChange={(e) => handleYearChange(Number(e.target.value))}
          className="w-full rounded-lg border border-slate-200 p-2 text-xs text-slate-800 font-medium focus:border-blue-50
0 focus:outline-none"
        >
          {yearOptions.map((y) => (
            <option key={y} value={y}>
              {y === 1 ? '1st Year' : y === 2 ? '2nd Year' : y === 3 ? '3rd Year' : `${y}th Year`}
            </option>
          ))}
        </select>
      </div>

      {/* Semester (Dependent on Year) */}
      <div>
        <label className="block text-slate-600 font-semibold mb-1">Semester</label>
        <select
          value={selectedSemester}
          onChange={(e) => setSelectedSemester(Number(e.target.value))}
          className="w-full rounded-lg border border-slate-200 p-2 text-xs text-slate-800 focus:border-blue-500 focus:outl
ine-none"
        >
          {semesterOptions.map((s) => (
            <option key={s} value={s}>
              Semester {s}
            </option>
          ))}
        </select>
      </div>

      {/* Section */}
      <div>
        <label className="block text-slate-600 font-semibold mb-1">Section</label>
        <select
          value={selectedSection}
          onChange={(e) => setSelectedSection(e.target.value)}
          className="w-full rounded-lg border border-slate-200 p-2 text-xs text-slate-800 focus:border-blue-500 focus:outl
ine-none"
        >
          <option value="A">Section A</option>

```

```

        <option value="B">Section B</option>
        <option value="C">Section C</option>
      </select>
    </div>

    {/ * Subject */}
    <div>
      <label className="block text-slate-600 font-semibold mb-1">Subject</label>
      <select
        value={selectedSubjectId}
        onChange={(e) => setSelectedSubjectId(e.target.value)}
        className="w-full rounded-lg border border-slate-200 p-2 text-xs text-slate-800 font-medium focus:border-blue-500 focus:outline-none"
      >
        {subjects.length > 0 ? (
          subjects.map((sub) => (
            <option key={sub._id} value={sub._id}>
              {sub.name} ({sub.code})
            </option>
          ))
        ) : (
          <option value="">No subjects found</option>
        )}
      </select>
    </div>

    {/ * Lecture Date Picker */}
    <div className="sm:col-span-2">
      <label className="block text-slate-600 font-semibold mb-1">
        Lecture Date (Select any date to view historical attendance)
      </label>
      <div className="relative">
        <Calendar className="pointer-events-none absolute left-3 top-2.5 h-4 w-4 text-blue-600" />
        <input
          type="date"
          value={selectedDate}
          onChange={(e) => setSelectedDate(e.target.value)}
          className="w-full rounded-lg border border-slate-200 pl-9 pr-3 py-2 text-xs font-bold text-slate-900 focus:border-blue-500 focus:outline-none bg-slate-50/50"
        />
      </div>
    </div>
  </div>
</div>

{/ * ===== */}
{/ * SESSION STATS BAR (Requirement 2 & 7) */}
{/ * ===== */}
<div className="grid grid-cols-2 sm:grid-cols-5 gap-3">
  <div className="rounded-xl border border-slate-200 bg-white p-3.5 shadow-xs">
    <span className="text-[11px] font-semibold text-slate-400">Total Enrolled</span>
    <p className="text-xl font-bold text-slate-900 mt-1">{totalCount}</p>
  </div>
  <div className="rounded-xl border border-emerald-100 bg-emerald-50/30 p-3.5 shadow-xs">
    <span className="text-[11px] font-semibold text-emerald-600">Present</span>
    <p className="text-xl font-bold text-emerald-600 mt-1">{presentCount}</p>
  </div>
  <div className="rounded-xl border border-red-100 bg-red-50/30 p-3.5 shadow-xs">
    <span className="text-[11px] font-semibold text-red-600">Absent</span>
    <p className="text-xl font-bold text-red-600 mt-1">{absentCount}</p>
  </div>
  <div className="rounded-xl border border-slate-200 bg-slate-50/60 p-3.5 shadow-xs">
    <span className="text-[11px] font-semibold text-slate-500">Not Marked</span>
    <p className="text-xl font-bold text-slate-600 mt-1">{notMarkedCount}</p>
  </div>
  <div className="rounded-xl border border-blue-100 bg-blue-50/30 p-3.5 shadow-xs">
    <span className="text-[11px] font-semibold text-blue-600">Day Attendance</span>
    <p className="text-xl font-bold text-blue-700 mt-1">
      {totalCount > 0 && (presentCount > 0 || absentCount > 0) ? `${dayAttendancePct}%` : totalCount > 0 ? '0%' : '-'}
    </p>
  </div>
</div>
</div>

{/ * Feedback message */}
{message && (
  <div
    className={`rounded-xl border p-4 text-xs font-semibold flex items-center gap-2 ${
      message.isError
        ? 'border-red-200 bg-red-50 text-red-800'
    }`}
  >
    {message}
  </div>
)}

```

```

      : 'border-emerald-200 bg-emerald-50 text-emerald-800'
    }` }
  >
  {message.isError ? (
    <AlertCircle className="h-4 w-4 text-red-600 shrink-0" />
  ) : (
    <CheckCircle2 className="h-4 w-4 text-emerald-600 shrink-0" />
  )}
  <span>{message.text}</span>
</div>
)}

{/* ===== */}
{/* ATTENDANCE SHEET TABLE (Requirement 6, 7 & 8) */}
{/* ===== */}
<div className="rounded-2xl border border-slate-200 bg-white p-6 shadow-xs space-y-4">
  <div className="flex flex-col sm:flex-row sm:items-center sm:justify-between gap-3 border-b border-slate-100 pb-3">
    <div>
      <div className="flex items-center gap-2">
        <h3 className="text-base font-bold text-slate-900">Attendance Sheet</h3>
        <span className="rounded-md bg-blue-50 text-blue-700 px-2 py-0.5 text-xs font-mono font-bold border border-blue-100">
          {selectedDate}
        </span>
      </div>
      <p className="text-xs text-slate-500 mt-0.5">
        Displays actual status recorded for this date. Unknown dates show &ldquo;Not Marked&rdquo;.
      </p>
    </div>

    <div className="flex items-center gap-2 flex-wrap">
      <button
        type="button"
        onClick={() => handleMarkAll('PRESENT')}
        className="rounded-lg border border-slate-200 bg-slate-50 px-3 py-1.5 text-xs font-semibold text-slate-700 hover:r:bg-slate-100 transition-colors"
      >
        Mark All Present
      </button>
      <button
        type="button"
        onClick={() => handleMarkAll('ABSENT')}
        className="rounded-lg border border-slate-200 bg-slate-50 px-3 py-1.5 text-xs font-semibold text-slate-700 hover:r:bg-slate-100 transition-colors"
      >
        Mark All Absent
      </button>
      {(isAdmin || isTeacher) && studentsSheet.length > 0 && (
        <button
          type="button"
          onClick={handleSaveAttendance}
          disabled={saving}
          className="inline-flex items-center gap-1.5 rounded-lg bg-blue-600 px-4 py-1.5 text-xs font-semibold text-white shadow-xs hover:bg-blue-700 transition-colors disabled:opacity-60"
        >
          <Save className="h-3.5 w-3.5" />
          <span>{saving ? 'Saving...' : 'Save Attendance'}</span>
        </button>
      )}
    </div>
  </div>
</div>

{sheetLoading ? (
  <LoadingSkeleton rows={6} />
) : studentsSheet.length === 0 ? (
  <EmptyState
    icon={Users}
    title="No Students Found"
    description={`No students found in ${selectedProgram} - ${selectedDepartment}, Year ${selectedYear}, Semester ${selectedSemester}, Section ${selectedSection}.`}
  />
) : (
  <div className="overflow-x-auto">
    <table className="w-full text-left text-xs">
      <thead>
        <tr className="border-b border-slate-200 bg-slate-50 text-[11px] font-bold uppercase tracking-wider text-slate-500">
          <th className="py-3 px-4">Student ID</th>
          <th className="py-3 px-4">Student Name</th>

```

```

    { /* Requirement 6: Current Attendance % column */ }
    <th className="py-3 px-4 text-center">Current Attendance %</th>
    { /* Requirement 6 & 8: Today's Status */ }
    <th className="py-3 px-4 text-center">Date Status ({selectedDate})</th>
    <th className="py-3 px-4 text-right">Actions</th>
  </tr>
</thead>
<tbody className="divide-y divide-slate-100">
  {studentsSheet.map((student) => {
    const currentPct = student.currentAttendance?.percentage ?? 0;
    const totalRecorded = student.currentAttendance?.totalClasses ?? 0;
    const presentRecorded = student.currentAttendance?.presentClasses ?? 0;

    const pctStyle =
      totalRecorded === 0
        ? 'bg-slate-50 text-slate-500 border-slate-200'
        : currentPct >= 85
        ? 'bg-emerald-50 text-emerald-700 border-emerald-200'
        : currentPct >= 75
        ? 'bg-blue-50 text-blue-700 border-blue-200'
        : currentPct >= 60
        ? 'bg-amber-50 text-amber-700 border-amber-200'
        : 'bg-red-50 text-red-700 border-red-200';

    return (
      <tr key={student.studentId} className="hover:bg-slate-50/50 transition-colors">
        <td className="py-3.5 px-4 font-mono font-semibold text-slate-700">
          {student.studentNumber}
        </td>
        <td className="py-3.5 px-4">
          <span className="font-bold text-slate-900">{student.studentName}</span>
          <div className="text-[11px] text-slate-400">
            {student.department || selectedDepartment} &bull; Sec {student.section || selectedSection}
          </div>
        </td>

        { /* Current Attendance % (Requirement 6 & 9) */ }
        <td className="py-3.5 px-4 text-center whitespace-nowrap">
          <div className="inline-flex flex-col items-center">
            <span className={`inline-flex items-center px-2.5 py-0.5 rounded-full text-xs font-bold border ${pct
Style}`}>
              {totalRecorded > 0 ? `${currentPct}%` : 'No History'}
            </span>
            {totalRecorded > 0 && (
              <span className="text-[10px] text-slate-400 mt-0.5">
                {presentRecorded} of {totalRecorded} attended
              </span>
            )}
          </div>
        </td>

        { /* Date Status (Requirement 7 & 8) */ }
        <td className="py-3.5 px-4 text-center whitespace-nowrap">
          {student.status === 'PRESENT' ? (
            <span className="inline-flex items-center gap-1 px-3 py-1 rounded-full text-xs font-bold bg-emerald-
50 text-emerald-700 border border-emerald-200">
              <span>✓</span>
              <span>Present</span>
            </span>
          ) : student.status === 'ABSENT' ? (
            <span className="inline-flex items-center gap-1 px-3 py-1 rounded-full text-xs font-bold bg-red-50 t
ext-red-700 border border-red-200">
              <span>✗</span>
              <span>Absent</span>
            </span>
          ) : (
            <span className="inline-flex items-center gap-1 px-3 py-1 rounded-full text-xs font-semibold bg-slat
e-100 text-slate-600 border border-slate-200">
              <span>○</span>
              <span>Not Marked</span>
            </span>
          )}
        </td>

        { /* Action buttons */ }
        <td className="py-3.5 px-4 text-right whitespace-nowrap">
          <div className="inline-flex rounded-lg border border-slate-200 p-0.5 bg-slate-50 shadow-2xs">
            <button
              type="button"

```

```

      onClick={() => handleToggleStatus(student.studentId, 'PRESENT')}
      className={`px-3 py-1 text-xs font-semibold rounded-md transition-colors ${
        student.status === 'PRESENT'
          ? 'bg-emerald-600 text-white shadow-2xs'
          : 'text-slate-600 hover:text-slate-900'
      }`}
    >
      Present
    </button>
  </button>
  type="button"
  onClick={() => handleToggleStatus(student.studentId, 'ABSENT')}
  className={`px-3 py-1 text-xs font-semibold rounded-md transition-colors ${
    student.status === 'ABSENT'
      ? 'bg-red-600 text-white shadow-2xs'
      : 'text-slate-600 hover:text-slate-900'
  }`}
  >
    Absent
  </button>
</div>
</td>
</tr>
);
}}
</tbody>
</table>

{(isAdmin || isTeacher) && studentsSheet.length > 0 && (
  <div className="pt-4 border-t border-slate-100 flex flex-col sm:flex-row items-center justify-between gap-3">
    <p className="text-xs text-slate-500">
      Changes will be saved to MongoDB for date <strong>{selectedDate}</strong> and immediately recalculate student attendance rates.
    </p>
    <button
      type="button"
      onClick={handleSaveAttendance}
      disabled={saving}
      className="inline-flex items-center gap-2 rounded-lg bg-blue-600 px-5 py-2 text-xs font-semibold text-white shadow-xs hover:bg-blue-700 transition-colors disabled:opacity-60"
    >
      <Save className="h-4 w-4" />
      <span>{saving ? 'Saving...' : `Save Attendance (${selectedDate})`}</span>
    </button>
  </div>
)}
</div>
)}
</div>
</div>
</AppLayout>
);
}

```

A.1.5 AI Prediction / Early Warning

Early Warning Risk Segmentation Interface

File: apps/web/src/app/early-warnings/page.tsx

TYPESCRIPT

Purpose & Architecture: Categorizes monitored students into High, Medium, and Low risk cohorts based on latest ANN prediction, providing transparent diagnostic reasons, contributing risk factors, and intervention triggers.

```

'use client';

import React, { useState, useEffect } from 'react';
import Link from 'next/link';
import { AppLayout } from '@components/layout/AppLayout';
import { EmptyState } from '@components/ui/EmptyState';
import { LoadingSkeleton } from '@components/ui/LoadingSkeleton';
import { InterventionModal } from '@components/students/InterventionModal';
import { studentsApi } from '@lib/apiClient';
import { StudentDTO } from '@eduguard/shared';
import {
  AlertTriangle,

```

```

    PlusCircle,
    ArrowRight,
    CheckCircle2,
  } from 'lucide-react';
import { useAuth } from '@context/AuthContext';

export default function EarlyWarningsPage() {
  const { isAdmin, isTeacher } = useAuth();
  const [students, setStudents] = useState<StudentDTO[]>([]);
  const [loading, setLoading] = useState(true);
  const [activeTab, setActiveTab] = useState<'HIGH' | 'MEDIUM' | 'LOW'>('HIGH');
  const [selectedStudentForIntervention, setSelectedStudentForIntervention] = useState<StudentDTO | null>(null);

  const fetchStudents = async () => {
    setLoading(true);
    try {
      const res = await studentsApi.getStudents({ limit: 250 });
      if (res.data) {
        setStudents(res.data);
      }
    } catch (e) {
      console.error(e);
    } finally {
      setLoading(false);
    }
  };

  useEffect(() => {
    fetchStudents();
  }, []);

  // Filter students per risk category
  const highRiskStudents = students.filter(
    (s) => s.latestPrediction?.riskLevel === 'HIGH'
  );
  const mediumRiskStudents = students.filter(
    (s) => s.latestPrediction?.riskLevel === 'MEDIUM'
  );
  const lowRiskStudents = students.filter(
    (s) => s.latestPrediction?.riskLevel === 'LOW'
  );

  const activeCohort =
    activeTab === 'HIGH'
      ? highRiskStudents
      : activeTab === 'MEDIUM'
      ? mediumRiskStudents
      : lowRiskStudents;

  // Format main attention areas into simple bullet points (Section 13)
  const getAttentionAreas = (student: StudentDTO) => {
    const factors = student.latestPrediction?.riskFactors || [];
    if (factors.length === 0) {
      return ['No critical risk factors identified'];
    }

    return factors.map((rf: any) => {
      const f = rf.factor.toLowerCase();
      if (f.includes('attendance')) return 'Low attendance';
      if (f.includes('previous')) return 'Low previous marks';
      if (f.includes('internal')) return 'Low internal assessment marks';
      if (f.includes('assignment')) return 'Low assignment completion';
      if (f.includes('study')) return 'Low self-study hours';
      if (f.includes('participation')) return 'Low class participation';
      return rf.factor;
    });
  };

  return (
    <AppLayout>
      <div className="space-y-6 max-w-5xl">
        { /* Header (Section 13) */ }
        <div className="border-b border-slate-200 pb-4">
          <h1 className="text-2xl font-bold tracking-tight text-slate-900">
            Early Warnings
          </h1>
          <p className="mt-1 text-xs text-slate-500">
            Students who may need additional academic support.
          </p>
        </div>
      </div>
    </AppLayout>
  );
}

```

```

</div>

{/* 3 Simple Risk Category Tabs (Section 13) */}
<div className="flex border-b border-slate-200 space-x-2">
  <button
    type="button"
    onClick={() => setActiveTab('HIGH')}
    className={`flex items-center gap-2 px-4 py-2.5 text-xs font-bold border-b-2 transition-colors ${
      activeTab === 'HIGH'
        ? 'border-red-600 text-red-600'
        : 'border-transparent text-slate-500 hover:text-slate-800'
    }`}
  >
    <span>🔴</span>
    <span>High Risk ({highRiskStudents.length})</span>
  </button>

  <button
    type="button"
    onClick={() => setActiveTab('MEDIUM')}
    className={`flex items-center gap-2 px-4 py-2.5 text-xs font-bold border-b-2 transition-colors ${
      activeTab === 'MEDIUM'
        ? 'border-amber-500 text-amber-600'
        : 'border-transparent text-slate-500 hover:text-slate-800'
    }`}
  >
    <span>🟡</span>
    <span>Medium Risk ({mediumRiskStudents.length})</span>
  </button>

  <button
    type="button"
    onClick={() => setActiveTab('LOW')}
    className={`flex items-center gap-2 px-4 py-2.5 text-xs font-bold border-b-2 transition-colors ${
      activeTab === 'LOW'
        ? 'border-emerald-600 text-emerald-600'
        : 'border-transparent text-slate-500 hover:text-slate-800'
    }`}
  >
    <span>🟢</span>
    <span>Low Risk ({lowRiskStudents.length})</span>
  </button>
</div>

{/* Students List */}
{loading ? (
  <LoadingSkeleton rows={4} />
) : activeCohort.length === 0 ? (
  <EmptyState
    icon={CheckCircle2}
    title={`No ${activeTab.toLowerCase()} risk students`}
    description={`There are currently no students classified under ${activeTab.toLowerCase()} risk.`}
  />
) : (
  <div className="space-y-3">
    {activeCohort.map((student) => {
      const pred = student.latestPrediction;
      const attentionAreas = getAttentionAreas(student);

      return (
        <div
          key={student._id}
          className="rounded-xl border border-slate-200 bg-white p-5 shadow-xs flex flex-col md:flex-row md:items-center md:justify-between gap-4"
        >
          <div className="space-y-1.5 flex-1">
            <div className="flex items-center gap-2 flex-wrap">
              <h3 className="text-base font-bold text-slate-900">{student.name}</h3>
              <span className="font-mono text-xs text-slate-500 font-semibold bg-slate-100 px-2 py-0.5 rounded border border-slate-200">
                {student.studentId}
              </span>
              <span className="text-xs font-semibold text-blue-700 bg-blue-50 px-2 py-0.5 rounded border border-blue-100">
                {student.program || student.course || 'B.Tech'}
              </span>
              <span className="text-xs text-slate-600 font-medium">
                &bull; {student.department} &bull; {student.year ? `${student.year} ?` : '1st' : student.year === 2 ? '2nd' : student.year === 3 ? '3rd' : `${student.year}th`} Year : 'Year -' &bull; Sem {student.semester}-{student.section}
              </span>
            </div>
          </div>
        </div>
      )
    })}
  </div>
)
}

```

```

    </span>
  </div>

  <div className="flex items-center gap-3 pt-1 flex-wrap">
    <div className="flex items-center gap-1.5">
      <span className="text-xs text-slate-500 font-medium">Predicted Score:</span>
      <span className="text-sm font-extrabold text-slate-900">
        {pred ? Math.round(pred.predictedScore) : '-'} / 100
      </span>
    </div>
    <span className="text-slate-300">&bull;</span>
    <span
      className={`inline-flex items-center gap-1 px-2.5 py-0.5 rounded-full text-[11px] font-bold ${
        student.latestPrediction?.riskLevel === 'HIGH'
          ? 'bg-red-50 text-red-700 border border-red-200'
          : student.latestPrediction?.riskLevel === 'MEDIUM'
          ? 'bg-amber-50 text-amber-700 border border-amber-200'
          : 'bg-emerald-50 text-emerald-700 border border-emerald-200'
        }`}
    >
      <span>
        {student.latestPrediction?.riskLevel === 'HIGH'
          ? '🔴'
          : student.latestPrediction?.riskLevel === 'MEDIUM'
          ? '🟡'
          : '🟢'}
      </span>
      <span>{student.latestPrediction?.riskLevel} RISK</span>
    </span>
  </div>

  { /* Main Attention Areas */ }
  <div className="pt-1">
    <p className="text-[11px] font-bold uppercase tracking-wider text-slate-400">
      Main Attention Areas:
    </p>
    <ul className="mt-1 space-y-0.5 text-xs text-slate-700">
      {attentionAreas.map((area, idx) => (
        <li key={idx} className="flex items-center gap-1.5">
          <span className="text-slate-400">•</span>
          <span>{area}</span>
        </li>
      ))}
    </ul>
  </div>

  { /* Actions (Requirement 19) */ }
  <div className="flex items-center gap-2 shrink-0 pt-2 md:pt-0">
    {(() => {
      const sId = typeof student._id === 'object' && student._id !== null
        ? String((student._id as any)._id || (student._id as any).toString())
        : String(student._id || student.studentId || '');

      return (
        <Link
          href={` /students/${sId}`}
          className="inline-flex items-center gap-1 rounded-lg border border-blue-200 bg-blue-50 px-3.5 py-1.5 text-xs font-semibold text-blue-700 hover:bg-blue-100 transition-colors shadow-2xs"
        >
          <span>View Profile</span>
          <ArrowRight className="h-3.5 w-3.5" />
        </Link>
      );
    })()}

    {(isAdmin || isTeacher) && (
      <button
        type="button"
        onClick={() => setSelectedStudentForIntervention(student)}
        className="inline-flex items-center gap-1.5 rounded-lg bg-blue-600 px-3.5 py-1.5 text-xs font-semibold text-white shadow-xs hover:bg-blue-700 transition-colors"
      >
        <PlusCircle className="h-3.5 w-3.5" />
        <span>Add Intervention</span>
      </button>
    )}
  </div>
</div>

```

```

    });
  }}
</div>
)}
</div>

{/* Intervention Modal */}
{selectedStudentForIntervention && (
  <InterventionModal
    studentId={selectedStudentForIntervention._id}
    studentName={selectedStudentForIntervention.name}
    predictionId={selectedStudentForIntervention.latestPrediction?._id}
    onSuccess={() => {
      fetchStudents();
      setSelectedStudentForIntervention(null);
    }}
    onClose={() => setSelectedStudentForIntervention(null)}
  />
)}
</AppLayout>
);
}

```

Interactive ANN Prediction Runner Modal

File: apps/web/src/components/students/PredictionRunModal.tsx

TYPESCRIPT

Purpose & Architecture: Enables interactive what-if simulation by adjusting attendance, prior scores, internal assessment, assignment completion, study hours, and participation. Invokes backend prediction API and displays immediate score and recommendations.

```
'use client';

import React, { useState } from 'react';
import { Cpu, X, Play, AlertCircle, CheckCircle2 } from 'lucide-react';
import { predictionsApi } from '@lib/apiClient';
import { FeatureInputs, PredictionResult } from '@eduguard/shared';
import { Badge } from '@components/ui/Badge';

interface PredictionRunModalProps {
  studentId: string;
  studentName: string;
  initialFeatures?: FeatureInputs;
  onSuccess: (result: PredictionResult) => void;
  onClose: () => void;
}

export function PredictionRunModal({
  studentId,
  studentName,
  initialFeatures,
  onSuccess,
  onClose,
}: PredictionRunModalProps) {
  const [features, setFeatures] = useState<FeatureInputs>({
    attendance: initialFeatures?.attendance ?? 75,
    previousScore: initialFeatures?.previousScore ?? 65,
    internalMarks: initialFeatures?.internalMarks ?? 60,
    assignmentCompletion: initialFeatures?.assignmentCompletion ?? 70,
    studyHours: initialFeatures?.studyHours ?? 8,
    participation: initialFeatures?.participation ?? 65,
  });

  const [loading, setLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);
  const [result, setResult] = useState<PredictionResult | null>(null);

  const handleRun = async (e: React.FormEvent) => {
    e.preventDefault();
    setLoading(true);
    setError(null);
    try {
      const res = await predictionsApi.generate(studentId, features);
      if (res.data) {
        setResult(res.data);
        onSuccess(res.data);
      }
    } catch {
```

```

    }
  } catch (err: any) {
    setError(
      err.response?.data?.message || 'Failed to communicate with AI prediction microservice.'
    );
  } finally {
    setLoading(false);
  }
};

return (
  <div className="fixed inset-0 z-50 flex items-center justify-center bg-slate-900/50 p-4">
    <div className="w-full max-w-xl rounded-xl bg-white shadow-2xl border border-slate-200 overflow-hidden flex flex-col max-h-[90vh]">
      {/* Header */}
      <div className="flex items-center justify-between border-b border-slate-200 px-6 py-4 bg-slate-50">
        <div className="flex items-center gap-2.5">
          <div className="rounded-md bg-blue-600 p-2 text-white shadow-xs">
            <Cpu className="h-5 w-5" />
          </div>
          <div>
            <h3 className="text-base font-bold text-slate-900">Run ANN Performance Prediction</h3>
            <p className="text-xs text-slate-500">Student: {studentName}</p>
          </div>
        </div>
        <button
          onClick={onClose}
          className="rounded p-1 text-slate-400 hover:bg-slate-200 hover:text-slate-600"
        >
          <X className="h-5 w-5" />
        </button>
      </div>

      {/* Content */}
      <div className="p-6 overflow-y-auto space-y-6">
        {error && (
          <div className="flex items-center gap-2 rounded-lg border border-red-200 bg-red-50 p-3.5 text-xs text-red-700">
            <AlertCircle className="h-4 w-4 shrink-0 text-red-600" />
            <span>{error}</span>
          </div>
        )}

        {result ? (
          <div className="space-y-4">
            <div className="rounded-lg border border-emerald-200 bg-emerald-50/50 p-4">
              <div className="flex items-center justify-between">
                <div className="flex items-center gap-2 text-emerald-800 font-semibold text-sm">
                  <CheckCircle2 className="h-4 w-4" />
                  <span>Neural Network Estimation Generated</span>
                </div>
                <Badge variant="risk">{result.riskLevel}</Badge>
              </div>
              <div className="mt-3 flex items-baseline gap-3">
                <span className="text-3xl font-extrabold text-slate-900">
                  {result.predictedScore.toFixed(1)} / 100
                </span>
                <span className="text-xs text-slate-500">Predicted Final Examination Score</span>
              </div>
              <p className="mt-1 text-[11px] text-slate-500">
                Model Engine: {result.modelVersion} • MSE: 17.38
              </p>
            </div>

            {/* Factors */}
            <div>
              <h4 className="text-xs font-bold uppercase tracking-wider text-slate-700 mb-2">
                Potential Contributing Factors ({result.riskFactors.length})
              </h4>
              {result.riskFactors.length === 0 ? (
                <p className="text-xs text-slate-500 italic">No adverse risk indicators identified.</p>
              ) : (
                <div className="space-y-2">
                  {result.riskFactors.map((rf, idx) => (
                    <div
                      key={idx}
                      className="rounded border border-slate-200 bg-white p-2.5 text-xs flex items-start justify-between"
                    >
                      <div>
                        <p className="font-semibold text-slate-800">{rf.factor}</p>

```

```

        <p className="text-slate-600 mt-0.5">{rf.description}</p>
      </div>
      <Badge variant="priority">{rf.severity}</Badge>
    </div>
  )}
</div>
)}
</div>

{/* Recommendations */}
<div>
  <h4 className="text-xs font-bold uppercase tracking-wider text-slate-700 mb-2">
    Recommended Interventions
  </h4>
  <ul className="list-disc list-inside space-y-1 text-xs text-slate-700">
    {result.recommendations.map((rec, idx) => (
      <li key={idx}>{rec}</li>
    ))}
  </ul>
</div>

<div className="pt-2 flex justify-end">
  <button
    type="button"
    onClick={onClose}
    className="rounded-md bg-blue-600 px-4 py-2 text-xs font-semibold text-white hover:bg-blue-700"
  >
    Done
  </button>
</div>
</div>
) : (
<form onSubmit={handleRun} className="space-y-4">
  <p className="text-xs text-slate-600">
    Adjust academic feature values to evaluate the Artificial Neural Network's non-linear regression response:
  </p>

  <div className="grid grid-cols-2 gap-4">
    {/* Attendance */}
    <div>
      <div className="flex justify-between text-xs font-medium text-slate-700 mb-1">
        <span>Attendance Rate</span>
        <span className="font-bold">{features.attendance}%</span>
      </div>
      <input
        type="range"
        min="0"
        max="100"
        value={features.attendance}
        onChange={(e) => setFeatures({ ...features, attendance: Number(e.target.value) })}
        className="w-full accent-blue-600"
      />
    </div>

    {/* Previous Score */}
    <div>
      <div className="flex justify-between text-xs font-medium text-slate-700 mb-1">
        <span>Previous Academic Score</span>
        <span className="font-bold">{features.previousScore}</span>
      </div>
      <input
        type="range"
        min="0"
        max="100"
        value={features.previousScore}
        onChange={(e) => setFeatures({ ...features, previousScore: Number(e.target.value) })}
        className="w-full accent-blue-600"
      />
    </div>

    {/* Internal Marks */}
    <div>
      <div className="flex justify-between text-xs font-medium text-slate-700 mb-1">
        <span>Internal Assessment Marks</span>
        <span className="font-bold">{features.internalMarks}</span>
      </div>
      <input
        type="range"
        min="0"

```

```

        max="100"
        value={features.internalMarks}
        onChange={(e) => setFeatures({ ...features, internalMarks: Number(e.target.value) })}
        className="w-full accent-blue-600"
      />
    </div>

    {/ Assignment Completion */}
    <div>
      <div className="flex justify-between text-xs font-medium text-slate-700 mb-1">
        <span>Assignment Completion</span>
        <span className="font-bold">{features.assignmentCompletion}%</span>
      </div>
      <input
        type="range"
        min="0"
        max="100"
        value={features.assignmentCompletion}
        onChange={(e) => setFeatures({ ...features, assignmentCompletion: Number(e.target.value) })}
        className="w-full accent-blue-600"
      />
    </div>

    {/ Weekly Study Hours */}
    <div>
      <div className="flex justify-between text-xs font-medium text-slate-700 mb-1">
        <span>Study Hours / Week</span>
        <span className="font-bold">{features.studyHours} hrs</span>
      </div>
      <input
        type="range"
        min="0"
        max="35"
        value={features.studyHours}
        onChange={(e) => setFeatures({ ...features, studyHours: Number(e.target.value) })}
        className="w-full accent-blue-600"
      />
    </div>

    {/ Class Participation */}
    <div>
      <div className="flex justify-between text-xs font-medium text-slate-700 mb-1">
        <span>Classroom Participation</span>
        <span className="font-bold">{features.participation}%</span>
      </div>
      <input
        type="range"
        min="0"
        max="100"
        value={features.participation}
        onChange={(e) => setFeatures({ ...features, participation: Number(e.target.value) })}
        className="w-full accent-blue-600"
      />
    </div>
  </div>

  <div className="border-t border-slate-200 pt-4 flex items-center justify-between">
    <span className="text-[11px] text-slate-400">
      Calls FastAPI &bull; TensorFlow 2.16 ANN Service
    </span>
    <div className="flex gap-2">
      <button
        type="button"
        onClick={onClose}
        className="rounded-md border border-slate-300 px-3 py-1.5 text-xs font-medium text-slate-700 hover:bg-slate-100"
      >
        Cancel
      </button>
      <button
        type="submit"
        disabled={loading}
        className="inline-flex items-center gap-1.5 rounded-md bg-blue-600 px-4 py-1.5 text-xs font-semibold text-white shadow-sm hover:bg-blue-700 disabled:opacity-50"
      >
        {loading ? (
          <div className="h-3 w-3 animate-spin rounded-full border-2 border-white border-r-transparent"></div>
        ) : (
          <Play className="h-3.5 w-3.5" />
        )}
      </button>
    </div>
  </div>

```

```
        })
        <span>Execute Prediction</span>
      </button>
    </div>
  </div>
</div>
</form>
  })
</div>
</div>
</div>
);
}
```

A.1.6 Teacher Dashboard

Architecture Note: In the EduGuard AI system architecture, the Teacher Dashboard is integrated directly within `apps/web/src/app/dashboard/page.tsx` (lines 283–541). When authenticated as a Teacher or Administrator, the dashboard displays college-wide risk counts, low/medium/high risk distribution cards, cohort filtering controls across all academic programs, student priority listings, and remedial actions. Complete source code is documented in Section A.1.1 above.

A.2 Backend Source Code

A.2.1 Server / Application Configuration

Express Server Lifecycle Entrypoint

File: apps/api/src/server.ts

TYPESCRIPT

Purpose & Architecture: Initializes database connectivity, starts HTTP listener for local development, and exports Express app for Vercel Serverless Function deployment.

```
import { app } from './app';
import { connectDatabase } from './config/database';
import { env } from './config/environment';

async function startServer() {
  try {
    await connectDatabase();

    const server = app.listen(env.PORT, () => {
      console.log('=====');
      console.log(` EduGuard AI Backend API Server`);
      console.log(` Status: RUNNING`);
      console.log(` Port: ${env.PORT}`);
      console.log(` Environment: ${env.NODE_ENV}`);
      console.log(` ML Service URL: ${env.ML_SERVICE_URL}`);
      console.log(` API Health: http://localhost:${env.PORT}/api/health`);
      console.log('=====');
    });

    server.on('error', (err: any) => {
      if (err.code === 'EADDRINUSE') {
        console.log(`Port ${env.PORT} in use, waiting 1.5s to retry...`);
        setTimeout(() => {
          server.close();
          server.listen(env.PORT);
        }, 1500);
      } else {
        console.error('Server error:', err);
      }
    });

    const shutdown = async (signal: string) => {
      console.log(`\nReceived ${signal}. Gracefully shutting down...`);
      server.close(() => {
        console.log('HTTP server closed.');
```

```
        process.exit(0);
      });
    };

    process.on('SIGTERM', () => shutdown('SIGTERM'));
    process.on('SIGINT', () => shutdown('SIGINT'));
  } catch (error: any) {
    console.error('Fatal error during server startup:', error.message);
    process.exit(1);
  }
}

// Only start persistent HTTP listener in standalone / local development mode (not when loaded as a Vercel Function)
if (!process.env.VERCEL) {
  startServer();
}

export { app };
export default app;
```

Express Application Setup & Routing Hub

TYPESCRIPT

File: apps/api/src/app.ts

Purpose & Architecture: Mounts security middleware (Helmet, CORS), body parsers, logging, serverless database connection pooling, and registers all 20 API route endpoints under /api.

```
import express, { Request, Response } from 'express';
import cors from 'cors';
import helmet from 'helmet';
import morgan from 'morgan';
import { env } from './config/environment';
import { errorHandler } from './middleware/errorHandler';

import authRoutes from './routes/authRoutes';
import studentRoutes from './routes/studentRoutes';
import teacherRoutes from './routes/teacherRoutes';
import classRoutes from './routes/classRoutes';
import subjectRoutes from './routes/subjectRoutes';
import academicRecordRoutes from './routes/academicRecordRoutes';
import predictionRoutes from './routes/predictionRoutes';
import interventionRoutes from './routes/interventionRoutes';
import dashboardRoutes from './routes/dashboardRoutes';
import notificationRoutes from './routes/notificationRoutes';
import auditLogRoutes from './routes/auditLogRoutes';
import settingsRoutes from './routes/settingsRoutes';
import reportRoutes from './routes/reportRoutes';
import attendanceRoutes from './routes/attendanceRoutes';
import assessmentRoutes from './routes/assessmentRoutes';
import assignmentRoutes from './routes/assignmentRoutes';
import studyLogRoutes from './routes/studyLogRoutes';
import participationRoutes from './routes/participationRoutes';
import programRoutes from './routes/programRoutes';
import departmentRoutes from './routes/departmentRoutes';
import { mlClientService } from './services/mlClientService';
import mongoose from 'mongoose';
import { connectDatabase } from './config/database';

export const app = express();

// Ensure database connection for serverless/function invocations
app.use(async (_req, _res, next) => {
  if (mongoose.connection.readyState === 0) {
    try {
      await connectDatabase();
    } catch (err) {
      return next(err);
    }
  }
  next();
});

// Security & Parsing
app.use(helmet());
app.use(
  cors({
    origin: (origin, callback) => {
      // Allow localhost dev origins or direct curl
      if (!origin || origin.startsWith('http://localhost') || origin.startsWith('http://127.0.0.1')) {
        callback(null, true);
      } else {
        callback(null, false);
      }
    },
    credentials: true,
  })
);
app.use(express.json({ limit: '10mb' }));
app.use(express.urlencoded({ extended: true, limit: '10mb' }));

if (env.NODE_ENV !== 'test') {
  app.use(morgan('dev'));
}

// Health check endpoint
app.get('/api/health', async (_req: Request, res: Response) => {
  const mlHealth = await mlClientService.checkHealth();
  res.json({
    status: 'HEALTHY',
  });
});
```

```

    service: 'EduGuard AI Backend API',
    timestamp: new Date().toISOString(),
    environment: env.NODE_ENV,
    mlService: mlHealth,
  });
});

// API Routes
app.use('/api/auth', authRoutes);
app.use('/api/students', studentRoutes);
app.use('/api/teachers', teacherRoutes);
app.use('/api/classes', classRoutes);
app.use('/api/subjects', subjectRoutes);
app.use('/api/academic-records', academicRecordRoutes);
app.use('/api/predictions', predictionRoutes);
app.use('/api/interventions', interventionRoutes);
app.use('/api/dashboard', dashboardRoutes);
app.use('/api/notifications', notificationRoutes);
app.use('/api/audit-logs', auditLogRoutes);
app.use('/api/settings', settingsRoutes);
app.use('/api/reports', reportRoutes);
app.use('/api/attendance', attendanceRoutes);
app.use('/api/assessments', assessmentRoutes);
app.use('/api/assignments', assignmentRoutes);
app.use('/api/study-logs', studyLogRoutes);
app.use('/api/participation', participationRoutes);
app.use('/api/programs', programRoutes);
app.use('/api/departments', departmentRoutes);

// 404 Route Not Found
app.use('*', (req: Request, res: Response) => {
  res.status(404).json({
    success: false,
    message: `Resource endpoint ${req.method} ${req.originalUrl} not found.`,
  });
});

// Error handling
app.use(errorHandler);

export default app;

```

Environment Configuration Loader

File: apps/api/src/config/environment.ts

TYPESCRIPT

Purpose & Architecture: Validates and parses environment variables with multi-location .env resolution and safe production fallbacks.

```

import dotenv from 'dotenv';
import path from 'path';

// Load environment variables from api directory or workspace root
dotenv.config();
dotenv.config({ path: path.resolve(__dirname, '../../.env') });
dotenv.config({ path: path.resolve(__dirname, '../../.env') });
dotenv.config({ path: path.resolve(process.cwd(), '../../.env') });
dotenv.config({ path: path.resolve(process.cwd(), '.env') });

export const env = {
  PORT: parseInt(process.env.PORT || '5000', 10),
  NODE_ENV: process.env.NODE_ENV || 'development',
  CORS_ORIGIN: process.env.CORS_ORIGIN || 'http://localhost:3000',
  MONGODB_URI: process.env.MONGODB_URI || 'mongodb://localhost:27017/eduguard_ai',
  JWT_SECRET: process.env.JWT_SECRET || 'fallback_secret_key_change_in_production_32chars!',
  JWT_EXPIRES_IN: process.env.JWT_EXPIRES_IN || '7d',
  ML_SERVICE_URL: process.env.ML_SERVICE_URL || 'http://localhost:8000',
  ML_SERVICE_SECRET: process.env.ML_SERVICE_SECRET || '<YOUR_ML_SERVICE_SECRET>',
};

```

Mongoose Database Connection Manager

File: apps/api/src/config/database.ts

TYPESCRIPT

Purpose & Architecture: Reusable MongoDB connection manager implementing connection reuse and caching across serverless invocations.

```
import mongoose from 'mongoose';
import { env } from './environment';

export async function connectDatabase(): Promise<typeof mongoose> {
  if (mongoose.connection.readyState >= 1) {
    return mongoose;
  }
  try {
    const conn = await mongoose.connect(env.MONGODB_URI, {
      serverSelectionTimeoutMS: 10000,
    });
    console.log(`[Database] Successfully connected to MongoDB at ${conn.connection.host}/${conn.connection.name}`);
    return conn;
  } catch (error: any) {
    console.error('[Database] Failed to connect to MongoDB:', error.message);
    throw error;
  }
}

export async function disconnectDatabase(): Promise<void> {
  await mongoose.disconnect();
  console.log('[Database] Disconnected from MongoDB');
}
```

A.2.2 Student API

Student Route Definitions

File: apps/api/src/routes/studentRoutes.ts

TYPESCRIPT

Purpose & Architecture: Registers REST endpoints for student directory listing, ID resolution, profile updates, and behavioral metrics.

```
import { Router } from 'express';
import multer from 'multer';
import {
  getStudents,
  getStudentById,
  getMyProfile,
  createStudent,
  updateStudent,
  importStudentsCSV,
  getStudentMetrics,
} from '../controllers/studentController';
import { authenticateToken, requireRole, authorizeStudentAccess } from '../middleware/auth';

const router = Router();
const upload = multer({ storage: multer.memoryStorage(), limits: { fileSize: 5 * 1024 * 1024 } });

router.use(authenticateToken);

router.get('/', getStudents);
router.get('/me', getMyProfile);
router.post('/', requireRole('ADMIN', 'TEACHER'), createStudent);
router.post('/import-csv', requireRole('ADMIN', 'TEACHER'), upload.single('file'), importStudentsCSV);
router.get('/:id', authorizeStudentAccess, getStudentById);
router.get('/:id/metrics', authorizeStudentAccess, getStudentMetrics);
router.put('/:id', requireRole('ADMIN', 'TEACHER'), authorizeStudentAccess, updateStudent);

export default router;
```

Student Controller Implementation

File: apps/api/src/controllers/studentController.ts

TYPESCRIPT

Purpose & Architecture: Executes business logic for student CRUD operations, cohort filtering, search queries, pagination, and metric computation.

```

import { Request, Response } from 'express';
import mongoose from 'mongoose';
import { Student } from '../models/Student';
import { User } from '../models/User';
import { AcademicRecord } from '../models/AcademicRecord';
import { Prediction } from '../models/Prediction';
import { Intervention } from '../models/Intervention';
import { Teacher } from '../models/Teacher';
import { Class } from '../models/Class';
import { logAuditEvent } from '../middleware/audit';
import { parseAndValidateStudentCSV } from '../services/csvImportService';
import { studentMetricService } from '../services/studentMetricService';
import { resolveCanonicalCohort, buildStudentQuery } from '../utils/canonicalCohort';

export async function getStudents(req: Request, res: Response): Promise<void> {
  try {
    const {
      search,
      program,
      programId,
      department,
      departmentId,
      year,
      semester,
      section,
      riskLevel,
      status = 'ACTIVE',
      page = '1',
      limit = '25',
    } = req.query;

    const pageNum = parseInt(page as string, 10) || 1;
    const limitNum = parseInt(limit as string, 10) || 25;
    const skip = (pageNum - 1) * limitNum;

    let baseQuery: any = {};
    if (status !== 'ALL') {
      baseQuery.status = status;
    }

    // Role-based restrictions
    if (req.user?.role === 'STUDENT') {
      baseQuery.userId = req.user.id;
    }

    // Resolve canonical cohort
    const yearParam = year || (req.query.academicYear as string);
    const resolved = await resolveCanonicalCohort({
      program: program as string,
      programId: programId as string,
      department: department as string,
      departmentId: departmentId as string,
      year: yearParam as string,
      semester: semester as string,
      section: section as string,
    });

    const query = buildStudentQuery(resolved, baseQuery);

    // Global Search across name, studentId, email
    if (search) {
      const searchRegex = new RegExp(search as string, 'i');
      const searchCondition = {
        $or: [{ name: searchRegex }, { studentId: searchRegex }, { email: searchRegex }],
      };
      if (query.$or) {
        query.$and = [{ $or: query.$or }, searchCondition];
        delete query.$or;
      } else {
        query.$or = searchCondition.$or;
      }
    }

    const total = await Student.countDocuments(query);
    const students = await Student.find(query)
      .sort({ studentId: 1 })
      .skip(skip)
      .limit(limitNum)

```

```

        .lean();

// Populate latest prediction for each student
const studentIds = students.map((s) => s._id);
const latestPredictions = await Prediction.aggregate([
  { $match: { studentId: { $in: studentIds } } },
  { $sort: { createdAt: -1 } },
  {
    $group: {
      _id: '$studentId',
      latest: { $first: '$$ROOT' },
    },
  },
]);

const predictionMap = new Map();
latestPredictions.forEach((p) => {
  predictionMap.set(p._id.toString(), p.latest);
});

let enrichedStudents = students.map((s) => ({
  ...s,
  latestPrediction: predictionMap.get(s._id.toString()) || null,
}));

// If risk level filter is specified
if (riskLevel && riskLevel !== 'ALL') {
  enrichedStudents = enrichedStudents.filter(
    (s) => s.latestPrediction?.riskLevel === riskLevel
  );
}

res.json({
  success: true,
  data: enrichedStudents,
  meta: {
    page: pageNum,
    limit: limitNum,
    total,
    totalPages: Math.ceil(total / limitNum),
  },
});
} catch (error: any) {
  res.status(500).json({
    success: false,
    message: 'Failed to retrieve students.',
    errors: [error.message],
  });
}
}

export async function getStudentById(req: Request, res: Response): Promise<void> {
  const { id } = req.params;

  try {
    let student = null;
    const isObjectId = mongoose.Types.ObjectId.isValid(id) && id.toString().length === 24;
    if (isObjectId) {
      student = await Student.findById(id).lean();
    } else {
      student = await Student.findOne({ studentId: id.toUpperCase() }).lean();
    }

    if (!student) {
      res.status(404).json({ success: false, message: 'Student record not found.' });
      return;
    }

    // Role security check
    if (req.user?.role === 'STUDENT' && student.userId?.toString() !== req.user.id) {
      res.status(403).json({ success: false, message: 'Access denied to this student profile.' });
      return;
    }

    // Fetch related records & calculate real metrics from MongoDB records
    const [academicRecords, predictions, interventions, calculatedMetrics, academicHistory] = await Promise.all([
      AcademicRecord.find({ studentId: student._id }).populate('subjectId').sort({ semester: 1 }).lean(),
      Prediction.find({ studentId: student._id }).sort({ createdAt: -1 }).lean(),
      Intervention.find({ studentId: student._id }).populate('teacherId').sort({ createdAt: -1 }).lean(),

```

```

    studentMetricService.calculateStudentMetrics(student._id).catch(() => null),
    studentMetricService.getFullAcademicHistory(student._id).catch(() => null),
  ]);

  res.json({
    success: true,
    data: {
      ...student,
      academicRecords,
      predictions,
      latestPrediction: predictions[0] || null,
      interventions,
      calculatedMetrics,
      academicHistory,
    },
  });
} catch (error: any) {
  res.status(500).json({
    success: false,
    message: 'Failed to retrieve student profile.',
    errors: [error.message],
  });
}
}

export async function getMyProfile(req: Request, res: Response): Promise<void> {
  try {
    const student = await Student.findOne({ userId: req.user?.id }).lean();
    if (!student) {
      res.status(404).json({ success: false, message: 'Student profile not linked to this account.' });
      return;
    }

    const [academicRecords, predictions, interventions, calculatedMetrics, academicHistory] = await Promise.all([
      AcademicRecord.find({ studentId: student._id }).populate('subjectId').sort({ semester: 1 }).lean(),
      Prediction.find({ studentId: student._id }).sort({ createdAt: -1 }).lean(),
      Intervention.find({ studentId: student._id }).populate('teacherId').sort({ createdAt: -1 }).lean(),
      studentMetricService.calculateStudentMetrics(student._id).catch(() => null),
      studentMetricService.getFullAcademicHistory(student._id).catch(() => null),
    ]);

    res.json({
      success: true,
      data: {
        ...student,
        academicRecords,
        predictions,
        latestPrediction: predictions[0] || null,
        interventions,
        calculatedMetrics,
        academicHistory,
      },
    });
  } catch (error: any) {
    res.status(500).json({
      success: false,
      message: 'Failed to retrieve personal profile.',
      errors: [error.message],
    });
  }
}

export async function getStudentMetrics(req: Request, res: Response): Promise<void> {
  const { id } = req.params;

  try {
    let student = null;
    if (mongoose.Types.ObjectId.isValid(id)) {
      student = await Student.findById(id).lean();
    } else {
      student = await Student.findOne({ studentId: id.toUpperCase() }).lean();
    }

    if (!student) {
      res.status(404).json({ success: false, message: 'Student record not found.' });
      return;
    }

    if (req.user?.role === 'STUDENT' && student.userId?.toString() !== req.user.id) {

```

```

    res.status(403).json({ success: false, message: 'Access denied to this student metrics.' });
    return;
  }

  const calculatedMetrics = await studentMetricService.calculateStudentMetrics(student._id);
  res.json({
    success: true,
    data: calculatedMetrics,
  });
} catch (error: any) {
  res.status(500).json({
    success: false,
    message: 'Failed to calculate student metrics.',
    errors: [error.message],
  });
}
}
}

export async function createStudent(req: Request, res: Response): Promise<void> {
  const {
    studentId,
    name,
    email,
    phone,
    department,
    course,
    year,
    section,
    academicYear,
    semester,
  } = req.body;

  if (!studentId || !name || !email || !department || !year || !section || !semester) {
    res.status(400).json({
      success: false,
      message: 'Missing required student details.',
    });
    return;
  }

  try {
    const existing = await Student.findOne({
      $or: [{ studentId: studentId.toUpperCase().trim() }, { email: email.toLowerCase().trim() }],
    });

    if (existing) {
      res.status(409).json({
        success: false,
        message: 'A student with this Student ID or Email already exists.',
      });
      return;
    }

    const student = await Student.create({
      studentId: studentId.toUpperCase().trim(),
      name: name.trim(),
      email: email.toLowerCase().trim(),
      phone: phone?.trim(),
      department: department.trim(),
      course: course?.trim() || 'B.Tech',
      year: Number(year),
      section: section.toUpperCase().trim(),
      academicYear: academicYear?.trim() || '2025-2026',
      semester: Number(semester),
      status: 'ACTIVE',
    });

    await logAuditEvent(
      req.user?.id,
      req.user?.name,
      req.user?.email,
      'CREATE_STUDENT',
      'STUDENT',
      student._id.toString(),
      { studentId: student.studentId, name: student.name },
      req.ip
    );

    res.status(201).json({

```

```
        success: true,
        message: 'Student record created successfully.',
        data: student,
    });
} catch (error: any) {
    res.status(500).json({
        success: false,
        message: 'Failed to create student record.',
        errors: [error.message],
    });
}
}

export async function updateStudent(req: Request, res: Response): Promise<void> {
    const { id } = req.params;

    try {
        const student = await Student.findById(id);
        if (!student) {
            res.status(404).json({ success: false, message: 'Student not found.' });
            return;
        }

        const fields = [
            'name',
            'email',
            'phone',
            'department',
            'course',
            'year',
            'section',
            'academicYear',
            'semester',
            'status',
        ];

        fields.forEach((field) => {
            if (req.body[field] !== undefined) {
                (student as any)[field] = req.body[field];
            }
        });

        await student.save();

        await logAuditEvent(
            req.user?.id,
            req.user?.name,
            req.user?.email,
            'UPDATE_STUDENT',
            'STUDENT',
            student._id.toString(),
            { studentId: student.studentId },
            req.ip
        );

        res.json({
            success: true,
            message: 'Student record updated successfully.',
            data: student,
        });
    } catch (error: any) {
        res.status(500).json({
            success: false,
            message: 'Failed to update student record.',
            errors: [error.message],
        });
    }
}

export async function importStudentsCSV(req: Request, res: Response): Promise<void> {
    if (!req.file) {
        res.status(400).json({ success: false, message: 'No CSV file provided.' });
        return;
    }

    try {
        const result = await parseAndValidateStudentCSV(req.file.buffer);

        let insertedCount = 0;
```

```

const importErrors = [...result.errors];

for (const validRow of result.validRows) {
  try {
    const existing = await Student.findOne({
      $or: [{ studentId: validRow.studentId }, { email: validRow.email }],
    });

    if (existing) {
      importErrors.push({
        row: 0,
        studentId: validRow.studentId,
        error: `Student ID ${validRow.studentId} or email ${validRow.email} already exists in database.`,
      });
      continue;
    }

    await Student.create(validRow);
    insertedCount++;
  } catch (err: any) {
    importErrors.push({
      row: 0,
      studentId: validRow.studentId,
      error: err.message,
    });
  }
}

await logAuditEvent(
  req.user?.id,
  req.user?.name,
  req.user?.email,
  'IMPORT_STUDENTS_CSV',
  'STUDENT',
  undefined,
  { insertedCount, failedCount: importErrors.length },
  req.ip
);

res.json({
  success: true,
  message: `CSV processing finished: ${insertedCount} imported, ${importErrors.length} rejected/skipped.`,
  data: {
    totalRows: result.totalRows,
    insertedCount,
    rejectedCount: importErrors.length,
    errors: importErrors,
  },
});
} catch (error: any) {
  res.status(500).json({
    success: false,
    message: 'Failed to process CSV file.',
    errors: [error.message],
  });
}
}

```

Student Data Model & Schema

File: apps/api/src/models/Student.ts

TYPESCRIPT

Purpose & Architecture: Mongoose Schema defining student identity, contact info, and hierarchical cohort mapping (Program, Department, Year, Semester, Section).

```

import mongoose, { Document, Schema } from 'mongoose';
import { StudentStatus } from '@eduguard/shared';

export interface IStudent extends Document {
  studentId: string;
  userId?: mongoose.Types.ObjectId;
  name: string;
  email: string;
  phone?: string;
  dateOfBirth?: Date;

```

```

program: string;
department: string;
course: string;
year: number;
section: string;
academicYear: string;
semester: number;
enrollmentDate?: Date;
status: StudentStatus;
profileImage?: string;
createdAt: Date;
updatedAt: Date;
}

const StudentSchema = new Schema<IStudent>(
  {
    studentId: { type: String, required: true, unique: true, uppercase: true, trim: true, index: true },
    userId: { type: Schema.Types.ObjectId, ref: 'User', index: true },
    name: { type: String, required: true, trim: true, index: true },
    email: { type: String, required: true, unique: true, lowercase: true, trim: true, index: true },
    phone: { type: String, trim: true },
    dateOfBirth: { type: Date },
    program: { type: String, required: true, default: 'B.Tech', trim: true, index: true },
    department: { type: String, required: true, trim: true, index: true },
    course: { type: String, required: true, default: 'B.Tech', trim: true },
    year: { type: Number, required: true, min: 1, max: 6 },
    section: { type: String, required: true, uppercase: true, trim: true },
    academicYear: { type: String, required: true, trim: true },
    semester: { type: Number, required: true, min: 1, max: 12, index: true },
    enrollmentDate: { type: Date, default: Date.now },
    status: { type: String, enum: ['ACTIVE', 'INACTIVE', 'ARCHIVED'], default: 'ACTIVE', index: true },
    profileImage: { type: String },
  },
  {
    timestamps: true,
  }
);

StudentSchema.index({ department: 1, semester: 1, section: 1 });
StudentSchema.index({ program: 1, department: 1, year: 1, semester: 1, section: 1 });

export const Student = mongoose.model<IStudent>('Student', StudentSchema);

```

A.2.3 Attendance API

Attendance Route Definitions

File: apps/api/src/routes/attendanceRoutes.ts

TYPESCRIPT

Purpose & Architecture: Exposes REST endpoints for querying attendance records, generating session rosters, and processing bulk mark operations.

```

import { Router } from 'express';
import {
  getAttendance,
  getClassAttendanceSheet,
  recordAttendanceBatch,
  updateAttendanceRecord,
} from '../controllers/attendanceController';
import { authenticateToken, requireRole } from '../middleware/auth';

const router = Router();

router.use(authenticateToken);

router.get('/', getAttendance);
router.get('/sheet', getClassAttendanceSheet);
router.post('/batch', requireRole('ADMIN', 'TEACHER'), recordAttendanceBatch);
router.put('/:id', requireRole('ADMIN', 'TEACHER'), updateAttendanceRecord);

export default router;

```

Attendance Controller Implementation

TYPESCRIPT

File: apps/api/src/controllers/attendanceController.ts

Purpose & Architecture: Handles session roster assembly with canonical cohort resolution, individual attendance recording, and atomic bulk batch updates.

```
import { Request, Response } from 'express';
import mongoose from 'mongoose';
import { AttendanceRecord } from '../models/AttendanceRecord';
import { Student } from '../models/Student';
import { Class } from '../models/Class';
import { studentMetricService } from '../services/studentMetricService';
import { logAuditEvent } from '../middleware/audit';
import { resolveCanonicalCohort, buildStudentQuery } from '../utils/canonicalCohort';

/**
 * Normalizes any date string (YYYY-MM-DD or ISO) to UTC midnight range
 * Eliminates timezone conversion skew across environments
 */
export function parseDateRange(dateStr: string) {
  const clean = (dateStr || '').split('T')[0];
  const parts = clean.split('-').map(Number);
  const y = parts[0] || 2026;
  const m = parts[1] || 9;
  const d = parts[2] || 1;
  const startOfDay = new Date(Date.UTC(y, m - 1, d, 0, 0, 0));
  const endOfDay = new Date(Date.UTC(y, m - 1, d, 23, 59, 59, 999));
  return { startOfDay, endOfDay, normalizedDate: startOfDay };
}

export async function getAttendance(req: Request, res: Response): Promise<void> {
  const { classId, subjectId, date, studentId } = req.query;

  try {
    const query: any = {};
    if (studentId) query.studentId = studentId;
    if (subjectId) query.subjectId = subjectId;
    if (classId) query.classId = classId;

    if (date) {
      const { startOfDay, endOfDay } = parseDateRange(date as string);
      query.date = { $gte: startOfDay, $lte: endOfDay };
    }

    const records = await AttendanceRecord.find(query)
      .populate('studentId', 'name studentId department section semester')
      .populate('subjectId', 'name code')
      .populate('markedBy', 'name email role')
      .sort({ date: -1 });

    res.json({
      success: true,
      data: records,
    });
  } catch (error: any) {
    res.status(500).json({
      success: false,
      message: 'Failed to retrieve attendance records.',
      errors: [error.message],
    });
  }
}

export async function getClassAttendanceSheet(req: Request, res: Response): Promise<void> {
  const { classId, program, programId, department, departmentId, year, semester, section, subjectId, date } = req.query;

  if (!date) {
    res.status(400).json({ success: false, message: 'Date is required.' });
    return;
  }

  try {
    let studentQuery: any = { status: 'ACTIVE' };

    if (classId && mongoose.Types.ObjectId.isValid(classId as string)) {
      const targetClass = await Class.findById(classId);
    }
  }
}
```

```

    if (targetClass) {
      studentQuery.department = targetClass.department;
      studentQuery.semester = targetClass.semester;
      studentQuery.section = targetClass.section;
      if (targetClass.year) studentQuery.year = targetClass.year;
      if (targetClass.program) studentQuery.program = targetClass.program;
    }
  } else {
    const resolved = await resolveCanonicalCohort({
      program: program as string,
      programId: programId as string,
      department: department as string,
      departmentId: departmentId as string,
      year: year as string,
      semester: semester as string,
      section: section as string,
    });

    studentQuery = buildStudentQuery(resolved, { status: 'ACTIVE' });
  }

  const students = await Student.find(studentQuery)
    .sort({ studentId: 1 })
    .lean();

  const studentIds = students.map((s) => s._id);

  // 1. Calculate overall/subject-wise current attendance for each student
  const historyMatch: any = { studentId: { $in: studentIds } };
  if (subjectId && mongoose.Types.ObjectId.isValid(subjectId as string)) {
    historyMatch.subjectId = new mongoose.Types.ObjectId(subjectId as string);
  }

  const attendanceStats = await AttendanceRecord.aggregate([
    { $match: historyMatch },
    {
      $group: {
        _id: '$studentId',
        totalClasses: { $sum: 1 },
        presentClasses: {
          $sum: { $cond: [{ $eq: ['$status', 'PRESENT'] }, 1, 0] },
        },
        absentClasses: {
          $sum: { $cond: [{ $eq: ['$status', 'ABSENT'] }, 1, 0] },
        },
      },
    },
  ]);

  const statsMap = new Map();
  attendanceStats.forEach((st) => {
    statsMap.set(st._id.toString(), st);
  });

  // 2. Fetch today's records for the selected date
  const { startOfDay, endOfDay } = parseDateRange(date as string);
  const todayQuery: any = {
    studentId: { $in: studentIds },
    date: { $gte: startOfDay, $lte: endOfDay },
  };
  if (subjectId && mongoose.Types.ObjectId.isValid(subjectId as string)) {
    todayQuery.subjectId = new mongoose.Types.ObjectId(subjectId as string);
  }

  const todayRecords = await AttendanceRecord.find(todayQuery).lean();
  const todayMap = new Map();
  todayRecords.forEach((r) => {
    todayMap.set(r.studentId.toString(), r);
  });

  // 3. Assemble sheet items
  const sheet = students.map((s) => {
    const sIdStr = s._id.toString();
    const existing = todayMap.get(sIdStr);
    const hist = statsMap.get(sIdStr) || { totalClasses: 0, presentClasses: 0, absentClasses: 0 };

    const percentage =
      hist.totalClasses > 0
        ? Math.round((hist.presentClasses / hist.totalClasses) * 1000) / 10

```

```

        : 0;

    return {
        studentId: s._id,
        studentNumber: s.studentId,
        studentName: s.name,
        department: s.department,
        section: s.section,
        semester: s.semester,
        recordId: existing ? existing._id : null,
        // Requirement 8: If no record exists for this date, status MUST be 'NOT_MARKED'!
        status: existing ? existing.status : 'NOT_MARKED',
        markedBy: existing ? existing.markedBy : null,
        currentAttendance: {
            totalClasses: hist.totalClasses,
            presentClasses: hist.presentClasses,
            absentClasses: hist.absentClasses,
            percentage,
            formatted: `${percentage}% (${hist.presentClasses} of ${hist.totalClasses} classes attended)`,
        },
    };
});

res.json({
    success: true,
    data: {
        date: startOfDay,
        students: sheet,
    },
});
} catch (error: any) {
    res.status(500).json({
        success: false,
        message: 'Failed to retrieve attendance sheet.',
        errors: [error.message],
    });
}
}

export async function recordAttendanceBatch(req: Request, res: Response): Promise<void> {
    const { classId, subjectId, date, records } = req.body;

    if (!subjectId || !date || !Array.isArray(records)) {
        res.status(400).json({
            success: false,
            message: 'Subject, Date, and Records array are required.',
        });
        return;
    }

    try {
        const { normalizedDate } = parseDateRange(date as string);

        // Only process marked records (PRESENT or ABSENT)
        const validRecords = records.filter(
            (r: any) => r && (r.status === 'PRESENT' || r.status === 'ABSENT')
        );

        const bulkOps = validRecords.map((r: { studentId: string; status: 'PRESENT' | 'ABSENT' }) => ({
            updateOne: {
                filter: {
                    studentId: r.studentId,
                    subjectId,
                    date: normalizedDate,
                },
                update: {
                    $set: {
                        studentId: r.studentId,
                        subjectId,
                        classId: classId || undefined,
                        date: normalizedDate,
                        status: r.status,
                        markedBy: req.user?.id,
                    },
                },
            },
            upsert: true,
        }));
    }
}

```

```

    if (bulkOps.length > 0) {
      await AttendanceRecord.bulkWrite(bulkOps as any);
    }

    // Trigger recalculation for affected students
    const studentIds = validRecords.map((r: any) => r.studentId);
    Promise.all(
      studentIds.map((sid: string) => studentMetricService.calculateStudentMetrics(sid).catch(() => null))
    ).catch(() => null);

    await logAuditEvent(
      req.user?.id,
      req.user?.name,
      req.user?.email,
      'RECORD_ATTENDANCE_BATCH',
      'ATTENDANCE',
      classId?.toString(),
      { count: validRecords.length, date: normalizedDate, subjectId },
      req.ip
    );

    res.status(200).json({
      success: true,
      message: `Successfully saved attendance records for ${validRecords.length} students on ${date}.`,
    });
  } catch (error: any) {
    res.status(500).json({
      success: false,
      message: 'Failed to save attendance records.',
      errors: [error.message],
    });
  }
}

export async function updateAttendanceRecord(req: Request, res: Response): Promise<void> {
  const { id } = req.params;
  const { status, date } = req.body;

  try {
    const record = await AttendanceRecord.findById(id);
    if (!record) {
      res.status(404).json({ success: false, message: 'Attendance record not found.' });
      return;
    }

    if (status) record.status = status;
    if (date) {
      const { normalizedDate } = parseDateRange(date as string);
      record.date = normalizedDate;
    }
    record.markedBy = req.user?.id ? new mongoose.Types.ObjectId(req.user.id) : record.markedBy;

    await record.save();

    // Recalculate metrics
    if (record.studentId) {
      studentMetricService.calculateStudentMetrics(record.studentId).catch(() => null);
    }

    await logAuditEvent(
      req.user?.id,
      req.user?.name,
      req.user?.email,
      'UPDATE_ATTENDANCE_RECORD',
      'ATTENDANCE',
      record._id.toString(),
      { newStatus: record.status },
      req.ip
    );

    res.json({
      success: true,
      message: 'Attendance record updated successfully.',
      data: record,
    });
  } catch (error: any) {
    res.status(500).json({
      success: false,
      message: 'Failed to update attendance record.',
    });
  }
}

```

```

    errors: [error.message],
  });
}
}

```

Attendance Record Schema

[TYPESCRIPT](#)

File: apps/api/src/models/AttendanceRecord.ts

Purpose & Architecture: Mongoose Schema storing lecture attendance records with compound index uniqueness on studentId, subjectId, and date.

```

import mongoose, { Document, Schema } from 'mongoose';
import { AttendanceStatus } from '@eduguard/shared';

export interface IAttendanceRecord extends Document {
  studentId: mongoose.Types.ObjectId;
  subjectId: mongoose.Types.ObjectId;
  classId?: mongoose.Types.ObjectId;
  date: Date;
  status: AttendanceStatus;
  markedBy?: mongoose.Types.ObjectId;
  percentage?: number;
  createdAt: Date;
  updatedAt: Date;
}

const AttendanceRecordSchema = new Schema<IAttendanceRecord>(
  {
    studentId: { type: Schema.Types.ObjectId, ref: 'Student', required: true, index: true },
    subjectId: { type: Schema.Types.ObjectId, ref: 'Subject', required: true, index: true },
    classId: { type: Schema.Types.ObjectId, ref: 'Class', index: true },
    date: { type: Date, required: true, index: true },
    status: { type: String, enum: ['PRESENT', 'ABSENT', 'LATE', 'EXCUSED'], required: true },
    markedBy: { type: Schema.Types.ObjectId, ref: 'User' },
    percentage: { type: Number, min: 0, max: 100 },
  },
  {
    timestamps: true,
  }
);

AttendanceRecordSchema.index({ studentId: 1, subjectId: 1, date: 1 }, { unique: true });
AttendanceRecordSchema.index({ classId: 1, subjectId: 1, date: 1 });
AttendanceRecordSchema.index({ studentId: 1, date: 1 });

export const AttendanceRecord = mongoose.model<IAttendanceRecord>('AttendanceRecord', AttendanceRecordSchema);

```

Student Metric Service & Attendance Calculation

[TYPESCRIPT](#)

File: apps/api/src/services/studentMetricService.ts

Purpose & Architecture: Computes comprehensive behavioral metrics for AI prediction: aggregate attendance percentage, assignment submission rate, internal assessment performance, and study time.

```

import mongoose from 'mongoose';
import { Student } from '../models/Student';
import { AttendanceRecord } from '../models/AttendanceRecord';
import { Assessment } from '../models/Assessment';
import { Assignment } from '../models/Assignment';
import { AssignmentSubmission } from '../models/AssignmentSubmission';
import { StudyLog } from '../models/StudyLog';
import { ParticipationRecord } from '../models/ParticipationRecord';
import { AcademicRecord } from '../models/AcademicRecord';
import { Subject } from '../models/Subject';
import { Program } from '../models/Program';
import '../models/Class';
import '../models/User';
import {
  FeatureInputs,
  StudentCalculatedMetricsDTO,
  StudentFullAcademicHistoryDTO,
}

```

```

YearAcademicHistory,
SemesterAcademicHistory,
SubjectAcademicHistory,
} from '@eduguard/shared';

export class StudentMetricService {
  /**
   * Calculate all 6 student metrics from underlying raw MongoDB records
   */
  async calculateStudentMetrics(studentId: string | mongoose.Types.ObjectId): Promise<StudentCalculatedMetricsDTO> {
    const student = await Student.findById(studentId);
    if (!student) {
      throw new Error(`Student not found with ID: ${studentId}`);
    }

    const sId = student._id;

    // 1. ATTENDANCE CALCULATION
    const attendanceRecords = await AttendanceRecord.find({ studentId: sId })
      .populate('subjectId', 'name code')
      .sort({ date: -1 })
      .lean();

    const totalAttendanceCount = attendanceRecords.length;
    const presentAttendanceCount = attendanceRecords.filter((r) => r.status === 'PRESENT').length;
    const absentAttendanceCount = attendanceRecords.filter((r) => r.status === 'ABSENT').length;

    let calculatedAttendance = 0;
    let attendanceSubtext = 'No attendance recorded';
    let attendanceFormula = '0 / 0 x 100 = 0%';

    // Subject-wise attendance calculation (Requirement 10 & 13)
    const subjectMap = new Map<string, { subjectId: string; name: string; code: string; total: number; present: number; absent:
number; percentage: number }>();
    attendanceRecords.forEach((r) => {
      const sub = r.subjectId as any;
      const subId = sub?._id?.toString() || r.subjectId?.toString() || 'unknown';
      const subName = sub?.name || 'General Curriculum';
      const subCode = sub?.code || 'GEN';
      if (!subjectMap.has(subId)) {
        subjectMap.set(subId, {
          subjectId: subId,
          name: subName,
          code: subCode,
          total: 0,
          present: 0,
          absent: 0,
          percentage: 0,
        });
      }
      const item = subjectMap.get(subId)!;
      item.total += 1;
      if (r.status === 'PRESENT') item.present += 1;
      else if (r.status === 'ABSENT') item.absent += 1;
    });

    const bySubject = Array.from(subjectMap.values()).map((s) => ({
      ...s,
      percentage: s.total > 0 ? Math.round((s.present / s.total) * 1000) / 10 : 0,
    }));

    if (totalAttendanceCount > 0) {
      calculatedAttendance = Math.round((presentAttendanceCount / totalAttendanceCount) * 1000) / 10;
      attendanceSubtext = `${presentAttendanceCount} of ${totalAttendanceCount} classes attended`;
      attendanceFormula = `${presentAttendanceCount} / ${totalAttendanceCount} x 100 = ${calculatedAttendance}%`;
    } else {
      // Fallback to legacy academic record if no granular attendance logged yet
      const legacyRecord = await AcademicRecord.findOne({ studentId: sId });
      if (legacyRecord && legacyRecord.attendance !== undefined) {
        calculatedAttendance = legacyRecord.attendance;
        attendanceSubtext = 'From academic profile record';
        attendanceFormula = `Stored baseline: ${calculatedAttendance}%`;
      }
    }

    // 2. PREVIOUS SCORE CALCULATION
    // Requirement 14: Comes from student's previous academic history (earlier semesters)
    let calculatedPreviousScore = 75;
    let previousScoreSubtext = 'No prior semester records';
  }
}

```

```

let previousScoreFormula = 'Standard entry baseline';
let previousAcademicRecords: any[] = [];

// Prior semester assessments
const priorAssessments = await Assessment.find({
  studentId: sId,
  semester: { $lt: student.semester },
}).lean();

if (priorAssessments.length > 0) {
  const priorObtained = priorAssessments.reduce((sum, a) => sum + (a.obtainedMarks || 0), 0);
  const priorMax = priorAssessments.reduce((sum, a) => sum + (a.maximumMarks || 0), 0);
  if (priorMax > 0) {
    calculatedPreviousScore = Math.round((priorObtained / priorMax) * 100) / 10;
    const priorSem = student.semester > 1 ? student.semester - 1 : 1;
    previousScoreSubtext = `From Semester 1 to ${priorSem} assessments`;
    previousScoreFormula = `${Math.round(priorObtained * 10) / 10} / ${priorMax} × 100 = ${calculatedPreviousScore}%`;
  }
} else {
  // Search for prior semester academic records (semester < current semester)
  const priorRecords = await AcademicRecord.find({
    studentId: sId,
    semester: { $lt: student.semester },
  })
  .populate('subjectId', 'name code')
  .sort({ semester: -1 })
  .lean();

  if (priorRecords.length > 0) {
    previousAcademicRecords = priorRecords;
    const priorSem = priorRecords[0].semester;
    const semRecords = priorRecords.filter((r) => r.semester === priorSem);

    const totalScore = semRecords.reduce((sum, r) => {
      const val = r.finalScore != null ? r.finalScore : r.previousScore;
      return sum + (val || 0);
    }, 0);

    calculatedPreviousScore = Math.round((totalScore / semRecords.length) * 10) / 10;
    previousScoreSubtext = `From Semester ${priorSem}`;
    previousScoreFormula = `Average of Semester ${priorSem} records: ${calculatedPreviousScore} / 100`;
  } else {
    const anyRecord = await AcademicRecord.findOne({ studentId: sId });
    if (anyRecord && anyRecord.previousScore !== undefined) {
      calculatedPreviousScore = anyRecord.previousScore;
      const priorSem = student.semester > 1 ? student.semester - 1 : 1;
      previousScoreSubtext = `From Semester ${priorSem}`;
      previousScoreFormula = `Semester ${priorSem} baseline: ${calculatedPreviousScore} / 100`;
    }
  }
}

// 3. CURRENT ASSESSMENT MARKS CALCULATION
// Requirement 4 & 9: Assessment Performance from current semester assessments
const currentSemAssessments = await Assessment.find({
  studentId: sId,
  semester: student.semester,
})
.populate('subjectId', 'name code')
.sort({ date: -1 })
.lean();

const allAssessments =
  currentSemAssessments.length > 0
    ? currentSemAssessments
    : await Assessment.find({ studentId: sId })
      .populate('subjectId', 'name code')
      .sort({ date: -1 })
      .lean();

let calculatedInternalMarks = 0;
let internalMarksSubtext = 'No assessments recorded';
let internalMarksFormula = '0 / 0 × 100 = 0%';
let totalObtainedMarks = 0;
let totalMaximumMarks = 0;

if (allAssessments.length > 0) {
  totalObtainedMarks = allAssessments.reduce((sum, a) => sum + (a.obtainedMarks || 0), 0);
  totalMaximumMarks = allAssessments.reduce((sum, a) => sum + (a.maximumMarks || 0), 0);
}

```

```

    if (totalMaximumMarks > 0) {
      calculatedInternalMarks = Math.round((totalObtainedMarks / totalMaximumMarks) * 1000) / 10;
      internalMarksSubtext = `Assessment Performance: ${Math.round(totalObtainedMarks * 10) / 10} of ${totalMaximumMarks} mark
s`;
      internalMarksFormula = `${Math.round(totalObtainedMarks * 10) / 10} / ${totalMaximumMarks} × 100 = ${calculatedInternalM
arks}%`;
    }
  } else {
    const legacyRecord = await AcademicRecord.findOne({ studentId: sId });
    if (legacyRecord && legacyRecord.internalMarks !== undefined) {
      calculatedInternalMarks = legacyRecord.internalMarks;
      internalMarksSubtext = 'From academic profile record';
      internalMarksFormula = `Stored baseline: ${calculatedInternalMarks} / 100`;
    }
  }
}

// 4. ASSIGNMENT CALCULATION
// Requirement 12: Completed Assignments / Total Assignments × 100
const submissions = await AssignmentSubmission.find({ studentId: sId })
  .populate('assignmentId')
  .sort({ createdAt: -1 })
  .lean();

let calculatedAssignments = 0;
let assignmentSubtext = 'No assignments tracked';
let assignmentFormula = '0 / 0 × 100 = 0%';
const totalAssignments = submissions.length;
const completedAssignments = submissions.filter(
  (s) => s.status === 'SUBMITTED' || (s as any).status === 'GRADED'
).length;
const notCompletedAssignments = totalAssignments - completedAssignments;

if (totalAssignments > 0) {
  calculatedAssignments = Math.round((completedAssignments / totalAssignments) * 1000) / 10;
  assignmentSubtext = `${completedAssignments} of ${totalAssignments} completed`;
  assignmentFormula = `${completedAssignments} / ${totalAssignments} × 100 = ${calculatedAssignments}%`;
} else {
  const legacyAssignments = await Assignment.find({ studentId: sId }).lean();
  if (legacyAssignments.length > 0) {
    const total = legacyAssignments.length;
    const comp = legacyAssignments.filter((a) => a.status === 'SUBMITTED' || a.status === 'GRADED').length;
    calculatedAssignments = Math.round((comp / total) * 1000) / 10;
    assignmentSubtext = `${comp} of ${total} completed`;
    assignmentFormula = `${comp} / ${total} × 100 = ${calculatedAssignments}%`;
  } else {
    const legacyRecord = await AcademicRecord.findOne({ studentId: sId });
    if (legacyRecord && legacyRecord.assignmentCompletion !== undefined) {
      calculatedAssignments = legacyRecord.assignmentCompletion;
      assignmentSubtext = 'From academic profile record';
      assignmentFormula = `Stored baseline: ${calculatedAssignments}%`;
    }
  }
}

// 5. STUDY HOURS CALCULATION
const studyLogs = await StudyLog.find({ studentId: sId })
  .populate('subjectId', 'name code')
  .sort({ date: -1 })
  .lean();

let calculatedStudyHours = 0;
let studyHoursSubtext = 'No study hours logged';
let studyHoursFormula = '0 hours logged';

if (studyLogs.length > 0) {
  const oneWeekAgo = new Date();
  oneWeekAgo.setDate(oneWeekAgo.getDate() - 7);

  const recentLogs = studyLogs.filter((l) => new Date(l.date) >= oneWeekAgo);
  const effectiveLogs = recentLogs.length > 0 ? recentLogs : studyLogs.slice(0, 14);

  const totalHoursLogged = effectiveLogs.reduce((sum, l) => sum + (l.hours || 0), 0);
  calculatedStudyHours = Math.round(totalHoursLogged * 10) / 10;
  studyHoursSubtext = 'Based on study logs';
  studyHoursFormula = `${effectiveLogs.map((l) => `${l.hours}h`).slice(0, 5).join(' + ')}${effectiveLogs.length > 5 ? ' +
...': ''} = ${calculatedStudyHours} hrs/week`;
} else {
  const legacyRecord = await AcademicRecord.findOne({ studentId: sId });

```

```

    if (legacyRecord && legacyRecord.studyHours !== undefined) {
      calculatedStudyHours = legacyRecord.studyHours;
      studyHoursSubtext = 'From academic profile record';
      studyHoursFormula = `Stored baseline: ${calculatedStudyHours} hrs/week`;
    }
  }

// 6. PARTICIPATION CALCULATION
const participationRecords = await ParticipationRecord.find({ studentId: sId })
  .populate('subjectId', 'name code')
  .sort({ date: -1 })
  .lean();

let calculatedParticipation = 0;
let participationSubtext = 'No participation recorded';
let participationFormula = '0 / 0 × 100 = 0%';
let partObtainedTotal = 0;
let partMaxTotal = 0;

if (participationRecords.length > 0) {
  partObtainedTotal = participationRecords.reduce((sum, p) => sum + (p.obtainedScore || 0), 0);
  partMaxTotal = participationRecords.reduce((sum, p) => sum + (p.maximumScore || 0), 0);

  if (partMaxTotal > 0) {
    calculatedParticipation = Math.round((partObtainedTotal / partMaxTotal) * 1000) / 10;
    participationSubtext = `Based on ${partObtainedTotal} of ${partMaxTotal} points`;
    participationFormula = `${partObtainedTotal} / ${partMaxTotal} × 100 = ${calculatedParticipation}%`;
  }
} else {
  const legacyRecord = await AcademicRecord.findOne({ studentId: sId });
  if (legacyRecord && legacyRecord.participation !== undefined) {
    calculatedParticipation = legacyRecord.participation;
    participationSubtext = 'From academic profile record';
    participationFormula = `Stored baseline: ${calculatedParticipation}%`;
  }
}

// Compose feature inputs strictly for ANN
const featureInputs: FeatureInputs = {
  attendance: Math.max(0, Math.min(100, calculatedAttendance)),
  previousScore: Math.max(0, Math.min(100, calculatedPreviousScore)),
  internalMarks: Math.max(0, Math.min(100, calculatedInternalMarks)),
  assignmentCompletion: Math.max(0, Math.min(100, calculatedAssignments)),
  studyHours: Math.max(0, Math.min(100, calculatedStudyHours)),
  participation: Math.max(0, Math.min(100, calculatedParticipation)),
};

return {
  attendance: {
    value: calculatedAttendance,
    formattedValue: `${calculatedAttendance}%`,
    subtext: attendanceSubtext,
    formula: attendanceFormula,
    source: 'Attendance Records',
    records: attendanceRecords as any,
    bySubject: bySubject as any,
    stats: {
      present: presentAttendanceCount,
      absent: absentAttendanceCount,
      total: totalAttendanceCount,
    },
  },
  previousScore: {
    value: calculatedPreviousScore,
    formattedValue: `${calculatedPreviousScore}%`,
    subtext: previousScoreSubtext,
    formula: previousScoreFormula,
    source: 'Previous Academic Record',
    records: previousAcademicRecords,
    stats: {
      score: calculatedPreviousScore,
    },
  },
  internalMarks: {
    value: calculatedInternalMarks,
    formattedValue: `${calculatedInternalMarks}%`,
    subtext: internalMarksSubtext,
    formula: internalMarksFormula,
    source: 'Assessment Records',
  },
};

```

```

    records: allAssessments as any,
    stats: {
      obtained: totalObtainedMarks,
      maximum: totalMaximumMarks,
      count: allAssessments.length,
    },
  },
  assignmentCompletion: {
    value: calculatedAssignments,
    formattedValue: `${calculatedAssignments}%`,
    subtext: assignmentSubtext,
    formula: assignmentFormula,
    source: 'Assignment Submissions',
    records: submissions as any,
    stats: {
      completed: completedAssignments,
      notCompleted: notCompletedAssignments,
      total: totalAssignments,
    },
  },
  studyHours: {
    value: calculatedStudyHours,
    formattedValue: `${calculatedStudyHours} hrs/week`,
    subtext: studyHoursSubtext,
    formula: studyHoursFormula,
    source: 'Student Study Logs',
    records: studyLogs as any,
    stats: {
      weeklyHours: calculatedStudyHours,
      totalEntries: studyLogs.length,
    },
  },
  participation: {
    value: calculatedParticipation,
    formattedValue: `${calculatedParticipation}%`,
    subtext: participationSubtext,
    formula: participationFormula,
    source: 'Teacher Participation Records',
    records: participationRecords as any,
    stats: {
      obtained: partObtainedTotal,
      maximum: partMaxTotal,
      totalEntries: participationRecords.length,
    },
  },
  featureInputs,
};
}

/**
 * Generates complete structured academic history for a student:
 * Program -> Year -> Semester -> Subject -> Standard Assessments (Mid 1, Mid 2, Internal Lab 1, Internal Lab 2, External Lab,
Assignment)
 */
async getFullAcademicHistory(studentId: string | mongoose.Types.ObjectId): Promise<StudentFullAcademicHistoryDTO> {
  const student = await Student.findById(studentId).lean();
  if (!student) {
    throw new Error(`Student not found with ID: ${studentId}`);
  }

  const programName = student.program || student.course || 'B.Tech';
  const programDoc = await Program.findOne({ name: programName }).lean();
  const durationYears = programDoc?.durationYears || (programName === 'B.Tech' ? 4 : 3);
  const totalSemesters = programDoc?.totalSemesters || durationYears * 2;

  // Find all subjects for student's department & program
  const departmentSubjects = await Subject.find({
    $or: [{ department: student.department }, { program: programName }],
  }).lean();

  // Fetch all student assessments
  const assessments = await Assessment.find({ studentId: student._id })
    .populate('subjectId', 'name code credits')
    .lean();

  // Group assessments by semester and subjectId
  const semSubAssessments = new Map<string, any[]>();
  for (const a of assessments) {
    const subId = (a.subjectId as any)?._id?.toString() || a.subjectId?.toString();

```

```

    if (!subId) continue;
    const key = `${a.semester}_${subId}`;
    if (!semSubAssessments.has(key)) {
        semSubAssessments.set(key, []);
    }
    semSubAssessments.get(key)!.push(a);
}

const yearLabels = ['1st Year', '2nd Year', '3rd Year', '4th Year', '5th Year'];
const years: YearAcademicHistory[] = [];

const standardTypes = [
    'Mid 1',
    'Mid 2',
    'Internal Lab 1',
    'Internal Lab 2',
    'External Lab',
    'Assignment',
];

for (let y = 1; y <= durationYears; y++) {
    const sem1 = (y - 1) * 2 + 1;
    const sem2 = (y - 1) * 2 + 2;
    const semestersInYear = [sem1, sem2].filter((s) => s <= totalSemesters);

    const semesterHistories: SemesterAcademicHistory[] = [];

    for (const semNum of semestersInYear) {
        // Find subjects for this semester
        let semSubjects = departmentSubjects.filter((s) => s.semester === semNum);

        // Also include any subject where student already has an assessment in this semester
        for (const a of assessments) {
            if (a.semester === semNum && a.subjectId) {
                const aSub = a.subjectId as any;
                const aSubId = aSub._id?.toString() || aSub.toString();
                if (!semSubjects.some((s) => s._id.toString() === aSubId)) {
                    semSubjects.push(aSub);
                }
            }
        }

        // If no subjects exist yet for this semester and student is in or past this semester, provide default curriculum subjects
        if (semSubjects.length === 0 && semNum <= student.semester) {
            semSubjects = [
                {
                    _id: new mongoose.Types.ObjectId() as any,
                    name: semNum === 1 ? 'Programming Fundamentals' : semNum === 2 ? 'Object Oriented Programming' : `Core Subject ${semNum}.1`,
                    code: `CS${semNum}01`,
                    credits: 4,
                    program: programName,
                    department: student.department,
                    semester: semNum,
                } as any,
                {
                    _id: new mongoose.Types.ObjectId() as any,
                    name: semNum === 1 ? 'Engineering Mathematics I' : semNum === 2 ? 'Engineering Mathematics II' : `Core Subject ${semNum}.2`,
                    code: `CS${semNum}02`,
                    credits: 4,
                    program: programName,
                    department: student.department,
                    semester: semNum,
                } as any,
            ];
        }

        const subjectHistories: SubjectAcademicHistory[] = [];
        let semTotalObtained = 0;
        let semTotalMaximum = 0;

        for (const sub of semSubjects) {
            const subId = sub._id.toString();
            const key = `${semNum}_${subId}`;
            const subAssessments = semSubAssessments.get(key) || [];

            const marksMap: Record<string, { obtainedMarks: number; maximumMarks: number }> = {};

```

```

    let subObtained = 0;
    let subMaximum = 0;

    for (const st of standardTypes) {
      const found = subAssessments.find((a) => {
        if (a.assessmentType === st) return true;
        if (st === 'Mid 1' && (a.assessmentType === 'Internal Assessment 1' || a.assessmentType === 'Mid Examination')) re
turn true;
        if (st === 'Mid 2' && a.assessmentType === 'Internal Assessment 2') return true;
        return false;
      });

      if (found) {
        marksMap[st] = {
          obtainedMarks: found.obtainedMarks,
          maximumMarks: found.maximumMarks,
        };
        subObtained += found.obtainedMarks;
        subMaximum += found.maximumMarks;
      }
    }

    const assessmentPerformance = subMaximum > 0 ? Math.round((subObtained / subMaximum) * 1000) / 10 : 0;
    semTotalObtained += subObtained;
    semTotalMaximum += subMaximum;

    subjectHistories.push({
      subjectId: subId,
      name: sub.name,
      code: sub.code,
      credits: sub.credits,
      marks: marksMap,
      totalObtained: subObtained,
      totalMaximum: subMaximum,
      assessmentPerformance,
    });
  }

  const semPerformance = semTotalMaximum > 0 ? Math.round((semTotalObtained / semTotalMaximum) * 1000) / 10 : 0;

  semesterHistories.push({
    semester: semNum,
    year: y,
    subjects: subjectHistories,
    totalObtained: semTotalObtained,
    totalMaximum: semTotalMaximum,
    assessmentPerformance: semPerformance,
  });
}

years.push({
  year: y,
  yearLabel: yearLabels[y - 1] || `${y}th Year`,
  semesters: semesterHistories,
});
}

return {
  program: programName,
  durationYears,
  totalSemesters,
  currentYear: student.year,
  currentSemester: student.semester,
  years,
};
}
}

export const studentMetricService = new StudentMetricService();

```

A.2.4 Prediction API

Prediction Route Definitions

File: apps/api/src/routes/predictionRoutes.ts

TYPESCRIPT

Purpose & Architecture: Defines REST routes for retrieving historical prediction audit logs and triggering on-demand neural network inference.

```
import { Router } from 'express';
import { generatePrediction, getPredictions } from '../controllers/predictionController';
import { authenticateToken, requireRole } from '../middleware/auth';

const router = Router();

router.use(authenticateToken);
router.get('/', getPredictions);
router.post('/', requireRole('ADMIN', 'TEACHER'), generatePrediction);

export default router;
```

Prediction Controller Implementation

TYPESCRIPT

File: apps/api/src/controllers/predictionController.ts

Purpose & Architecture: Coordinates feature extraction from database records, dispatches payloads to the Python ML microservice, generates diagnostic risk factors, and persists predictions.

```
import { Request, Response } from 'express';
import { Student } from '../models/Student';
import { AcademicRecord } from '../models/AcademicRecord';
import { Prediction } from '../models/Prediction';
import { SystemSettings } from '../models/SystemSettings';
import { Notification } from '../models/Notification';
import { User } from '../models/User';
import { mlClientService } from '../services/mlClientService';
import { studentMetricService } from '../services/studentMetricService';
import { analyzeRiskFactors } from '../services/riskFactorService';
import { generateRecommendations } from '../services/recommendationService';
import { logAuditEvent } from '../middleware/audit';
import { FeatureInputs, RiskLevel } from '@eduguard/shared';

export async function generatePrediction(req: Request, res: Response): Promise<void> {
  const { studentId, features } = req.body;

  if (!studentId) {
    res.status(400).json({ success: false, message: 'Student ID is required.' });
    return;
  }

  try {
    const student = await Student.findById(studentId);
    if (!student) {
      res.status(404).json({ success: false, message: 'Student not found.' });
      return;
    }

    // Resolve feature values: either explicit from request or computed from real underlying student records
    let inputFeatures: FeatureInputs;

    if (features) {
      inputFeatures = {
        attendance: Number(features.attendance),
        previousScore: Number(features.previousScore),
        internalMarks: Number(features.internalMarks),
        assignmentCompletion: Number(features.assignmentCompletion),
        studyHours: Number(features.studyHours),
        participation: Number(features.participation),
      };
    } else {
      const metricResult = await studentMetricService.calculateStudentMetrics(student._id);
      inputFeatures = metricResult.featureInputs;
    }

    // Validate feature ranges
    const validationErrors: string[] = [];
    if (isNaN(inputFeatures.attendance) || inputFeatures.attendance < 0 || inputFeatures.attendance > 100) {
      validationErrors.push('Attendance must be between 0 and 100%');
    }
    if (isNaN(inputFeatures.previousScore) || inputFeatures.previousScore < 0 || inputFeatures.previousScore > 100) {
```

```

    validationErrors.push('Previous score must be between 0 and 100');
  }
  if (isNaN(inputFeatures.internalMarks) || inputFeatures.internalMarks < 0 || inputFeatures.internalMarks > 100) {
    validationErrors.push('Internal marks must be between 0 and 100');
  }
  if (isNaN(inputFeatures.assignmentCompletion) || inputFeatures.assignmentCompletion < 0 || inputFeatures.assignmentCompletion > 100) {
    validationErrors.push('Assignment completion must be between 0 and 100%');
  }
  if (isNaN(inputFeatures.participation) || inputFeatures.participation < 0 || inputFeatures.participation > 100) {
    validationErrors.push('Participation must be between 0 and 100%');
  }
  if (isNaN(inputFeatures.studyHours) || inputFeatures.studyHours < 0 || inputFeatures.studyHours > 100) {
    validationErrors.push('Study hours must be between 0 and 100 hrs/week');
  }

  if (validationErrors.length > 0) {
    res.status(400).json({
      success: false,
      message: 'Invalid feature values provided.',
      errors: validationErrors,
    });
    return;
  }

  // Load active institutional risk thresholds
  const settings = await (SystemSettings as any).getOrCreateSettings();
  const thresholds = settings.riskThresholds;

  // Call Python FastAPI ML Service
  const mlResult = await mlClientService.predict(inputFeatures);

  // Clamp predicted score between 0 and 100
  const rawScore = Number(mlResult.predictedScore);
  const predictedScore = Math.max(0, Math.min(100, Math.round(rawScore * 10) / 10));

  // Dynamic Risk Classification based on institutional thresholds
  let riskLevel: RiskLevel = 'LOW';
  if (predictedScore < thresholds.highRiskBelow) {
    riskLevel = 'HIGH';
  } else if (predictedScore < thresholds.mediumRiskBelow) {
    riskLevel = 'MEDIUM';
  } else {
    riskLevel = 'LOW';
  }

  // Transparent Factor Analysis
  const riskFactors = analyzeRiskFactors(inputFeatures, thresholds);

  // Supportive Guidance Recommendations
  const recommendations = generateRecommendations(riskLevel, predictedScore, riskFactors);

  // Store Prediction Snapshot for full historical reproducibility
  const prediction = await Prediction.create({
    studentId: student._id,
    modelVersion: mlResult.modelVersion || settings.activeModelVersion,
    inputSnapshot: inputFeatures,
    predictedScore,
    riskLevel,
    riskFactors,
    recommendations,
    thresholdSnapshot: thresholds,
    createdBy: req.user?.id,
  });

  // Check if risk alert notification is needed
  if (riskLevel === 'HIGH') {
    // Find institutional admins and teachers
    const adminUsers = await User.find({ role: 'ADMIN', status: 'ACTIVE' }).limit(5);
    for (const admin of adminUsers) {
      await Notification.create({
        recipientId: admin._id,
        type: 'RISK_ALERT',
        title: `High Risk Alert: ${student.name}`,
        message: `Student ${student.name} (${student.studentId}) in ${student.department} has been identified with Predicted Score ${predictedScore.toFixed(1)} and marked as High Risk.`,
        relatedStudentId: student._id,
        relatedPredictionId: prediction._id,
      });
    }
  }

```

```

    }
  }

  // Audit Log
  await logAuditEvent(
    req.user?.id,
    req.user?.name,
    req.user?.email,
    'GENERATE_PREDICTION',
    'PREDICTION',
    prediction._id.toString(),
    {
      studentId: student.studentId,
      predictedScore,
      riskLevel,
      modelVersion: mlResult.modelVersion,
    },
    req.ip
  );

  res.status(201).json({
    success: true,
    message: 'AI prediction and risk analysis completed successfully.',
    data: {
      _id: prediction._id,
      studentId: student._id,
      studentName: student.name,
      predictedScore,
      riskLevel,
      riskFactors,
      recommendations,
      modelVersion: mlResult.modelVersion,
      thresholdSnapshot: thresholds,
      inputSnapshot: inputFeatures,
      createdAt: prediction.createdAt,
    },
  });
} catch (error: any) {
  const statusCode = error.statusCode || 500;
  res.status(statusCode).json({
    success: false,
    message: error.message || 'Failed to generate prediction.',
    errors: [error.message],
  });
}
}

export async function getPredictions(req: Request, res: Response): Promise<void> {
  const { studentId, riskLevel, page = '1', limit = '20' } = req.query;

  try {
    const pageNum = parseInt(page as string, 10) || 1;
    const limitNum = parseInt(limit as string, 10) || 20;
    const skip = (pageNum - 1) * limitNum;

    const query: any = {};
    if (studentId) query.studentId = studentId;
    if (riskLevel && riskLevel !== 'ALL') query.riskLevel = riskLevel;

    const total = await Prediction.countDocuments(query);
    const predictions = await Prediction.find(query)
      .populate('studentId', 'studentId name department section semester')
      .populate('createdBy', 'name email role')
      .sort({ createdAt: -1 })
      .skip(skip)
      .limit(limitNum);

    res.json({
      success: true,
      data: predictions,
      meta: {
        page: pageNum,
        limit: limitNum,
        total,
        totalPages: Math.ceil(total / limitNum),
      },
    });
  } catch (error: any) {
    res.status(500).json({

```

```

    success: false,
    message: 'Failed to retrieve predictions.',
    errors: [error.message],
  });
}
}

```

Prediction Data Model & Schema

File: apps/api/src/models/Prediction.ts

TYPESCRIPT

Purpose & Architecture: Schema storing AI model output, input feature snapshot, risk classification level, identified risk factors, and institutional thresholds.

```

import mongoose, { Document, Schema } from 'mongoose';
import { FeatureInputs, RiskFactor, RiskLevel, SystemRiskThresholds } from '@eduguard/shared';

export interface IPrediction extends Document {
  studentId: mongoose.Types.ObjectId;
  modelVersion: string;
  inputSnapshot: FeatureInputs;
  predictedScore: number;
  riskLevel: RiskLevel;
  riskFactors: RiskFactor[];
  recommendations: string[];
  thresholdSnapshot: SystemRiskThresholds;
  createdBy?: mongoose.Types.ObjectId;
  createdAt: Date;
  updatedAt: Date;
}

const RiskFactorSchema = new Schema(
  {
    factor: { type: String, required: true },
    category: { type: String, required: true },
    severity: { type: String, enum: ['LOW', 'MEDIUM', 'HIGH'], required: true },
    description: { type: String, required: true },
    observedValue: { type: Number },
    threshold: { type: Number },
  },
  { _id: false }
);

const PredictionSchema = new Schema<IPrediction>(
  {
    studentId: { type: Schema.Types.ObjectId, ref: 'Student', required: true, index: true },
    modelVersion: { type: String, required: true },
    inputSnapshot: {
      attendance: { type: Number, required: true },
      previousScore: { type: Number, required: true },
      internalMarks: { type: Number, required: true },
      assignmentCompletion: { type: Number, required: true },
      studyHours: { type: Number, required: true },
      participation: { type: Number, required: true },
    },
    predictedScore: { type: Number, required: true, min: 0, max: 100 },
    riskLevel: { type: String, enum: ['LOW', 'MEDIUM', 'HIGH'], required: true, index: true },
    riskFactors: [RiskFactorSchema],
    recommendations: [{ type: String, required: true }],
    thresholdSnapshot: {
      highRiskBelow: { type: Number, default: 50.0 },
      mediumRiskBelow: { type: Number, default: 70.0 },
      attendanceThreshold: { type: Number, default: 75.0 },
      previousScoreThreshold: { type: Number, default: 55.0 },
      internalMarksThreshold: { type: Number, default: 50.0 },
      assignmentCompletionThreshold: { type: Number, default: 70.0 },
      studyHoursThreshold: { type: Number, default: 8.0 },
      participationThreshold: { type: Number, default: 50.0 },
    },
    createdBy: { type: Schema.Types.ObjectId, ref: 'User' },
  },
  {
    timestamps: true,
  }
);

```

```
PredictionSchema.index({ studentId: 1, createdAt: -1 });
PredictionSchema.index({ riskLevel: 1, createdAt: -1 });

export const Prediction = mongoose.model<IPrediction>('Prediction', PredictionSchema);
```

A.2.5 Risk and Recommendation Logic

Explainable Risk Factor Evaluation Service

TYPESCRIPT

File: apps/api/src/services/riskFactorService.ts

Purpose & Architecture: Evaluates student academic indicators against institutional warning thresholds to generate explainable diagnostic flags with priority levels.

```
import { FeatureInputs, RiskFactor, SystemRiskThresholds } from '@eduguard/shared';

/**
 * Transparent factor-analysis layer.
 * System avoids claiming AI 'knows' causation; instead identifies measurable
 * academic indicators falling below institutional reference thresholds.
 */
export function analyzeRiskFactors(
  inputs: FeatureInputs,
  thresholds: SystemRiskThresholds
): RiskFactor[] {
  const factors: RiskFactor[] = [];

  // Attendance check
  if (inputs.attendance < thresholds.attendanceThreshold) {
    const diff = thresholds.attendanceThreshold - inputs.attendance;
    factors.push({
      factor: 'Low Attendance Rate',
      category: 'ATTENDANCE',
      severity: diff > 20 ? 'HIGH' : diff > 10 ? 'MEDIUM' : 'LOW',
      description: `Observed attendance (${inputs.attendance.toFixed(1)}%) is below institutional reference threshold of ${thresholds.attendanceThreshold}%`,
      observedValue: inputs.attendance,
      threshold: thresholds.attendanceThreshold,
    });
  }

  // Previous Academic Score check
  if (inputs.previousScore < thresholds.previousScoreThreshold) {
    const diff = thresholds.previousScoreThreshold - inputs.previousScore;
    factors.push({
      factor: 'Historical Examination Performance',
      category: 'HISTORICAL_PERFORMANCE',
      severity: diff > 20 ? 'HIGH' : diff > 10 ? 'MEDIUM' : 'LOW',
      description: `Previous score (${inputs.previousScore.toFixed(1)}) indicates foundational difficulty compared to standard benchmark (${thresholds.previousScoreThreshold})`,
      observedValue: inputs.previousScore,
      threshold: thresholds.previousScoreThreshold,
    });
  }

  // Internal Assessment Marks check
  if (inputs.internalMarks < thresholds.internalMarksThreshold) {
    const diff = thresholds.internalMarksThreshold - inputs.internalMarks;
    factors.push({
      factor: 'Low Internal Assessment Marks',
      category: 'INTERNAL_ASSESSMENT',
      severity: diff > 20 ? 'HIGH' : diff > 10 ? 'MEDIUM' : 'LOW',
      description: `Internal assessment score (${inputs.internalMarks.toFixed(1)}) indicates learning gaps in ongoing coursework benchmark (${thresholds.internalMarksThreshold})`,
      observedValue: inputs.internalMarks,
      threshold: thresholds.internalMarksThreshold,
    });
  }

  // Assignment Completion check
  if (inputs.assignmentCompletion < thresholds.assignmentCompletionThreshold) {
    const diff = thresholds.assignmentCompletionThreshold - inputs.assignmentCompletion;
    factors.push({
      factor: 'Low Assignment Completion Rate',
```

```

        category: 'ASSIGNMENT_COMPLETION',
        severity: diff > 25 ? 'HIGH' : diff > 10 ? 'MEDIUM' : 'LOW',
        description: `Assignment submission completion (${inputs.assignmentCompletion.toFixed(1)}%) is below expected completion t
hreshold (${thresholds.assignmentCompletionThreshold}%).`,
        observedValue: inputs.assignmentCompletion,
        threshold: thresholds.assignmentCompletionThreshold,
    });
}

// Study Hours check
if (inputs.studyHours < thresholds.studyHoursThreshold) {
    const diff = thresholds.studyHoursThreshold - inputs.studyHours;
    factors.push({
        factor: 'Limited Self-Study Hours',
        category: 'STUDY_HABITS',
        severity: diff > 4 ? 'HIGH' : 'MEDIUM',
        description: `Reported self-study time (${inputs.studyHours.toFixed(1)} hrs/week) is below recommended independent study t
arget (${thresholds.studyHoursThreshold} hrs/week).`,
        observedValue: inputs.studyHours,
        threshold: thresholds.studyHoursThreshold,
    });
}

// Participation check
if (inputs.participation < thresholds.participationThreshold) {
    const diff = thresholds.participationThreshold - inputs.participation;
    factors.push({
        factor: 'Low Classroom Participation',
        category: 'ENGAGEMENT',
        severity: diff > 25 ? 'HIGH' : 'LOW',
        description: `Course engagement & active classroom participation (${inputs.participation.toFixed(1)}%) is lower than typic
al baseline (${thresholds.participationThreshold}%).`,
        observedValue: inputs.participation,
        threshold: thresholds.participationThreshold,
    });
}

return factors;
}

```

Pedagogical Recommendation Advisory Service

File: apps/api/src/services/recommendationService.ts

TYPESCRIPT

Purpose & Architecture: Translates detected academic risk factors into tailored pedagogical guidance and actionable remedial recommendations.

```

import { RiskFactor, RiskLevel } from '@eduguard/shared';

/**
 * Educational guidance engine generating supportive, non-punitive intervention suggestions.
 */
export function generateRecommendations(
    riskLevel: RiskLevel,
    predictedScore: number,
    riskFactors: RiskFactor[]
): string[] {
    const recommendations: string[] = [];

    // General risk level guidance
    if (riskLevel === 'HIGH') {
        recommendations.push(
            'Recommend early academic advising session with class advisor to discuss overall coursework challenges.'
        );
        recommendations.push(
            'Consider enrolling the student in dedicated peer tutoring or remedial tutorial sessions before midterm exams.'
        );
    } else if (riskLevel === 'MEDIUM') {
        recommendations.push(
            'Encourage regular check-ins during faculty office hours to review difficult topic comprehension.'
        );
    } else {
        recommendations.push(
            'Student is performing on track. Encourage continuous learning and participation in advanced topic seminars.'
        );
    }
}

```

```

    }

    // Factor-specific recommendations
    for (const factor of riskFactors) {
      switch (factor.category) {
        case 'ATTENDANCE':
          recommendations.push(
            'Review attendance patterns with the student to identify and resolve timetable or commute constraints.'
          );
          break;
        case 'HISTORICAL_PERFORMANCE':
          recommendations.push(
            'Provide prerequisite refresher materials and diagnostic assessments for fundamental subject concepts.'
          );
          break;
        case 'INTERNAL_ASSESSMENT':
          recommendations.push(
            'Offer formative assessment retake opportunities or guided problem-solving workshops for upcoming internal tests.'
          );
          break;
        case 'ASSIGNMENT_COMPLETION':
          recommendations.push(
            'Establish a structured milestone submission calendar and provide assignment drafting guidelines.'
          );
          break;
        case 'STUDY_HABITS':
          recommendations.push(
            'Assist student in building a sustainable weekly study plan, balancing lecture hours with independent revision.'
          );
          break;
        case 'ENGAGEMENT':
          recommendations.push(
            'Encourage collaborative study groups and interactive laboratory participation to increase engagement.'
          );
          break;
      }
    }

    // Deduplicate and return at most 5 focused recommendations
    return Array.from(new Set(recommendations)).slice(0, 5);
  }
}

```

A.2.6 Intervention API

Intervention Route Definitions

File: apps/api/src/routes/interventionRoutes.ts

TYPESCRIPT

Purpose & Architecture: Exposes endpoints to query student counseling interventions, schedule new remedial sessions, and log intervention progress.

```

import { Router } from 'express';
import {
  createIntervention,
  getInterventions,
  updateIntervention,
} from '../controllers/interventionController';
import { authenticateToken, requireRole } from '../middleware/auth';

const router = Router();

router.use(authenticateToken);
router.get('/', getInterventions);
router.post('/', requireRole('ADMIN', 'TEACHER'), createIntervention);
router.put('/:id', requireRole('ADMIN', 'TEACHER'), updateIntervention);

export default router;

```

Intervention Controller Implementation

File: apps/api/src/controllers/interventionController.ts

TYPESCRIPT

Purpose & Architecture: Manages the lifecycle of academic interventions from assignment to completion, ensuring accountability and support follow-through.

```
import { Request, Response } from 'express';
import { Intervention } from '../models/Intervention';
import { Student } from '../models/Student';
import { Teacher } from '../models/Teacher';
import { Notification } from '../models/Notification';
import { logAuditEvent } from '../middleware/audit';

export async function createIntervention(req: Request, res: Response): Promise<void> {
  const {
    studentId,
    teacherId,
    predictionId,
    type,
    title,
    description,
    priority = 'MEDIUM',
    dueDate,
  } = req.body;

  if (!studentId || !type || !title || !description || !dueDate) {
    res.status(400).json({
      success: false,
      message: 'studentId, type, title, description, and dueDate are required.',
    });
    return;
  }

  try {
    const student = await Student.findById(studentId);
    if (!student) {
      res.status(404).json({ success: false, message: 'Student not found.' });
      return;
    }

    // Resolve assigned teacher
    let resolvedTeacherId = teacherId;
    if (!resolvedTeacherId && req.user?.teacherId) {
      resolvedTeacherId = req.user.teacherId;
    }

    if (!resolvedTeacherId) {
      const anyTeacher = await Teacher.findOne();
      resolvedTeacherId = anyTeacher?._id;
    }

    const intervention = await Intervention.create({
      studentId,
      teacherId: resolvedTeacherId,
      predictionId,
      type,
      title: title.trim(),
      description: description.trim(),
      priority,
      status: 'OPEN',
      dueDate: new Date(dueDate),
    });

    // Notify student if user account exists
    if (student.userId) {
      await Notification.create({
        recipientId: student.userId,
        type: 'INTERVENTION_DUE',
        title: `Academic Support Scheduled: ${title}`,
        message: `An academic support initiative (${type.replace('_', ' ')} has been scheduled for you due on ${new Date(dueDate).toLocaleDateString()}`,
        relatedStudentId: student._id,
      });
    }

    await logAuditEvent(
      req.user?.id,
      req.user?.name,
      req.user?.email,
      'CREATE_INTERVENTION',
    );
  } catch (error) {
    res.status(500).json({ success: false, message: 'Internal server error.' });
  }
}
```

```

    'INTERVENTION',
    intervention._id.toString(),
    { studentId, type, priority },
    req.ip
  );

  res.status(201).json({
    success: true,
    message: 'Intervention created successfully.',
    data: intervention,
  });
} catch (error: any) {
  res.status(500).json({
    success: false,
    message: 'Failed to create intervention.',
    errors: [error.message],
  });
}
}

export async function getInterventions(req: Request, res: Response): Promise<void> {
  const { studentId, teacherId, status, priority, page = '1', limit = '20' } = req.query;

  try {
    const pageNum = parseInt(page as string, 10) || 1;
    const limitNum = parseInt(limit as string, 10) || 20;
    const skip = (pageNum - 1) * limitNum;

    const query: any = {};
    if (status && status !== 'ALL') query.status = status;
    if (priority && priority !== 'ALL') query.priority = priority;
    if (studentId) query.studentId = studentId;

    if (req.user?.role === 'STUDENT' && req.user.studentId) {
      query.studentId = req.user.studentId;
    } else if (req.user?.role === 'TEACHER' && req.user.teacherId) {
      query.teacherId = req.user.teacherId;
    } else if (teacherId) {
      query.teacherId = teacherId;
    }

    const total = await Intervention.countDocuments(query);
    const interventions = await Intervention.find(query)
      .populate('studentId', 'studentId name department section semester')
      .populate({
        path: 'teacherId',
        populate: { path: 'userId', select: 'name email' },
      })
      .sort({ dueDate: 1, createdAt: -1 })
      .skip(skip)
      .limit(limitNum);

    res.json({
      success: true,
      data: interventions,
      meta: {
        page: pageNum,
        limit: limitNum,
        total,
        totalPages: Math.ceil(total / limitNum),
      },
    });
  } catch (error: any) {
    res.status(500).json({
      success: false,
      message: 'Failed to retrieve interventions.',
      errors: [error.message],
    });
  }
}

export async function updateIntervention(req: Request, res: Response): Promise<void> {
  const { id } = req.params;
  const { status, outcome, notes, priority, dueDate } = req.body;

  try {
    const intervention = await Intervention.findById(id);
    if (!intervention) {
      res.status(404).json({ success: false, message: 'Intervention not found.' });
    }
  }
}

```

```

    return;
  }

  if (status) intervention.status = status;
  if (outcome !== undefined) intervention.outcome = outcome;
  if (notes !== undefined) intervention.notes = notes;
  if (priority) intervention.priority = priority;
  if (dueDate) intervention.dueDate = new Date(dueDate);

  if (status === 'COMPLETED' && !intervention.completedAt) {
    intervention.completedAt = new Date();
  }

  await intervention.save();

  await logAuditEvent(
    req.user?.id,
    req.user?.name,
    req.user?.email,
    'UPDATE_INTERVENTION',
    'INTERVENTION',
    intervention._id.toString(),
    { status: intervention.status },
    req.ip
  );

  res.json({
    success: true,
    message: 'Intervention updated successfully.',
    data: intervention,
  });
} catch (error: any) {
  res.status(500).json({
    success: false,
    message: 'Failed to update intervention.',
    errors: [error.message],
  });
}
}

```

Intervention Data Model & Schema

File: apps/api/src/models/Intervention.ts

TYPESCRIPT

Purpose & Architecture: Mongoose Schema defining intervention details, student references, designated faculty counselor, intervention type, and follow-up milestones.

```

import mongoose, { Document, Schema } from 'mongoose';
import { InterventionPriority, InterventionStatus, InterventionType } from '@eduguard/shared';

export interface IIntervention extends Document {
  studentId: mongoose.Types.ObjectId;
  teacherId: mongoose.Types.ObjectId;
  predictionId?: mongoose.Types.ObjectId;
  type: InterventionType;
  title: string;
  description: string;
  priority: InterventionPriority;
  status: InterventionStatus;
  dueDate: Date;
  completedAt?: Date;
  outcome?: string;
  notes?: string;
  createdAt: Date;
  updatedAt: Date;
}

const InterventionSchema = new Schema<IIntervention>(
  {
    studentId: { type: Schema.Types.ObjectId, ref: 'Student', required: true, index: true },
    teacherId: { type: Schema.Types.ObjectId, ref: 'Teacher', required: true, index: true },
    predictionId: { type: Schema.Types.ObjectId, ref: 'Prediction' },
    type: {
      type: String,
      enum: [

```

```

    'ACADEMIC_COUNSELING',
    'REMEDIAL_SUPPORT',
    'ASSIGNMENT_FOLLOWUP',
    'ATTENDANCE_FOLLOWUP',
    'STUDY_PLANNING',
    'FACULTY_MEETING',
    'OTHER',
  ],
  required: true,
},
title: { type: String, required: true, trim: true },
description: { type: String, required: true, trim: true },
priority: { type: String, enum: ['LOW', 'MEDIUM', 'HIGH', 'URGENT'], default: 'MEDIUM' },
status: { type: String, enum: ['OPEN', 'IN_PROGRESS', 'COMPLETED', 'CANCELLED'], default: 'OPEN', index: true },
dueDate: { type: Date, required: true, index: true },
completedAt: { type: Date },
outcome: { type: String, trim: true },
notes: { type: String, trim: true },
},
{
  timestamps: true,
}
});

InterventionSchema.index({ studentId: 1, status: 1 });

export const Intervention = mongoose.model<IIIntervention>('Intervention', InterventionSchema);

```

A.2.7 ML Client Service

Python ML Microservice HTTP Client

File: apps/api/src/services/mlClientService.ts

TYPESCRIPT

Purpose & Architecture: Encapsulates authenticated HTTP communication with the Python FastAPI ML microservice, managing prediction payloads and health checks.

```

import axios from 'axios';
import { FeatureInputs } from '@eduguard/shared';
import { env } from '../config/environment';

export interface MLPredictionResponse {
  predictedScore: number;
  modelVersion: string;
  latencyMs?: number;
}

export class MLClientService {
  private getBaseUrl(): string {
    const rawUrl = process.env.ML_SERVICE_URL || env.ML_SERVICE_URL || 'http://localhost:8000';
    return rawUrl.endsWith('/') ? rawUrl : `${rawUrl}/`;
  }

  /**
   * Send student features to Python FastAPI ML service
   */
  async predict(features: FeatureInputs): Promise<MLPredictionResponse> {
    const targetUrl = new URL('predict', this.getBaseUrl()).toString();
    try {
      const response = await axios.post<MLPredictionResponse>(
        targetUrl,
        {
          attendance: features.attendance,
          previousScore: features.previousScore,
          internalMarks: features.internalMarks,
          assignmentCompletion: features.assignmentCompletion,
          studyHours: features.studyHours,
          participation: features.participation,
        },
        {
          headers: {
            'Content-Type': 'application/json',
            'X-Service-Secret': env.ML_SERVICE_SECRET,
          },
          timeout: 6000,
        }
      );
    } catch (error) {
      // Handle error
    }
  }
}

```

```
    }  
  );  
  
  return response.data;  
} catch (error: any) {  
  console.error('[MLClientService] Call failed to ML service at', targetUrl, error.message);  
  const customError: any = new Error(  
    'AI prediction service is currently unavailable. Please try again later.'  
  );  
  customError.statusCode = 503;  
  throw customError;  
}  
}  
  
/**  
 * Health check for ML Service  
 */  
async checkHealth(): Promise<{ status: string; modelVersion?: string; ready: boolean }> {  
  const targetUrl = new URL('health', this.getBaseUrl()).toString();  
  try {  
    const response = await axios.get(targetUrl, { timeout: 3000 });  
    return response.data;  
  } catch (error: any) {  
    return {  
      status: 'UNAVAILABLE',  
      ready: false,  
    };  
  }  
}  
}  
  
export const mlClientService = new MLClientService();
```

A.3 Machine Learning Source Code

A.3.1 Data Preprocessing & Synthetic Dataset Generation

Academic Dataset Generator

File: apps/ml-service/training/generate_dataset.py

PYTHON

Purpose & Architecture: Generates realistic synthetic academic datasets modeling educational statistical distributions and inter-feature correlations for neural network training.

```
import numpy as np
import pandas as pd
import os

def generate_synthetic_academic_dataset(num_samples: int = 2000, seed: int = 42, output_path: str = "data/student_performance_data.csv"):
    np.random.seed(seed)
    os.makedirs(os.path.dirname(output_path), exist_ok=True)

    # 1. Attendance (0 - 100%) - slightly skewed towards higher attendance
    attendance = np.random.beta(a=5, b=2, size=num_samples) * 100
    attendance = np.clip(attendance, 15.0, 100.0)

    # 2. Previous Academic Score (0 - 100) - normal distribution
    previous_score = np.random.normal(loc=68.0, scale=14.0, size=num_samples)
    previous_score = np.clip(previous_score, 20.0, 98.0)

    # 3. Weekly Self-Study Hours (0 - 40 hrs) - log-normal
    study_hours = np.random.lognormal(mean=2.1, sigma=0.45, size=num_samples)
    study_hours = np.clip(study_hours, 1.0, 36.0)

    # 4. Assignment Completion Rate (0 - 100%) - correlated with attendance & study hours
    assignment_latent = 0.5 * (attendance / 100.0) + 0.3 * (study_hours / 30.0) + 0.2 * np.random.uniform(0.3, 1.0, size=num_samples)
    assignment_completion = np.clip(assignment_latent * 100.0 + np.random.normal(0, 5, num_samples), 10.0, 100.0)

    # 5. Internal Marks (0 - 100) - correlated with past score, attendance, and assignments
    internal_latent = 0.4 * previous_score + 0.3 * attendance + 0.2 * assignment_completion + 0.1 * (study_hours * 2.5)
    internal_marks = np.clip(internal_latent + np.random.normal(0, 4.5, num_samples), 15.0, 98.0)

    # 6. Participation / Engagement (0 - 100%)
    participation_latent = 0.6 * (attendance / 100.0) + 0.4 * np.random.uniform(0.2, 1.0, size=num_samples)
    participation = np.clip(participation_latent * 100.0 + np.random.normal(0, 6, num_samples), 10.0, 100.0)

    # Target: Final Examination Score (0 - 100)
    # Realistic educational model:
    # 25% Internal Continuous Assessment
    # 22% Previous Foundational Score
    # 20% Lecture/Lab Attendance
    # 15% Assignment Mastery
    # 12% Independent Study Hours (scaled)
    # 6% Active Classroom Participation
    # + Realistic Exam Variance (-4 to +4)
    raw_final = (
        0.26 * internal_marks +
        0.23 * previous_score +
        0.20 * attendance +
        0.15 * assignment_completion +
        0.10 * np.minimum(study_hours * 3.0, 100.0) +
        0.06 * participation +
        np.random.normal(0, 3.5, size=num_samples)
    )
    final_score = np.clip(raw_final, 10.0, 99.5)

    df = pd.DataFrame({
        "attendance": np.round(attendance, 1),
        "previousScore": np.round(previous_score, 1),
        "internalMarks": np.round(internal_marks, 1),
        "assignmentCompletion": np.round(assignment_completion, 1),
        "studyHours": np.round(study_hours, 1),
    })
```

```

        "participation": np.round(participation, 1),
        "finalScore": np.round(final_score, 1)
    })

    df.to_csv(output_path, index=False)
    print(f"[Dataset Generator] Generated {num_samples} academic samples at {output_path}")
    print(df.describe().round(2))
    return df

if __name__ == "__main__":
    generate_synthetic_academic_dataset()

```

A.3.2 ANN Model Architecture, A.3.3 Training & A.3.4 Evaluation

End-to-End ANN Training & Evaluation Pipeline

[PYTHON](#)

File: apps/ml-service/training/train.py

Purpose & Architecture: Defines the 4-layer Artificial Neural Network architecture (64-32-16-1) in Keras, applies StandardScaler normalization, trains with early stopping and learning rate reduction, evaluates performance metrics (MAE, RMSE, R^2), and serializes trained artifacts.

```

import os
import sys
import json
import argparse
import datetime
import numpy as np
import pandas as pd
import joblib
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# Set Keras/TensorFlow logging
os.environ["TF_CPP_MIN_LOG_LEVEL"] = "2"
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, callbacks

FEATURE_COLUMNS = [
    "attendance",
    "previousScore",
    "internalMarks",
    "assignmentCompletion",
    "studyHours",
    "participation",
]
TARGET_COLUMN = "finalScore"
MODEL_VERSION = "v1.0.0"

def train_model(data_path: str = "data/student_performance_data.csv", output_dir: str = "models", epochs: int = 80, seed: int = 42):
    print("=====")
    print(" EduGuard AI - Deep Learning Model Training Pipeline")
    print(f" Target Model Version: {MODEL_VERSION}")
    print("=====")

    # Set reproducible seeds
    np.random.seed(seed)
    tf.random.set_seed(seed)

    # Ensure output directory exists
    os.makedirs(output_dir, exist_ok=True)

    # 1. Load Dataset
    if not os.path.exists(data_path):
        print(f"Dataset not found at {data_path}. Generating synthetic academic dataset...")
        from generate_dataset import generate_synthetic_academic_dataset
        generate_synthetic_academic_dataset(output_path=data_path, seed=seed)

    print(f"[Pipeline] Loading dataset from: {data_path}")
    df = pd.read_csv(data_path)

```

```

# 2. Validate columns
required_cols = FEATURE_COLUMNS + [TARGET_COLUMN]
for col in required_cols:
    if col not in df.columns:
        raise ValueError(f"Required column '{col}' missing from dataset. Found: {list(df.columns)}")

# 3. Handle missing values & validate bounds
initial_count = len(df)
df = df.dropna(subset=required_cols)
# Filter valid academic ranges
df = df[
    (df["attendance"] >= 0) & (df["attendance"] <= 100) &
    (df["previousScore"] >= 0) & (df["previousScore"] <= 100) &
    (df["internalMarks"] >= 0) & (df["internalMarks"] <= 100) &
    (df["assignmentCompletion"] >= 0) & (df["assignmentCompletion"] <= 100) &
    (df["studyHours"] >= 0) & (df["studyHours"] <= 100) &
    (df["participation"] >= 0) & (df["participation"] <= 100) &
    (df["finalScore"] >= 0) & (df["finalScore"] <= 100)
]
print(f"[Pipeline] Validated {len(df)} / {initial_count} records (dropped {initial_count - len(df)} invalid/missing).")

X = df[FEATURE_COLUMNS].values
y = df[TARGET_COLUMN].values

# 4. Train / Test Split (80 / 20)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=seed
)
print(f"[Pipeline] Training samples: {len(X_train)}, Testing samples: {len(X_test)}")

# 5. Feature Normalization
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 6. Build Artificial Neural Network (ANN) Architecture
model = keras.Sequential([
    layers.Input(shape=(len(FEATURE_COLUMNS),)),
    layers.Dense(64, activation="relu", name="dense_1"),
    layers.BatchNormalization(name="bn_1"),
    layers.Dropout(0.2, name="dropout_1"),
    layers.Dense(32, activation="relu", name="dense_2"),
    layers.Dropout(0.1, name="dropout_2"),
    layers.Dense(16, activation="relu", name="dense_3"),
    layers.Dense(1, activation="linear", name="output_regression")
])

optimizer = keras.optimizers.Adam(learning_rate=0.001)
model.compile(
    optimizer=optimizer,
    loss="mean_squared_error",
    metrics=["mae"]
)

model.summary()

# 7. Model Training with Early Stopping
early_stop = callbacks.EarlyStopping(
    monitor="val_loss",
    patience=12,
    restore_best_weights=True,
    verbose=1
)

history = model.fit(
    X_train_scaled,
    y_train,
    validation_split=0.2,
    epochs=epochs,
    batch_size=32,
    callbacks=[early_stop],
    verbose=1
)

# 8. Evaluation on Unseen Test Set
y_pred = model.predict(X_test_scaled).flatten()
mae = float(mean_absolute_error(y_test, y_pred))
mse = float(mean_squared_error(y_test, y_pred))
rmse = float(np.sqrt(mse))

```

```

r2 = float(r2_score(y_test, y_pred))

print("\n-----")
print(" Model Evaluation Results on Test Dataset:")
print(f" Mean Absolute Error (MAE): {mae:.3f} points")
print(f" Root Mean Squared Error (RMSE): {rmse:.3f} points")
print(f" Coefficient of Determination (R²): {r2:.4f}")
print("-----\n")

# 9. Save Model, Scaler & Metadata Artifacts
model_path = os.path.join(output_dir, "ann_model.keras")
scaler_path = os.path.join(output_dir, "scaler.joblib")
metadata_path = os.path.join(output_dir, "model_metadata.json")

model.save(model_path)
joblib.dump(scaler, scaler_path)

metadata = {
    "version": MODEL_VERSION,
    "createdAt": datetime.datetime.utcnow().isoformat() + "Z",
    "features": FEATURE_COLUMNS,
    "target": TARGET_COLUMN,
    "datasetIdentifier": os.path.basename(data_path),
    "datasetSize": len(df),
    "testSplit": 0.2,
    "metrics": {
        "mae": round(mae, 3),
        "rmse": round(rmse, 3),
        "r2": round(r2, 4)
    },
    "hyperparameters": {
        "layers": [64, 32, 16, 1],
        "activations": ["relu", "relu", "relu", "linear"],
        "dropout": [0.2, 0.1],
        "optimizer": "Adam",
        "learningRate": 0.001,
        "loss": "mean_squared_error",
        "epochsTrained": len(history.history["loss"])
    },
    "frameworkVersion": f"TensorFlow {tf.__version__} / Keras {keras.__version__}",
    "status": "ACTIVE"
}

with open(metadata_path, "w") as f:
    json.dump(metadata, f, indent=2)

print(f"[Pipeline] Successfully saved artifacts:")
print(f" - Model: {model_path}")
print(f" - Scaler: {scaler_path}")
print(f" - Metadata: {metadata_path}")

# 10. Automatically export lightweight NumPy deployment artifacts
try:
    from export_model import export_model_artifacts
except ImportError:
    from training.export_model import export_model_artifacts
export_model_artifacts(model_dir=output_dir)

print("=====")

return metadata

if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Train EduGuard ANN Model")
    parser.add_argument("--data-path", default="data/student_performance_data.csv", help="Path to input dataset CSV")
    parser.add_argument("--output-dir", default="models", help="Output directory for artifacts")
    parser.add_argument("--epochs", type=int, default=80, help="Max training epochs")
    parser.add_argument("--seed", type=int, default=42, help="Random seed")
    args = parser.parse_args()

    train_model(
        data_path=args.data_path,
        output_dir=args.output_dir,
        epochs=args.epochs,
        seed=args.seed
    )

```

A.3.5 Inference / Predictor Engine

Production Model Predictor Service

PYTHON

File: apps/ml-service/inference/predictor.py

Purpose & Architecture: Loads serialized Keras model and Scikit-Learn scaler into memory, validates input feature dictionaries, applies standard scaling, and executes regression inference.

```
import os
import json
import numpy as np

class StudentPredictor:
    """
    High-Performance, Zero-TensorFlow Student Performance Predictor.

    Executes the trained 4-layer Artificial Neural Network (ANN) regression model
    (64 -> 32 -> 16 -> 1) using vectorized NumPy matrix multiplications and exact
    BatchNormalization forward-pass equations.

    This eliminates all heavy ML framework dependencies (TensorFlow, PyTorch, SciPy,
    Scikit-Learn, Pandas) from the production serverless function runtime, reducing
    the bundle size by >90% to fit comfortably within Vercel's 500 MB limit.
    """

    FEATURE_ORDER = [
        "attendance",
        "previousScore",
        "internalMarks",
        "assignmentCompletion",
        "studyHours",
        "participation",
    ]

    def __init__(self, model_dir: str = "models"):
        if model_dir == "models":
            abs_models = os.path.abspath(os.path.join(os.path.dirname(__file__), "..", "models"))
            self.model_dir = abs_models if os.path.exists(abs_models) else "models"
        else:
            self.model_dir = model_dir

        self.model_version = "v1.0.0"
        self.metadata = None

        # Lightweight NumPy weights
        self.W1 = None
        self.b1 = None
        self.bn_gamma = None
        self.bn_beta = None
        self.bn_moving_mean = None
        self.bn_moving_variance = None
        self.bn_epsilon = 1e-3
        self.W2 = None
        self.b2 = None
        self.W3 = None
        self.b3 = None
        self.W_out = None
        self.b_out = None

        # Preprocessing Scaler
        self.scaler_mean = None
        self.scaler_scale = None

        self._load_artifacts()

    def _load_artifacts(self):
        npz_path = os.path.join(self.model_dir, "ann_model_weights.npz")
        scaler_json_path = os.path.join(self.model_dir, "scaler_params.json")
        metadata_path = os.path.join(self.model_dir, "model_metadata.json")

        # 1. Load Metadata
        if os.path.exists(metadata_path):
            try:
                with open(metadata_path, "r", encoding="utf-8") as f:
                    self.metadata = json.load(f)
```

```

        self.model_version = self.metadata.get("version", "v1.0.0")
    except Exception as e:
        print(f"[Predictor] Warning reading metadata: {e}")

# 2. Load Weights from Compressed NumPy Archive
if os.path.exists(npz_path):
    print(f"[Predictor] Loading lightweight NumPy weights from: {npz_path}")
    data = np.load(npz_path)
    self.W1 = data["W1"].astype(np.float32)
    self.b1 = data["b1"].astype(np.float32)

    self.bn_gamma = data["bn_gamma"].astype(np.float32)
    self.bn_beta = data["bn_beta"].astype(np.float32)
    self.bn_moving_mean = data["bn_moving_mean"].astype(np.float32)
    self.bn_moving_variance = data["bn_moving_variance"].astype(np.float32)
    self.bn_epsilon = float(data["bn_epsilon"]) if "bn_epsilon" in data else 1e-3

    self.W2 = data["W2"].astype(np.float32)
    self.b2 = data["b2"].astype(np.float32)

    self.W3 = data["W3"].astype(np.float32)
    self.b3 = data["b3"].astype(np.float32)

    self.W_out = data["W_out"].astype(np.float32)
    self.b_out = data["b_out"].astype(np.float32)

    if "scaler_mean" in data and "scaler_scale" in data:
        self.scaler_mean = data["scaler_mean"].astype(np.float32)
        self.scaler_scale = data["scaler_scale"].astype(np.float32)

    print(f"[Predictor] Model loaded successfully (NumPy Engine). Version: {self.model_version}")

# 3. Fallback: Load Scaler from JSON if not in NPZ
if self.scaler_mean is None and os.path.exists scaler_json_path):
    with open(scaler_json_path, "r", encoding="utf-8") as f:
        sdata = json.load(f)
        self.scaler_mean = np.array(sdata["mean"], dtype=np.float32)
        self.scaler_scale = np.array(sdata["scale"], dtype=np.float32)
    print(f"[Predictor] Loaded scaler parameters from: {scaler_json_path}")

# 4. Optional local dev fallback if Keras model exists and TensorFlow is installed
if self.W1 is None:
    keras_path = os.path.join(self.model_dir, "ann_model.keras")
    if os.path.exists(keras_path):
        try:
            os.environ["TF_CPP_MIN_LOG_LEVEL"] = "2"
            from tensorflow import keras
            print(f"[Predictor] Falling back to Keras model: {keras_path}")
            keras_model = keras.models.load_model(keras_path)
            self.W1, self.b1 = keras_model.get_layer("dense_1").get_weights()
            bn = keras_model.get_layer("bn_1")
            self.bn_gamma, self.bn_beta, self.bn_moving_mean, self.bn_moving_variance = bn.get_weights()
            self.bn_epsilon = float(bn.epsilon)
            self.W2, self.b2 = keras_model.get_layer("dense_2").get_weights()
            self.W3, self.b3 = keras_model.get_layer("dense_3").get_weights()
            self.W_out, self.b_out = keras_model.get_layer("output_regression").get_weights()
            print("[Predictor] Successfully extracted weights from Keras model in memory.")
        except ImportError:
            print("[Predictor] WARNING: NumPy weights not found and TensorFlow is not installed.")

def is_ready(self) -> bool:
    weights_loaded = (
        self.W1 is not None and
        self.b1 is not None and
        self.bn_gamma is not None and
        self.W2 is not None and
        self.W3 is not None and
        self.W_out is not None
    )
    scaler_loaded = (self.scaler_mean is not None and self.scaler_scale is not None)
    return weights_loaded and scaler_loaded

def forward(self, x: np.ndarray) -> np.ndarray:
    """
    Pure NumPy forward pass reproducing exact Keras sequential model:
    Dense(64, relu) -> BatchNorm -> Dense(32, relu) -> Dense(16, relu) -> Dense(1, linear)
    """
    # Dense 1 + ReLU
    z1 = np.dot(x, self.W1) + self.b1

```

```

a1 = np.maximum(0.0, z1)

# Batch Normalization (inference mode: uses moving mean & moving variance)
a1_bn = self.bn_gamma * (a1 - self.bn_moving_mean) / np.sqrt(self.bn_moving_variance + self.bn_epsilon) + self.bn_beta

# Dense 2 + ReLU (Dropout is identity during inference)
z2 = np.dot(a1_bn, self.W2) + self.b2
a2 = np.maximum(0.0, z2)

# Dense 3 + ReLU (Dropout is identity during inference)
z3 = np.dot(a2, self.W3) + self.b3
a3 = np.maximum(0.0, z3)

# Output Regression (Linear)
out = np.dot(a3, self.W_out) + self.b_out
return out

def predict(self, features: dict) -> tuple[float, str]:
    if not self.is_ready():
        raise RuntimeError("Model artifacts are not loaded. Ensure ann_model_weights.npz exists.")

    # Extract features in strict canonical order
    raw_values = []
    for feat in self.FEATURE_ORDER:
        if feat not in features:
            raise ValueError(f"Missing required feature: '{feat}'")
        val = float(features[feat])
        raw_values.append(val)

    input_array = np.array([raw_values], dtype=np.float32)

    # StandardScaler transformation: (x - mean) / scale
    scaled_input = (input_array - self.scaler_mean) / self.scaler_scale

    # ANN Inference via NumPy forward pass
    prediction = self.forward(scaled_input)
    raw_score = float(prediction[0][0])
    predicted_score = round(max(0.0, min(100.0, raw_score)), 2)

    return predicted_score, self.model_version

# Singleton predictor instance
predictor = StudentPredictor()

```

A.3.6 FastAPI ML Microservice Application

FastAPI Microservice Application & Endpoints

PYTHON

File: apps/ml-service/main.py

Purpose & Architecture: High-performance Python FastAPI service providing token-secured REST endpoints: GET /health for system readiness and POST /predict for low-latency inference.

```

import time
import os
from fastapi import FastAPI, HTTPException, Header, status
from pydantic import BaseModel, Field
from inference.predictor import predictor

app = FastAPI(
    title="EduGuard AI - ML Prediction Microservice",
    description="Dedicated Deep Learning ANN Inference Microservice for Student Academic Performance Prediction",
    version="1.0.0",
)

EXPECTED_SECRET = os.getenv("ML_SERVICE_SECRET", "<YOUR_ML_SERVICE_SECRET>")

class PredictionRequest(BaseModel):
    attendance: float = Field(..., ge=0.0, le=100.0, description="Attendance percentage (0-100)")
    previousScore: float = Field(..., ge=0.0, le=100.0, description="Previous academic score (0-100)")
    internalMarks: float = Field(..., ge=0.0, le=100.0, description="Internal assessment marks (0-100)")
    assignmentCompletion: float = Field(..., ge=0.0, le=100.0, description="Assignment completion percentage (0-100)")
    studyHours: float = Field(..., ge=0.0, le=100.0, description="Self-study hours per week (0-100)")
    participation: float = Field(..., ge=0.0, le=100.0, description="Classroom participation score (0-100)")

```

```
class PredictionResponse(BaseModel):
    predictedScore: float
    modelVersion: str
    latencyMs: float

class HealthResponse(BaseModel):
    status: str
    service: str
    modelReady: bool
    modelVersion: str
    timestamp: float

@app.get("/health", response_model=HealthResponse)
def health():
    return HealthResponse(
        status="HEALTHY" if predictor.is_ready() else "DEGRADED",
        service="EduGuard AI ML Microservice",
        modelReady=predictor.is_ready(),
        modelVersion=predictor.model_version,
        timestamp=time.time(),
    )

@app.post("/predict", response_model=PredictionResponse)
def predict(payload: PredictionRequest, x_service_secret: str | None = Header(default=None)):
    # Verify internal secret if provided
    if x_service_secret and x_service_secret != EXPECTED_SECRET:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Invalid service secret for internal ML microservice",
        )

    if not predictor.is_ready():
        raise HTTPException(
            status_code=status.HTTP_503_SERVICE_UNAVAILABLE,
            detail="ANN model is not loaded or ready. Training must be run.",
        )

    start_time = time.perf_counter()
    try:
        predicted_score, model_version = predictor.predict(payload.model_dump())
        latency_ms = round((time.perf_counter() - start_time) * 1000, 2)

        return PredictionResponse(
            predictedScore=predicted_score,
            modelVersion=model_version,
            latencyMs=latency_ms,
        )
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
```

A.4 API Endpoint Summary

Comprehensive catalog of all REST endpoints implemented across the EduGuard AI Express backend and FastAPI ML microservices:

SERVICE	METHOD	ENDPOINT PATH	PURPOSE	ACTUAL SOURCE FILE
Express API	POST	/api/auth/login	User credential authentication & JWT token issuance	apps/api/src/controllers/authController.ts
Express API	POST	/api/auth/logout	Session revocation and cookie clearing	apps/api/src/controllers/authController.ts
Express API	GET	/api/auth/me	Retrieve authenticated profile and role permissions	apps/api/src/controllers/authController.ts
Express API	GET	/api/health	Backend system and ML microservice connectivity status	apps/api/src/app.ts
Express API	GET	/api/dashboard/summary	College-wide summary metrics and priority attention queue	apps/api/src/controllers/dashboardController.ts
Express API	GET	/api/students	Filtered student cohort directory with pagination	apps/api/src/controllers/studentController.ts
Express API	GET	/api/students/me	Logged-in student personal academic record and standing	apps/api/src/controllers/studentController.ts
Express API	GET	/api/students/:id	Detailed student profile with multi-year academic history	apps/api/src/controllers/studentController.ts
Express API	GET	/api/students/:id/metrics	Calculated 6 behavioral AI metric values breakdown	apps/api/src/controllers/studentController.ts
Express API	POST	/api/students	Register a new student record (Admin / Teacher)	apps/api/src/controllers/studentController.ts
Express API	PUT	/api/students/:id	Update student profile information and cohort assignment	apps/api/src/controllers/studentController.ts
Express API	GET	/api/attendance	Query historical attendance records with date/subject filters	apps/api/src/controllers/attendanceController.ts
Express API	GET	/api/attendance/sheet	Load classroom session roster for cohort date and subject	apps/api/src/controllers/attendanceController.ts

SERVICE	METHOD	ENDPOINT PATH	PURPOSE	ACTUAL SOURCE FILE
Express API	POST	/api/attendance/batch	Bulk record classroom lecture attendance session	apps/api/src/controllers/attendanceController.ts
Express API	GET	/api/academic-records	Fetch student marks, exam grades and semester coursework	apps/api/src/controllers/academicRecordController.ts
Express API	POST	/api/academic-records	Upsert student internal marks and final exam grades	apps/api/src/controllers/academicRecordController.ts
Express API	GET	/api/predictions	Query past ANN prediction audits and risk histories	apps/api/src/controllers/predictionController.ts
Express API	POST	/api/predictions	Trigger ANN inference and generate early warning alert	apps/api/src/controllers/predictionController.ts
Express API	GET	/api/interventions	Query academic counseling and remedial action plans	apps/api/src/controllers/interventionController.ts
Express API	POST	/api/interventions	Create pedagogical action plan for at-risk student	apps/api/src/controllers/interventionController.ts
Express API	PUT	/api/interventions/:id	Update intervention progress status and counseling notes	apps/api/src/controllers/interventionController.ts
Express API	GET	/api/programs	List registered academic degree programs (e.g. B.Tech, BBA, BSC)	apps/api/src/controllers/programController.ts
Express API	GET	/api/departments	List institutional departments, codes and mappings	apps/api/src/controllers/departmentController.ts
Express API	GET	/api/subjects	List subjects mapped to program, department and semester	apps/api/src/controllers/subjectController.ts
Express API	GET	/api/classes	Fetch academic cohort sections and student counts	apps/api/src/controllers/classController.ts
FastAPI ML	GET	/health	ANN model readiness check and version metadata	apps/ml-service/main.py
FastAPI ML	POST	/predict	Deep learning neural network score inference	apps/ml-service/main.py